N-DHT

Architecture, Implementation, and Deployment

David A. Smith
YZ.social

Version 0.3.50 Simulator v0.70.22 2026-05-10

The technology is shaped by the mission.

Contents

Part I

Foundations

Introduction

The technology is shaped by the mission.

The Mission

THIS DOCUMENT is the engineering reference for N-DHT, a learning-adaptive distributed hash table. The protocol is designed for a specific job: to carry the routing and publish/subscribe workload of a privacy-first decentralized internet, on consumer devices, in real browsers, against the actual physics of the network it runs on. Everything else in this volume — the architecture, the algorithms, the simulator, the red team, the deployment path, the application analysis — is shaped by that job.

The job is hard for an unfamiliar reason. The internet of 2026 has no shortage of clever distributed systems, but almost all of them are *distributed underneath, central on top*: a constellation of servers that the user reaches through a single corporate gateway.[1] A real peer-to-peer internet — one where two strangers can find each other, exchange a message, and route around an outage without asking permission of any intermediary — requires a different kind of substrate. The substrate must be *federation-free at every layer*: addressing, routing, discovery, group communication, eventual delivery under churn.

[1] The pattern is so common we have stopped noticing it. CDNs, federated identity providers, cloud auth, app stores, the "serverless" frameworks that run on three or four hyperscaler clouds — all examples of distribution-as-implementation-detail sitting under a single point of decision.

A distributed hash table is the addressing layer of that substrate. Without one, every other peer-to-peer primitive collapses back into a centralized service: there is no way to find the user you want to talk to, no way to discover the file you want to download, no way to subscribe to the channel you want to follow, without somebody telling you where it lives. With one, those operations are mechanical: the network itself routes you to the right peer.

The mission is to build the addressing layer that a real peer-to-peer internet needs. The protocol described here is what that addressing layer looks like.

What a DHT Is, in One Page

A DISTRIBUTED HASH TABLE is the machinery that lets a swarm of independent computers collectively store and find data without a central directory. Every peer is assigned an identifier from a large keyspace. Every piece of content (a file, a topic name, a public key) is also mapped into the same keyspace through a hash function. To find a piece of content, a peer asks the closest peer it knows, who in turn asks a peer closer still — repeat until the responsible peer is reached. The trick is making "closer" mean something the network can compute locally, with no global view.

DHTs are the addressing layer behind BitTorrent's trackerless swarms, IPFS, parts of Ethereum's peer discovery, and the hidden services on Tor. Every node gets a random ID; every piece of content gets an ID; lookups route from the source ID to whoever holds the destination ID, hop by hop, with no authoritative directory in the middle.

The dominant design — Kademlia, 2002 — defines closeness as XOR distance in a 160-bit ID space.[2] Each lookup hop at least halves the remaining XOR distance, so any target is reachable in $\log_2 N$ steps. With a million-node network that is twenty hops, every time. The math is clean, the algorithm is provably correct, and Kademlia is in production all over the internet, twenty-four years after publication.

[2] Maymounkov & Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric." IPTPS 2002. The XOR metric is symmetric and has a useful recursive property: a peer that knows one peer in each XOR-bucket covering the space can route to anyone in the network.

There is just one problem.

The Problem with the Status Quo

TWENTY HOPS sounds fast. But each hop is a message between two computers somewhere on Earth, and Kademlia node IDs are random. Two random points on the planet's surface are, on average, about half the planet apart. A round trip between them is roughly 100 ms. Twenty hops at 100 ms per hop is two seconds. Every lookup. Every time.

The actual one-way median latency between random pairs of internet nodes was about 42 ms in 2004 and is in the same neighborhood today (Dabek et al., NSDI 2004, §5.1). Round trip is roughly 2δ.

For real-time applications — voice, video, multiplayer, live messaging, push notifications, anything a user expects to feel instant — two seconds is unusable. The math says routing is logarithmic, which is fast. The physics says each step is slow.

That tension is the latency tax, and Kademlia pays it in full. There is a deeper structural cost too: Kademlia treats a lookup to the next city the same as a lookup to the opposite hemisphere. The XOR metric is geographically blind. A message destined for a node twenty kilometers away may bounce through Tokyo, São Paulo, and Helsinki before arriving. Real-world workloads are dominated by local traffic[3] — friends in the same city, regional content caches, same-organization peers — and every one of those local lookups pays the global tax.

[3] A messaging app's neighbor-to-neighbor chats, content-sharing services' regional replication, multiplayer games' lobby discovery — the vast majority of lookups are between peers in the same city, the same country, or at worst the same continent. Long-haul traffic is the minority, but Kademlia weights it equally.

That is the pothole the next-generation DHT has to fill.

What This Protocol Adds

THE PROTOCOL described in this volume sits on top of Kademlia's correctness guarantee — specifically, the property that a peer with one well-chosen entry per XOR bucket can reach any target in the network — and makes three substantive changes to how routing tables are populated and which peer to ask first.

1. Geographic identity. The first eight bits of every node ID encode the node's S2 cell prefix — a geographic fingerprint.[4] XOR distance now approximately tracks physical distance. The 20-hop world tour collapses into a 4–6-hop journey around the neighborhood, where each hop costs single-digit milliseconds rather than 100. We call this layer *G-DHT*, discussed at length in Chapter I.

[4] S2 is Google's spherical-geometry library, which divides Earth's surface into cells along a Hilbert curve. Hilbert curves are space-filling: consecutive cells on the curve remain neighbors in space. Two cells whose IDs share a prefix are geographically close. §II describes the construction in detail.

2. Adaptive routing tables. Each peer maintains a *synaptome*: a set of weighted connections that strengthens on successful use and decays through disuse. Routing tables are no longer populated once and lazily repaired — they are *learned*. Successful lookups deposit shortcuts at every intermediate node along the path; observed transit pairs introduce themselves directly; geographic neighbors propagate the news. The network's structure is shaped by the traffic it carries, not by rules an engineer guessed at. The biology is direct, not metaphorical: the synaptome is the digital cousin of the synaptic matrix in a biological brain, and the rule that governs which connections persist is the same rule (long-term potentiation) that your brain uses to consolidate memory.

3. In-network publish/subscribe. The protocol delivers a genuine pub/sub primitive — a publisher sends to a topic, every subscriber receives. The implementation is an *axonal tree*: a delivery structure rooted at a publisher-prefixed topic ID, grown by routed subscribe messages, healed continuously by the same routed re-subscribe path that built it. Under 5% per-cycle node churn the tree delivers 100% of messages with zero special machinery.

The combined effect is a routing layer that hits within a small constant of the analytical 3δ latency floor (Dabek et al., NSDI 2004) on global lookups, that converges to single-hop performance on regional traffic, that delivers pub/sub at the robustness ceiling against churn, and that distributes its bandwidth load broadly across the population — with no node saturating disproportionately at the scales we have been able to measure. Chapter V reports the numbers; the rest of the document explains how they are obtained, tested, and what they do and do not justify.

What This Document Is

This is the engineering reference for the protocol — the document from which a working production system will be built. That puts a high bar on the contents. Three commitments shape what follows.

Completeness. Every algorithm, every parameter, every constant, every default, every test, every measurement, every deferred concern. If a future implementer needs to know it to build the system, it lives here. The current document is approximately *[final page count]* pages, with vector figures throughout and no length limit imposed on clarity.

Honesty. The simulator that generates every quantitative claim in this volume has limits. Its omissions are catalogued explicitly[5] — there is no claim made on the basis of an unmodeled behavior, and no measurement reported without surfacing the mechanism that generated it. The simulator is part of the design discipline (§IV), not a tool we used and put away.

[5] See Chapter IV for what the simulator does and does not model, and Chapter VI for the third-party adversarial review of the protocol's deployment risks.

Lineage. A protocol design without a clear genealogy is either a copy or a bluff. Every mechanism in N-DHT has an ancestor in twenty-five years of distributed-systems literature (Chapter I), and most have an ancestor in a hundred years of neuroscience (Chapter II). Where we have followed prior art, we cite it. Where we have departed, we explain why and what we tried that did not work (Appendix VIII).

Audience and How to Read It

The Tufte design is deliberate. The body of every chapter is written for a reader who wants to understand the system without first having read three textbooks — a high-school student with a strong math background, an engineer in an adjacent field, a journalist, a policymaker. The margins carry the technical precision that makes the body's claims defensible: definitions, units, citations, the technical asides that would otherwise interrupt the prose. The reader chooses how deep to go.

This is one. Margin notes carry definitions, units, citations, and the technical asides that would otherwise interrupt a sentence. Reading them is optional; ignoring them costs the reader nothing in continuity.

The chapters are organized so that successive parts assume the previous parts. *If you only have an hour:* read Chapter I (this one), II (the five operations), and V (the headline performance numbers). *If you are an engineer planning a deployment:* add Part III (the NH-1 implementation), Part IV (the simulator), and Part VI (deployment and hardening). *If you are evaluating the protocol against an alternative:* read Chapter I (the DHT lineage) and Chapter V (the bandwidth concentration analysis). *If you are building an application on top of*

N-DHT: read Part VII (existing applications: a deep dive). *If you want to understand why every design decision is the way it is:* read the whole document.

Scope of the Document

What this document covers: the routing layer (Chapters II–II), the pub/sub layer (Chapter II), the consolidated NH-1 implementation of both (Chapters III–III), the simulator that we use to test them (Chapters IV–IV), measured performance against three comparison protocols (Chapters V–V), the deployment pathway and operational requirements (Chapters VI–VI), and a deep analysis of how existing applications would change if they migrated to this routing substrate (Chapter VII).

What it does not cover: forward-looking applications enabled by performance characteristics that this routing layer makes possible but that depend on additional design work (streaming-class media, real-time multiplayer, federated AI training, V2V coordination). Those analyses are deferred to a separate volume because each requires substantial protocol-specific groundwork on top of the routing layer described here. We do not make claims about them in this document. Treating them as future volumes also lets the present document remain what it is: the engineering reference for the routing and pub/sub substrate, and nothing else.

Summary of Contributions

The protocol contributes the following:

1. **A unified vitality model** for routing-table maintenance that consolidates eighteen rules from prior generations (NX-1 through NX-17) into twelve, governed by a single equation: vitality = weight × recency. This is described in Chapters II and III.

2. **Five-operation organizational frame** (NAVIGATE, LEARN, FORGET, EXPLORE, STRUCTURE) under which every routing rule is classified, making the rule set comprehensible and ablation-amenable rather than an inscrutable tangle. Chapter II.

3. **Performance within a small constant of the analytical 3δ floor** (Dabek et al., NSDI 2004) at every tested network size from 5,000 to 50,000 nodes, and *decreasing* latency on regional workloads as the network scales — the opposite of Kademlia's behavior. Chapter V.

4. **An axonal pub/sub primitive** delivering 100% of messages at baseline and recovering to 100% within three refresh intervals after 5% instantaneous churn, without K-closest replication, gossip, parent tracking, or any failure-detection machinery beyond the routed re-subscribe that built the tree. Chapter II.

5. **Empirical resolution of the bandwidth-concentration question** that has historically dogged adaptive routing protocols. At every tested scale, N-DHT distributes load broadly (Gini ∼0.55–0.69) while Kademlia and G-DHT concentrate it catastrophically (Gini ∼0.87–0.94, with 56–62 nodes processing more than 100× the network mean at 50,000 nodes). The volume cost of N-DHT's load distribution is a single tunable parameter (`annealRateScale`) with a clean knee. Chapter V.

6. **An open-source simulator** of approximately *[final line count]* lines of JavaScript, instrumented at every routing chokepoint, capable of reproducing every measurement in this volume from a clean checkout in under *[final runtime]*.[6]

7. **A red-team analysis** that catalogues thirteen deployment-relevant gaps between the simulator's environment and the production internet, ranked by deploy-blocker status and time-to-fix, with the bandwidth-concentration concern empirically reclassified. Chapter VI.

[6] The simulator is described in Chapters IV–IV; the methodology is given in Chapter V; the source is available at the URL listed in the References.

A Note on Naming

Throughout this volume the family name *N-DHT* refers to the neuromorphic-DHT family of protocols in general: the architecture, the operations, the vitality model. When a chapter discusses a specific generation under detailed scrutiny — typically NH-1, occasionally NX-17 or earlier members of the lineage — we name the generation explicitly. G-DHT and K-DHT (Kademlia) appear when they serve as comparison protocols. The generation-specific chapters live in Part III; the rest of the body discusses the family as a whole.

The fact that the most recent generation, NH-1, fits the family-level description *most cleanly* is not coincidence: NH-1 is the generation in which the consolidation work converged. Earlier generations carried more bespoke machinery; future generations will inherit NH-1's structure and continue to refine it. We expect this naming convention to remain useful for several iterations.

The DHT Lineage

A protocol design without a clear genealogy is either a copy or a bluff.

THE NEUROMORPHIC DHT has two parents. One is twenty-five years of distributed-systems research — the line of work that runs from Plaxton's prefix routing in the late 1990s through Chord, CAN, Pastry, Tapestry, Kademlia, Coral, Vivaldi, and a dozen named mechanisms in between. The other is a hundred years of neuroscience, which we discuss in Chapter II. This chapter walks the first lineage. Every mechanism in N-DHT has an ancestor in the literature, and most have several. Where we have followed prior art, we cite it; where we have departed, we explain in terms of what came before what we did and what we tried that did not work.

The chapter is organized chronologically by paper publication. Each protocol gets the same five-element treatment: *the routing primitive*, *the problem the protocol solved*, *the problem it left open*, *what N-DHT inherits from it*, and *where N-DHT continues from it*. The chapter closes with a table that maps each protocol's mechanisms to the five operations of N-DHT (Chapter II).

Chord (2001)

CHORD[7] placed every node on a circular identifier space modulo 2^m and made distance asymmetric: *successor distance*. To find the node responsible for key k, a Chord node consults its *finger table* of m pointers — the i-th finger points to the successor of $n + 2^i$ — and forwards to whichever finger is closest to k without overshooting.

Each hop at least halves the remaining distance, so lookups complete in $O(\log N)$. The original paper proved correctness, churn resilience under stabilization, and load-balance properties under random hashing.

What it solved. The first widely-cited DHT to give a proof of $O(\log N)$ lookup in the average case, plus a maintenance protocol (periodic stabilization) that kept the ring connected under churn. Chord

[7] Stoica, Morris, Karger, Kaashoek, Balakrishnan, "Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications." SIGCOMM 2001.

made the field's central claim — that you could build a global lookup service with no servers — defensible.

What it left open. *Geographic blindness*: Chord IDs are random hashes, so the ring topology is unrelated to the physical network. *Maintenance overhead*: stabilization runs on a fixed schedule whether the network is changing or not (the failure mode that motivates Ghinita-Teo, below). *Pub/sub*: not part of the protocol. *Adaptation*: the finger table is structurally determined — i-th finger is 2^i ahead, full stop. There is no mechanism by which observed traffic shapes the routing structure.

The contemporaneous paper *Looking up data in P2P systems* (Balakrishnan, Kaashoek, Karger, Morris, Stoica, *CACM* 2003) framed the field. Chord, CAN, Pastry, Tapestry, and Kademlia all appeared within an eighteen-month window in 2001–2002, and the comparison literature treats them as a cohort.

What N-DHT inherits. The succession-style argument that a peer with one well-chosen entry per power-of-two distance bucket can route to anyone in the network. The XOR metric of Kademlia (and of N-DHT) is the symmetric variant of Chord's asymmetric successor distance.

Where N-DHT continues. The fixed finger table becomes a vitality-scored synaptome whose entries are chosen by traffic, not by structural mandate. The fixed-schedule stabilization becomes event-triggered repair (temperature reheat on dead-peer discovery, LTP reinforcement on successful traffic).

CAN (2001)

CAN[8] embedded peers in a d-dimensional Cartesian toroidal space. Each node owned a hyperrectangular zone; lookups were forwarded to the neighbor closest to the target along any one axis, and routing terminated in $O(d \cdot N^{1/d})$ hops. With $d = \log_2 N$ the protocol matches Chord's logarithmic bound.

[8] Ratnasamy, Francis, Handley, Karp, Schenker, "A Scalable Content-Addressable Network." SIGCOMM 2001.
A torus in this sense is a d-dimensional cube with opposite faces identified. The routing argument depends on the torus topology so that there are no edges to fall off; in practice d is small (4–10).

What it solved. A constructive argument that a DHT could be built on geometric intuition rather than ID-space arithmetic. CAN also gave the first clean account of *zone splitting* as a join procedure — a new node finds an overloaded neighbor and takes half its zone — which influenced every subsequent load-balance discussion.

What it left open. For practical d, the lookup hop count $O(N^{1/d})$ is much worse than Chord's $O(\log N)$; for $d = \log N$ the routing-table size exceeds Chord's. Geographic blindness again. No pub/sub. No adaptation.

What N-DHT inherits. The notion that locality of *any* kind in the routing structure produces measurable benefit. The G-DHT geographic prefix (Chapter II) is in spirit a Cartesian-style embedding, restricted to two dimensions on the surface of a sphere and stamped only into the high bits of a node's identifier.

Where N-DHT continues. CAN's geometry was static; N-DHT's

locality structure is reinforced by traffic and decayed by disuse. The S2 prefix is a starting condition, not a fixed truth.

Pastry (2001) and SCRIBE (2002)

PASTRY[9] took a different turn: *prefix routing*. Each node maintained three structures.

[9] Rowstron & Druschel, "Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems." Middleware 2001.

- A **routing table** R: $\lceil \log_{2^b} N \rceil$ rows, each row covering peers that share a prefix with this node up to that row's depth. At base $b = 4$ and $N = 10^6$, that is roughly seventy-five entries.
- A **leaf set** L: the sixteen to thirty-two *numerically-closest* peers, used for the final routing hop and for ring-wraparound continuity.
- A **neighborhood set** M: the sixteen to thirty-two *RTT-closest* peers — not used for routing, but kept fresh as a candidate pool for repairing R.

A lookup matches one more digit of the destination ID per hop; the final hop uses L to find the absolute closest peer.

The contribution Pastry is most cited for is *locality preserving join*: when a new node X joins through a nearby existing node A, each node along the path $A \to \text{root}(X)$ sends X its tables, and X *takes row n of its routing table from the n-th node along the route*. A triangulation argument preserves locality without exhaustive search.

SCRIBE[10], layered on Pastry, is the structural ancestor of N-DHT's axonal pub/sub. Subscribe routes toward a topic ID; *every node along the path records the subscriber*; when a publisher publishes, the message follows the *reverse paths of all the subscribes* to form a multicast tree. N-DHT's axonal trees use precisely the same construction (Chapter II).

[10] Castro, Druschel, Kermarrec, Rowstron, "SCRIBE: A large-scale and decentralized application-level multicast infrastructure." IEEE JSAC 2002.

What it solved. The three-tier routing state introduced the canonical separation between *routing* (the table R), *terminal hop* (the leaf set L), and *maintenance pool* (the neighborhood set M). The locality-preserving join made routing tables RTT-aware at startup.

What it left open. Locality is set once at join and lazily repaired thereafter; there is no mechanism by which traffic *evolves* the locality structure. The leaf set is fixed-size and ID-proximal; there is no notion of a peer earning its place through usage. SCRIBE multicast trees are fixed by routing topology; the only repair is re-subscribe, but it lands at the same parent every time.

What N-DHT inherits. Three things, deeply. **(1)** The three-tier routing state; in N-DHT the tiers collapse into a single vitality-scored

synaptome whose high-, mid-, and low-vitality entries play the leaf-set, routing-table, and neighborhood-set roles respectively. **(2)** The locality-preserving join, generalized to XOR strata (Chapter III). **(3)** SCRIBE's reverse-path multicast — this is exactly what an axonal tree is.

Where N-DHT continues. The leaf set's terminal-hop role is played by traffic-earned high-vitality peers, not the ID-numerically-closest. SCRIBE's multicast tree is fixed at construction; N-DHT's axonal tree adapts via incoming-promotion and re-subscribe rebinding (§II). Pastry's locality is static-after-bootstrap; N-DHT's locality evolves via LTP, hop caching, lateral spread, and triadic closure.

Tapestry and DOLR (2004)

TAPESTRY[11] was Pastry's near-twin from the Berkeley side of the field. Same prefix-routing primitive (base $\beta = 16$, $\log_{16} N$ hops); same general routing-table shape. Two contributions distinguished it.

[11] Zhao, Huang, Stribling, Rhea, Joseph, Kubiatowicz, "Tapestry: A Resilient Global-Scale Overlay for Service Deployment." IEEE JSAC 2004.

First, *at each routing-table slot, the entry stored is the closest node by RTT that matches the prefix at that level.* The routing table is built at node insertion via iterative nearest-neighbor search. This is a stronger commitment to locality than Pastry's neighborhood-set repair: locality is in the routing slots themselves, not just adjacent to them.

Second, the **DOLR** (Decentralized Object Location and Routing) API. Where a DHT's interface is `put(key, value)` / `get(key)`, DOLR offers `PublishObject(GUID, server)`, `RouteToObject(GUID)`, `RouteToNode(nodeID)`. When a server stores object O at GUID, it routes a *publish* message toward O's deterministic root; *every node along the publish path stores* $\langle GUID, server \rangle$ *as a soft-state location pointer.* Queries route toward the root and *intersect the publish path early.* The "tree" of replica locations emerges from the union of publish paths, not from explicit construction.

Tapestry also introduced **surrogate routing**: if a deterministic root is dead, traffic routes to the closest live ID in its place, transparently. The location pointers along the former root's path remain valid, and the network heals around the loss.

What it solved. Three of the four design pillars of N-DHT: locality (in the routing table), soft-state replication (via publish-path location pointers), and resilience (via surrogate routing and Patchwork-style background probes). The median Relative Delay Penalty in the wide-area was reported as ≈ 1.

Tapestry's signature metric: $RDP =$ (Tapestry route latency) $\div$ (direct IP latency). Median ≈ 1 in the wide area; 90th percentile $\sim$ 2–3 with location-pointer optimization. This is the metric N-DHT's 3δ-floor analysis is the descendant of.

What it left open. Like Pastry, locality was set at node insertion and not subsequently evolved. The publish-path location pointers were soft-state with periodic repair, but the repair schedule was static.

There was no learning over which routes actually carry traffic.

What N-DHT inherits. Three pillars. **(1)** Locality in the routing primitive — in N-DHT, AP scoring's latency penalty plays the same role Tapestry's RTT-closest-per-slot played, made dynamic. **(2)** Soft-state replication along the route — this is structurally identical to N-DHT's *hop caching* (§II): every intermediate node along a successful path remembers the destination. **(3)** Surrogate routing as the failure-mode handler — this is exactly N-DHT's *iterative fallback* (Chapter II), inherited through NX-4. Patchwork-style probing is the family of mechanism that N-DHT's temperature reheat and dead-peer eviction belong to.

Where N-DHT continues. Tapestry's locality is static after bootstrap; N-DHT's locality is continuously evolved via LTP, triadic closure, hop caching, and lateral spread. Tapestry's pub/sub was layered (Bayeux); N-DHT's is built into the protocol (axonal trees). Tapestry's deployment was server-class on PlanetLab; N-DHT targets browser-class WebRTC.

The Berkeley OceanStore family is the closest-shaped historical ancestor of the entire N-DHT architecture. We are an evolutionary step on that line, not a clean-room reinvention.

Kademlia (2002)

KADEMLIA[12] took the simplest of the early designs and shipped it. Two ideas; both productively conservative.

[12] Maymounkov & Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric." IPTPS 2002.

The first was the XOR metric. Distance between IDs a and b is $d(a, b) = a \oplus b$, interpreted as an integer. The metric is *symmetric* (unlike Chord's successor distance) and obeys a useful triangle-inequality variant. The symmetry meant that maintenance was cheaper: A's view of distance to B is the same as B's view of distance to A, so entries learned in one direction are equally informative in the other.

The second was *k-buckets and iterative α-parallel* `FIND_NODE`. Each node maintained $k = 20$ peers per XOR-distance bucket (160 buckets at 160-bit IDs, covering distance ranges $[2^i, 2^{i+1})$). A lookup issued $\alpha = 3$ parallel asynchronous `FIND_NODE` queries to the closest known peers and iterated until the k closest reachable peers had been identified. Bucket eviction was LRU-with-old-bias: *live old contacts were kept*, a deliberate hedge on Saroiu's empirical observation that longer uptime predicts longer uptime.

$k = 20$ is a tradeoff: small enough that maintenance traffic stays bounded, large enough that even with significant churn at least one peer per bucket is likely live. The number is empirically chosen; it is the same k N-DHT inherits as the bucket size in G-DHT and as the synaptome core size in NH-1's bootstrap.

Kademlia is the routing substrate that *every N-DHT node still uses at its bottom layer.* The XOR metric, the bucket-per-stratum structure, the $k = 20$ and $\alpha = 3$ defaults, the iterative lookup — these all carry through to NH-1 unchanged. What N-DHT adds, it adds on

top.

What it solved. A correctness proof that survived contact with reality. Twenty-four years after publication, Kademlia is in production on BitTorrent's Mainline DHT, IPFS / libp2p, Ethereum devp2p, and Tor v3 hidden services. It is the default the rest of the field is measured against.

What it left open. Geographic blindness, exhaustively. Kademlia treats a 500 km lookup the same as a 12,000 km lookup; no aspect of the protocol references the physical network. No pub/sub. No adaptation. No load awareness. The bucket eviction policy preserves stable contacts but does not respond to traffic.

What N-DHT inherits. The XOR metric. Iterative α-parallel lookup as the underlying routing primitive. k-buckets as the synaptome's stratum structure. The $k = 20$ / $\alpha = 3$ defaults. The Saroiu bias toward stable contacts — N-DHT's vitality function inherits this through the `recency` factor, which gives recently-confirmed entries elevated standing.

Where N-DHT continues. Bucket eviction by LRU-with-old-bias becomes vitality-scored eviction (§II); the "stable contacts" notion generalizes to "contacts the actual traffic uses." The fixed bucket structure becomes a single synaptome whose entries are placed by traffic. Two-hop AP scoring, LTP reinforcement, hop caching, and lateral spread are all added as separate mechanisms over the Kademlia substrate.

Vivaldi (2004)

Vivaldi[13] is not a DHT; it is a *coordinate system* for one. Each node continuously adjusts a synthetic position vector — typically in three-dimensional Euclidean space plus a small *height* component to capture last-mile asymmetry — so that the *Euclidean distance* between two nodes' coordinates approximates the *measured round-trip time* between them.

[13] Dabek, Cox, Kaashoek, Morris, "Vivaldi: A Decentralized Network Coordinate System." SIGCOMM 2004.

The mechanism is a spring-force update: every observed RTT pulls or pushes the local coordinate toward better consistency. Confidence-weighted, decentralized, requires no coordinator. Over many observations the system converges to a stable configuration where pair-wise Euclidean distances $\approx$ pair-wise RTTs.

The result is that *any node can predict RTT to any peer it has never directly measured*, purely from coordinates. Routing layers can then choose "nearby" peers without an active probe round.

Vivaldi was deployed in Coral CDN, Azureus / Vuze BitTorrent, and several content-addressable overlays. It is the most influential

predecessor for latency-aware peer-to-peer design.

What it solved. A learned, low-cost, decentralized way to rank peers by latency without explicit measurement. Vivaldi is the canonical answer to "how does a peer-to-peer system know which of its peers is closer without asking them?"

What it left open. Vivaldi gives you a coordinate; what you do with it is up to the routing layer. Convergence requires sustained traffic, and an attacker who falsifies their coordinate disrupts neighbors' position estimates.

What N-DHT inherits. The principle that latency is a queryable property of the network, not an external measurement event. N-DHT's AP scoring (Chapter II) factors observed per-synapse latency into routing decisions; the synapse's `latency` field is the descendant of a Vivaldi-style position estimate, computed from haversine distance in the simulator and intended to be replaced by a Vivaldi-style learned coordinate in a future implementation (§VIII).[14]

Where N-DHT continues. Vivaldi-as-replacement-for-S2 is a clean future direction; in the meantime, the structural locality of the geographic prefix gives us most of the benefit at a small fraction of the implementation cost.

[14] The G-DHT geographic prefix is a *structural* approximation of locality; a Vivaldi-style coordinate would be the *learned* version. N-DHT today uses the structural form for simplicity; the migration path to a learned form is described in Chapter VI.

Coral DSHT (2004)

Coral[15] coined the term *distributed sloppy hash table*. Each node measured RTT to peers it encountered and joined *multiple nested clusters* at increasing RTT thresholds (e.g., < 20 ms, < 60 ms, global). Each cluster ran its own internal DHT.

Lookup was local-first: a node queried the tightest cluster it belonged to; if no result was found, it expanded outward. Most queries terminated in the local cluster. *Cross-cluster fan-out happened only when necessary.*

The "sloppy" semantics relaxed the strict DHT promise. A key could be stored at *any* node in the lookup cluster, and queries returned the *first* hit, not the canonical XOR-closest one. This made Coral a CDN: the same content was replicated at many local nodes, and clients pulled from the nearest available.

What it solved. A working content-distribution overlay that ran in production from 2004 to roughly 2015 (Coral CDN), serving millions of cache requests per day at peak as a free CDN for academic and small-publisher sites.

What it left open. Coral's hierarchical-cluster design *enforces* locality structurally; nodes *know* what is near because they measure it at join. There is no learning over which peers are most useful within a

[15] Freedman, Freudenthal, Mazières, "Democratizing Content Publication with Coral." NSDI 2004.

cluster; cluster boundaries are static thresholds.

What N-DHT inherits. The principle that local-first routing dominates global routing in real workloads. Coral's nested-cluster structure is the same observation that motivates N-DHT's S2 prefix and AP-scoring's latency penalty.

Where N-DHT continues. Coral makes a hard structural choice (multiple nested DHTs, sloppy semantics); N-DHT keeps a single flat synaptome and *learns* locality through usage. Coral relaxes the DHT promise to win locality; N-DHT preserves the strict DHT promise and adds shortcuts on top.

Adaptive Stabilization (2006)

Ghinita and Teo[16] addressed one of the longest-running problems in DHT design: how often should a peer check that its routing table is still correct?

[16] Ghinita & Teo, "An Adaptive Stabilization Framework for Distributed Hash Tables." IPDPS 2006.

Most DHTs — Chord, Pastry, CAN — run *periodic stabilization*: a fixed-interval timer pings neighbors and refreshes pointers. Their core claim was that a fixed rate is wrong almost everywhere. Too low and lookup failure spikes during churn bursts (their data: 23.7% failure at 5 kills/sec). Too high and communication overhead reaches 400%+ even during quiet periods.

Mechanism. Each peer estimates its local node-failure rate μ and node-join rate λ in a rolling window. From those plus an estimate of network size N, it computes the probability that a given pointer's target has died, $P_{\text{dead}} = 1 - e^{-\mu \Delta t}$, and the probability that a new joiner has come to sit between this pointer and its ideal target, P_{inacc}. Stabilization splits into two channels:

- A *liveness check* (one ping, $O(1)$) fires when $P_{\text{dead}} \cdot P_{\text{fwd}}$ exceeds a threshold.
- An *accuracy check* ($O(\log N)$ lookup) fires when P_{inacc} exceeds a threshold.

The system is operated against a single tunable knob: the target lookup-failure rate P_f. The peer adjusts its own thresholds to hit it.

Headline result. On Chord under variable churn, Adaptive Stabilization achieved 2.2% peak failure at 172% overhead versus periodic stabilization's 7.5% failure at 420% overhead. 3.4× *lower failure,* 2.4× *lower overhead simultaneously*, by self-tuning rather than hand-picking a rate.

What it solved. The fixed-cadence problem. The instinct is recognizable as the closest match in mechanism family of any protocol N-DHT can be compared against.

What it left open. Ghinita-Teo tunes *cadence* — when to check — not *content* — which entries to keep. Their stabilization brings a Chord routing table back toward its canonical XOR-ideal. N-DHT goes the other direction: the routing table evolves *away* from the canonical ideal toward a traffic-shaped optimum.

What N-DHT inherits. The control-loop architecture: local observation → probabilistic model → threshold-triggered protocol response. N-DHT's temperature reheat on dead-peer discovery, the per-lookup probabilistic anneal, and the proposed load-aware AP scoring all belong to the same family.

Where N-DHT continues. N-DHT lacks the closed-form analytical model and the explicit QoS knob; instead, vitality is a heuristic correlate that consolidates many such signals into one score. The synthesis would be *NH-1 with Ghinita-style statistical estimation of churn driving adaptive anneal/reheat parameters* — the metaplastic NH-1 we describe in §VIII.

The Geographic DHT (this work)

This is our own prior work, included in the lineage because N-DHT inherits and extends it. Earlier variants of G-DHT (`geo`, `geoa`) are retired; the current design (`geob`) is what we discuss here and use as the comparator throughout this volume.

The geographic DHT (G-DHT) is a single-pass change to Kademlia: stamp the high $\textsc{geoBits} = 8$ bits of every node ID with the node's S2 cell prefix.

$$\text{nodeId} = \underbrace{\text{S2 cell prefix}}_{8 \text{ bits}} \parallel \underbrace{H(\text{publicKey})}_{56 \text{ bits}}$$

The XOR metric is unchanged. The bucket structure is unchanged. The lookup algorithm is unchanged. But XOR distance now *approximates physical distance*, because S2 is a Hilbert space-filling curve and consecutive cells along the curve are adjacent on the Earth's surface (Chapter II).

What it solved. The latency tax for regional traffic. Regional lookups (under 2,000 km) drop from $\sim$ 510 ms in plain Kademlia to $\sim$ 150 ms in G-DHT at 25,000 nodes — a 3.4× improvement, achieved by changing *only the high bits of node IDs.*

What it left open. The S2 prefix is self-declared. A node can claim any prefix it wants; the protocol does not verify. The benign failure mode is degraded routing (mis-located peers misroute traffic). The adversarial failure mode is a Sybil swarm in a target cell. Locality is structural and static; it does not adapt to observed traffic. Pub/sub is still bolted on via Kademlia-style K-closest replication, which drifts under churn.

What N-DHT inherits. The S2 cell prefix, unchanged. The stratified-bootstrap construction (Chapter II). The geographic-prefix

injection point in node IDs.

Where N-DHT continues. Locality becomes *traffic-shaped* via LTP and the four reinforcement mechanisms (§II); pub/sub becomes a built-in axonal-tree primitive instead of a layered K-closest replication (§II). The verification gap on the S2 prefix remains and is addressed in the deployment chapter (Chapter VI).

The NX Series (2024–2026)

NX-1 THROUGH NX-17 were our experimental sequence. Each iteration added or removed one mechanism, ran the full benchmark suite, and either kept the change or retired it. Appendix VIII documents the sequence in detail and is the right reference for the question *"why is this mechanism the way it is?"* for any NH-1 mechanism. The capsule summary: NX-1 was a routing-table Hebbian baseline; NX-3 introduced the geographic three-layer initialization; NX-4 added iterative fallback; NX-5 added stratified bootstrap and incoming-synapse promotion; NX-6 added churn-resilient routing (temperature reheat, dead-synapse eviction); NX-7 through NX-9 were dendritic pub/sub experiments that we eventually retired in favor of axonal trees; NX-10 introduced routing-topology forwarding; NX-15 introduced the AxonManager abstraction; NX-17 closed the publisher-prefix-addressing question and was the state-of-the-art before the NH-1 consolidation.

What the sequence did. Established empirically that each mechanism either justified its complexity in the benchmark numbers or did not. Of the eighteen rules NX-17 carried, twelve survived the consolidation into NH-1; six were retired because their contributions could be absorbed into the unified vitality model without measurable performance loss.

What we kept. Vitality-scored synaptome, AP scoring with two-hop lookahead, LTP, hop caching, lateral spread, triadic closure, iterative fallback, temperature reheat, dead-peer eviction, stratified bootstrap, incoming-synapse promotion, publisher-prefixed topic IDs.

What we retired. Two-tier (highway) synaptome — absorbed into the single vitality-scored tier; stratum-floor enforcement — absorbed into the diversity component of vitality (later removed entirely as ablation showed it was harmful, see Appendix VIII); Markov prelearning — not retained in NH-1; load-tracked AP scoring — deferred to the production-pathway chapter (§VI); some bespoke relay-pinning logic — absorbed into the standard axon-tree recruit-policy; weight-scale parameter on AP scoring — ablation showed it was not significant.

Where N-DHT continues from NX-17. The consolidation

itself: the same routing behavior, fewer parameters, simpler vitality function, improved tunability. Chapter III describes the consolidated rule set; Appendix VIII gives the full justification.

Mapping the Lineage to the Five Operations

N-DHT organizes every routing rule under five operations (Chapter II): NAVIGATE, LEARN, FORGET, EXPLORE, STRUCTURE. The following table maps the contributions of the protocols above into that frame, both as a summary of the lineage walk and as a preview of the architecture chapter that follows.

Protocol	**Navigate**	**Learn**	**Forget**	**Explore**	**Structure**
Chord	succession ring	—	periodic stabilization	—	finger table
CAN	Cartesian zones	—	—	—	zone-splitting join
Pastry	prefix routing	—	neighborhood-set repair	—	locality-preserving join
Tapestry	prefix + RTT-closest slots	soft-state location pointers along publish path	Patchwork probes	surrogate routing	nearest-neighbor join
Kademlia	XOR + iterative α-parallel	*(none, but bucket eviction LRU-with-old-bias hints)*	LRU eviction	α-parallel	k-buckets
Vivaldi	latency as queryable	coordinate update from observed RTT	—	—	—
Coral DSHT	local-first nested-cluster lookup	RTT-cluster join	sloppy semantics	—	nested cluster construction
Ghinita-Teo	threshold-gated lookup	local μ, λ estimation	adaptive cadence	threshold trigger	—
G-DHT (this work)	XOR with geo-prefix bias	—	—	—	S2 cell prefix in ID
N-DHT	**AP scoring + 2-hop lookahead + iterative fallback**	**LTP, hop cache, lateral spread, triadic closure, incoming-synapse promotion**	**vitality eviction, decay, dead-peer reheat**	**temperature anneal, ϵ-greedy first hop**	**S2 prefix, stratified bootstrap, sponsor-chain join, axonal-tree pub/sub**

Table 1: Mapping prior-art mechanisms into N-DHT's five operations. Each row shows what a protocol contributed to the lineage; each column shows where that contribution lives in N-DHT.

The bottom row is the destination of the lineage walk. It is also the table of contents for Part II, where each cell becomes a section.

Vocabulary

THE TERMS IN THIS CHAPTER are used throughout the volume. Each definition is followed by a chapter reference where the concept is developed in full. The intent is that this chapter serves both as a forward-reference for a reader new to the field and as a back-reference for a reader who has skipped ahead and needs to look something up.

The glossary is alphabetical. Where a term has a precise formal definition (an equation, a parameter range, a unit) the formal definition appears in the margin alongside the prose definition in the body. The extended glossary in Appendix VIII repeats this material with additional cross-references to source code symbols, useful when reading the implementation.

Glossary

ACTION POTENTIAL (AP) SCORING The mechanism N-DHT uses to choose the next routing hop. For each candidate synapse, AP combines XOR-distance progress, the synapse's learned weight, and the synapse's observed latency into a single score; the highest-scoring candidate is chosen. The latency factor halves the score every 100 ms, so a synapse that is mathematically a small step closer but physically a large step closer wins. See §II.

The simplified default form, ignoring weight bias for clarity: $AP(s,t) = progress \times 2^{-\ell/100}$, where progress is the bit reduction in XOR distance and ℓ is the synapse's latency in milliseconds. Two-hop lookahead extends this with one additional forward probe.

ANNEALING The exploration mechanism by which a node periodically replaces a weak synapse with a candidate from its 2-hop neighborhood, biased toward under-represented strata. Triggered probabilistically per hop, scaled by the node's current temperature. Inherited from NX-6; see §II (EXPLORE).

ANNEALRATESCALE A multiplier (default 1.0) on the per-hop annealing trigger probability. `annealRateScale = 0.10` reduces total N-DHT message volume by approximately 5× at no measurable routing-quality cost (Chapter V). The single most important tuning knob in NH-1.

AXON In neuroscience, the long branching cable that delivers a neuron's output to many downstream targets. In N-DHT, the data structure representing a publisher's delivery tree for a topic — rooted at the publisher's S2 cell, growing branches as subscribers attach. See Chapter II.

AXONAL TREE The complete delivery structure for a pub/sub topic: a root, an arbitrary-depth branching of sub-axons, and the subscribers attached to leaves and intermediate nodes. The tree grows by routed subscribe messages and heals by routed re-subscribe under churn. Contrast with K-closest replication (e.g., Pastry SCRIBE trees), which fix the tree shape at construction time. See Chapter II.

BILATERAL CAP The constraint that any two peers must *both* consent to a connection: peer A may want to add B to its synaptome, but if B's connection slots are full, the connection cannot form. This is the simulator's first-order analogue of WebRTC's per-browser connection limit (typically 50–100 simultaneous data channels). See §IV.

BOOTSTRAP The process of populating a new node's synaptome with an initial set of peers when the node joins the network. N-DHT uses a *sponsor-chain join* (inherited from Pastry's locality-preserving join, generalized to XOR strata) where each node along the lookup path of `newNodeId` contributes a row of its own synaptome. See §II (STRUCTURE) and §III.

CHURN The continuous arrival and departure of nodes from the network. We quantify churn as a percentage of alive nodes replaced per unit time. The benchmark suite tests 5% instantaneous churn (the robustness threshold) and continuous variable-rate churn (the sustained-load test). See §IV and Chapter V.

DABEK 3δ FLOOR A 2004 result[17] establishing the analytical lower bound on lookup latency in any recursive DHT: 3δ, where δ is the median one-way RTT between random pairs of nodes on the network. The proof is geometric: each hop covers half the remaining distance, the resulting series sums to 2δ, and one final delivery hop adds a further δ. N-DHT achieves 1.18–1.28× the floor at 25,000 nodes (Chapter V).

[17] Dabek, Li, Sit, Robertson, Kaashoek, Morris. "Designing a DHT for low latency and high throughput." NSDI 2004.

DECAY The slow erosion of synapse weight in the absence of reinforcement. N-DHT applies a multiplicative decay (WEIGHT $\leftarrow$ WEIGHT $\times\gamma$) at fixed intervals where $\gamma = 0.995$ by default. LTP-locked synapses (those within their *inertia* window) skip decay. See §II (FORGET).

Gini coefficient A scalar in $[0, 1]$ summarizing the inequality of a distribution: 0 is perfectly even, 1 is perfectly concentrated. We use it to characterize per-node traffic distribution per cycle: K-DHT and G-DHT exhibit Gini ~ 0.94 at 50,000 nodes, indicating heavy concentration; N-DHT exhibits Gini ~ 0.66, broadly distributed. See Chapter V.

hop A single message between two adjacent peers in a routing path. Latency of one hop is determined by the haversine distance between the peers' coordinates plus a fixed simulator constant (10 ms). A typical N-DHT global lookup at 25,000 nodes completes in 4–6 hops.

hop caching The mechanism by which every intermediate node along a successful lookup path remembers the destination as a directly-routable peer. After traversing the path $A \to B \to C \to D$, both B and C install a synapse pointing to D, with weight 0.5 and fresh inertia. Future lookups originating from any peer who reaches B or C benefit from the shortcut. See §II (LEARN).

Hot > 10× / > 100× nodes Counts of nodes whose per-cycle traffic exceeds 10× or 100× the network mean, respectively. The 100× threshold is our operational definition of a bandwidth-saturation hazard: nodes processing two orders of magnitude above the mean. N-DHT produces zero such nodes at any tested scale; K-DHT produces 56 at 50,000 nodes (Chapter V).

incoming synapse A synapse that was added to a peer's view because *another peer* reached it for the first time, not because the peer reached out. Incoming synapses contribute candidates for routing but with weight 0.1; if their use count crosses the `promoteThreshold` they are promoted to first-class synapses with weight 0.5. The mechanism makes the network notice who is interested in a node, not just who the node is interested in. See §II (LEARN).

inertia The protection window after a synapse is reinforced. While $\mathit{simEpoch} - \mathit{syn.inertia} < \text{INERTIA\_DURATION}$ (default 20 ticks), the synapse is excluded from decay and eviction. The neuroscience analogue is the synaptic-tagging mechanism[18] that protects newly-strengthened synapses from competing plasticity. See §II.

[18] Frey, U., & Morris, R. G. M. "Synaptic tagging and long-term potentiation." *Nature* 385, 533–536 (1997).

iterative fallback The routing behavior when no synapse offers forward XOR-progress. The lookup falls back to the closest unvisited peer in the synaptome (regardless of strict XOR-progress),

trading optimality for reachability. The mechanism is N-DHT's surrogate routing (Tapestry's term); inherited from NX-4. See §II (NAVIGATE).

k-BUCKET The Kademlia bucket structure: $k = 20$ peers per XOR-distance range $[2^i, 2^{i+1})$. N-DHT's synaptome is bucket-organized at bootstrap (via stratified bootstrap, §II) but allows entries to migrate between buckets through traffic.

LATERAL SPREAD The mechanism by which a successful hop-cache pickup propagates to the originating node's geographic neighbors. After node A caches target C via hop caching, A tells its $k = 2$ geographically-nearest synaptome peers about C as well, biased by their existing weights. See §II (LEARN).

LONG-TERM POTENTIATION (LTP) A neuroscientific phenomenon[19] in which repeated stimulation of a synapse increases its strength for hours to weeks. N-DHT applies this directly: every hop in a successful lookup that completes faster than the running EMA receives a weight increment (+0.05, capped at 1.0) on every traversed synapse. See §II (LEARN) and §II.

[19] Bliss, T. V. P., & Lømo, T. "Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit." *J. Physiol.* 232, 331–356 (1973). Hebb, D. O. *The Organization of Behavior.* Wiley, 1949 (the conceptual ancestor: "cells that fire together, wire together").

LONG-TERM DEPRESSION (LTD) The complement of LTP: synaptic strength erodes through disuse. In N-DHT this is realized as multiplicative decay of weight at fixed intervals (see DECAY).

MAINTENANCE TRAFFIC Messages exchanged for routing-table upkeep rather than for end-user lookups: ping checks, bucket refreshes, sponsor-chain queries on join. N-DHT generates very little of this in steady state; the dominant traffic source is annealing's 2-hop probes. See Chapter V.

NODE ID A 64-bit integer addressing a node in the network. In G-DHT and N-DHT the high GEOBITS $= 8$ bits encode the S2 cell prefix; the low 56 bits are derived from a hash of the node's public key. See §II.

PUBLISHER-PREFIX TOPIC ID The construction topicId = (publisher's S2 prefix)$\|H_{56}$(topicName). The high bits pin the topic's axon root close to the publisher; subscribers and publishers must agree on the prefix out-of-band as a topic-naming convention (typically encoded as `@XX/domain/event`). Inherited from NX-17; see Chapter II.

RECENCY The exponential-decay multiplier in vitality, capturing how recently a synapse was reinforced. Within the inertia window (INERTIA_DURATION ticks), recency is 1.0. Outside it, recency decays as $e^{-\Delta t/\tau}$ where $\tau =$ RECENCY_HALF_LIFE where the half-life is 50 ticks by default. See §II.

REINFORCE The verb form of LTP application: increment a synapse's weight by +0.05, set its inertia field to the current epoch (entering the protection window), and increment its use count. See §II (LEARN).

REPLAY CACHE The bounded ring buffer (default 100 messages) that each axon node maintains for each topic, holding recently-published messages. Late-joining subscribers can request replay from the cache to fill the gap from missed publishes. See Chapter II.

S2 CELL PREFIX Google's S2 library partitions Earth's surface into a hierarchy of cells along a Hilbert space-filling curve.[20] The top GEOBITS = 8 bits of every G-DHT and N-DHT node ID encode the cell containing the node's claimed lat/lng. Two cells whose prefixes are bitwise close are physically adjacent on the Earth's surface. See §II.

[20] `https://s2geometry.io/`

SLICE WORLD The adversarial test in which the network is partitioned almost in half (Eastern hemisphere from Western), connected only through a single bridge node near Hawaii. The protocol is asked to route between hemispheres. K-DHT achieves 0% success; N-DHT achieves $\sim 94\%$ via hop caching and triadic closure converting the bridge into a seed crystal for new cross-hemisphere edges. See §IV and Chapter V.

STRATIFIED BOOTSTRAP A bootstrap construction that allocates the k-bucket budget across XOR strata so that each stratum receives roughly equal representation, with explicit fill on the geographic-prefix strata (top GEOBITS). Inherited from NX-5; see §II.

STRATUM An XOR-distance bucket: peers whose IDs differ from the local node's by a number in $[2^i, 2^{i+1})$ are stratum i. N-DHT uses the term interchangeably with "bucket."

SYNAPSE The data structure for a single peer entry in a synaptome. Fields: `peerId`, `latencyMs`, `stratum`, `weight` ($\in [0, 1]$), `inertia` (epoch of last reinforcement), `useCount`, plus a few diagnostic fields. See §II.

SYNAPTOME The set of synapses a node maintains: its routing table. N-DHT's synaptome is a single tier (no separate "highway" or "leaf-set" tier); entries are differentiated by their vitality score, not by structural placement. Default capacity 50 synapses, matching the WebRTC connection budget of a typical browser. See Chapter II.

TEMPERATURE A per-node scalar in [T_MIN = 0.05, T_INIT = 1.0] that governs the probability of triggering an annealing event

each hop. Cools by ANNEAL_COOLING $= 0.9997$ per hop the node participates in; reheats to T_REHEAT $= 0.5$ on dead-peer detection. See §II (EXPLORE).

TICK / SIMEPOCH The simulator's logical time unit: one lookup. The `simEpoch` field of a node advances by one each time the node participates in a lookup. Decay, inertia decay, and recency are measured in ticks, not wall-clock time.

TRIADIC CLOSURE The mechanism by which a node observing the same transit pair $A \to \text{this} \to C$ multiple times *introduces* A directly to C. After TRIADICTHRESHOLD $= 2$ observed transits of the pair, the introducer hints both A and C to form a direct synapse. The name comes from social-network theory: dense triangles emerge in any network shaped by repeated interaction. See §II (LEARN).

TWO-HOP LOOKAHEAD The AP-scoring mechanism that explores not just one hop but two: for each candidate first hop, N-DHT probes the candidate's synaptome to estimate where the second hop would land, and scores the joint pair. The probe set is bounded by LOOKAHEADALPHA $= 5$ candidates per hop. See §II (NAVIGATE).

VITALITY The score by which synapses are admitted, retained, and evicted. VITALITY = WEIGHT × RECENCY. When a new candidate synapse wants to enter a full synaptome, the synapse with the lowest vitality among non-LTP-locked candidates is evicted. See §II (FORGET).

The single equation that drives every admission decision in N-DHT. Earlier generations had eighteen rules and forty-four parameters; the consolidation around vitality collapses the bookkeeping into one score. See Chapter II for the per-synapse computation and §III for how it replaces the prior rule set.

WEIGHT The strengthening factor in vitality, in $[0, 1]$. Increased by $+0.05$ per LTP reinforcement, decayed multiplicatively by $\gamma = 0.995$ per decay tick. Bootstrap synapses begin at 0.5; introduced peers (triadic closure, hop caching) start at 0.5; annealed peers start at 0.1. See §II.

The Five Operations

N-DHT classifies every routing rule under one of five operations, introduced briefly here and developed in Chapter II. They are the organizational frame for the rest of the document.

NAVIGATE Choose the next hop for a lookup. Mechanisms: AP scoring, two-hop lookahead, iterative fallback, ϵ-greedy first hop, direct-target shortcut.

LEARN Strengthen what works on a successful path. Mechanisms: LTP reinforcement, hop caching, lateral spread, triadic closure, incoming-synapse promotion.

FORGET Decay what is unused; evict by vitality. Mechanisms: multiplicative weight decay, vitality-based admission, LTP-locked exemption.

EXPLORE Inject randomness so the protocol does not lock onto local optima. Mechanisms: temperature-driven annealing, ϵ-greedy first-hop randomness, dead-peer reheat.

STRUCTURE Bootstrap and maintain the network's basic shape. Mechanisms: S2-prefix node IDs, stratified bootstrap, sponsor-chain join, axonal-tree pub/sub construction.

The frame is universal in the sense that *any* adaptive system needs all five: act on present input, learn from feedback, forget the obsolete, occasionally try something new, maintain basic structure. N-DHT is one realization of that pattern with biology-inspired specifics.

Part II

The N-DHT Architecture

The Neuromorphic Substrate

Neurons that fire together, wire together.
Donald Hebb · 1949

The neuromorphic DHT is named for an analogy. The analogy is exact rather than ornamental: the engineering problem faced by a peer in a peer-to-peer network is the engineering problem faced by a neuron in a brain. Both have a hard upper bound on how many connections they can maintain. Both face vastly more candidate partners than they can afford to keep. Both must decide, on the basis of local information only, which connections are worth retaining.

The brain solved this problem hundreds of millions of years ago. This chapter explains the biology in enough detail to follow the translation, and then performs the translation in full. The mechanism that emerges is not metaphorical. The data structure that holds the routing table in N-DHT (the *synaptome*) is the digital cousin of the synaptic matrix of a biological neuron; the rule that governs which entries persist (the *vitality function*) is the engineering counterpart of long-term potentiation; the protected window we apply to recently-reinforced entries is the synaptic-tagging mechanism described by Frey and Morris in 1997. The rest of the document depends on this translation, so we do it carefully.

The Engineering Problem the Brain Solved

A single neuron in your cerebral cortex connects to roughly 10,000 others through structures called *synapses*.[21] There are about 86 billion neurons total in a human brain.[22] Any given neuron *could* connect to almost any other in principle, but maintaining a synapse is metabolically expensive — the cell has to manufacture and traffic receptor proteins, maintain the molecular machinery of vesicle release, and pay the running cost of resting potentials at every contact. So neurons live with a hard cap, surrounded by far more potential partners than they can afford to keep.

How does the brain decide which connections to retain?

[21] Drachman, "Do we have brain to spare?" *Neurology* 64(12), 2004. Estimates vary by region, age, and methodology; the general figure of 10^4 synapses per cortical neuron is broadly accepted.

[22] Azevedo *et al.*, "Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain." *J. Comp. Neurol.* 513(5), 2009.

The same problem faces a peer in a real-world peer-to-peer network. A typical web-browser deployment, restricted by WebRTC, can maintain about 50–100 simultaneous data channels. The network might have hundreds of thousands or millions of nodes; the routing table can hold only a small fraction of them. Which fraction?

The brain answers this question through a phenomenon called *long-term potentiation*, abbreviated LTP.

Browser-specific: Chrome desktop $\sim$ 256; Firefox $\sim$ 190; Safari $\sim$ 95; mobile browsers as low as $\sim$ 20. The simulator and most deployment targets in this document use a 50-connection cap, calibrated to the lowest of the desktop tier with headroom for the application's own WebRTC traffic outside the DHT.

Long-Term Potentiation in Five Pieces

In 1973, Tim Bliss and Terje Lømo published the foundational experimental result.[23] They stimulated a particular pathway in a rabbit's hippocampus with a high-frequency burst of electrical pulses — a few hundred milliseconds of 10–100 Hz stimulation — and observed that, for the *following several weeks*, the same pathway responded more strongly to subsequent inputs. The connection had physically strengthened. It stayed strengthened.

[23] Bliss, T. V. P., & Lømo, T. "Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit." *Journal of Physiology* 232, 331–356 (1973).

Decades of follow-up research filled in the molecular detail. Five elements are relevant to our analogy.

1. The coincidence detector. Synapses contain a special kind of glutamate receptor called the *NMDA*[24] receptor, which has an unusual property. Glutamate binding alone does not open the channel; the post-synaptic membrane must *also* be sufficiently depolarized at the same moment. Both conditions in the same temporal window are required. NMDA receptors are biological AND-gates: they fire only when "the input arrived" and "the cell was already active" both happen.

[24] *N-methyl-D-aspartate*: the synthetic agonist used to identify the receptor in the 1980s. The same receptor type is found in essentially every excitatory synapse in the cortex.

The timing window is short: roughly 10–20 milliseconds. Two events more than $\sim$ 50 ms apart do not register as coincident.

2. The strengthening signal. When the AND condition is met, calcium ions rush into the post-synaptic cell. Calcium triggers a cascade of intracellular reactions whose net effect is that the synapse inserts *more* of a different kind of receptor (called *AMPA*). After the cascade resolves, the same incoming signal produces a larger post-synaptic response. The synapse has become physically more sensitive.

3. Consolidation. Over the next half hour to several hours, the strengthening transitions from molecular to structural. New proteins are synthesized; the synapse physically grows; in some cases entirely new spines emerge on the dendrite. The change becomes durable in a way the early-phase strengthening was not. This is the difference between *early-LTP* (lasting hours, reversible) and *late-LTP* (lasting weeks, structural).

4. Decay. Connections that don't get used regularly weaken over time. The complementary process is called *long-term depression*, or LTD. The molecular mechanism is roughly the inverse: a different intracellular cascade, triggered by low-frequency activity, that removes

AMPA receptors and strengthens the inhibitory side of the synapse. Without LTD the brain would saturate every connection at maximum strength; with LTD, only the connections that prove useful retain their gain.

5. Synaptic tagging. The fifth element is the most subtle and the one with the largest engineering implication. Frey and Morris published in 1997[25] that recently-strengthened synapses receive a *tag* — a transient molecular state — that protects them from being overwritten by other plasticity events for a brief window. Without the tag, the protein synthesis required for consolidation would be *first* captured by whichever synapse fired most recently, regardless of whether it had the coincidence-detection signal. With the tag, the proteins are routed to the synapses that the AND-gate said deserved them.

[25] Frey, U., & Morris, R. G. M. "Synaptic tagging and long-term potentiation." *Nature* 385(6616), 533–536 (1997).

The five together — coincidence detection, strengthening, decay, consolidation, tagging — comprise the brain's solution to the "which connections to keep" problem. The pithy summary, which predates the molecular detail by twenty-four years, is Hebb's:

> *Cells that fire together, wire together.*

Every artificial neural network you have ever heard of ultimately runs on a digital descendant of this rule.

Including all of contemporary deep learning. The backpropagation algorithm is a digital descendant of Hebbian learning: weight updates are proportional to the product of pre-synaptic and post-synaptic activations. The form changes; the underlying "coincidence increases weight" principle does not.

From Neuron to Routing Table

The translation from biology to network is direct. Each row of the table maps a neural mechanism to a routing-protocol mechanism that does the same engineering work.

That table is what N-DHT *is*. The synaptome is a routing table whose entries have weights; weights go up on success, down on disuse; recently-strengthened entries are protected briefly; when the table is full, the lowest-scoring entry is evicted. The network's "memory" is the routing table, and the routing table evolves into whatever shape best serves the actual traffic.

We will spend the rest of this Part formalizing each row. The formalization is the architecture: Chapter II defines the vitality function and the five operations that implement it; Chapter II adds the geographic prefix that seeds the routing table at bootstrap; Chapter II extends the same machinery to publish/subscribe via axonal trees.

Why the Analogy is Literal

In the brain	In N-DHT
A synapse (point of connection between two neurons)	A connection to a peer (an entry in the synaptome)
Synaptic strength, in $[0, 1]$, gated by AMPA receptor density	A learned weight on each connection, in $[0, 1]$
LTP: coincidence increases weight	Successful lookup increments weights along the path
LTD: disuse decreases weight	Each connection's weight slowly decays each tick
Synaptic tagging (recent activity protected)	Recently-used connections are protected from eviction (INERTIA_DURATION ticks)
Pruning of unused connections at metabolic limit	Lowest-vitality connection evicted when synaptome is full
NMDA AND-gate: input *and* post-synaptic activity	AP scoring's combination of progress *and* latency *and* weight
Spike-timing dependent plasticity: "together" is timed	LTP applied only on paths fast enough relative to a moving average of recent path latencies
Coincidence at the network level (groups firing together)	Triadic closure: peers observed in transit pairs are introduced to each other

Two objections are reasonable here. The first: "The analogy is too neat. Engineering problems borrowed from biology usually don't survive contact with the engineering details." The second: "Even if it worked once, biological mechanisms are shaped by evolutionary accidents that have nothing to do with the specific problem of network routing."

The defense is empirical. The vitality function (Chapter II) collapses eighteen routing rules from prior generations (NX-1 through NX-17) into twelve, drops the parameter count from forty-four to twelve, and produces routing within a small constant of the analytical 3δ floor (Chapter V). The *simplification* is the evidence: a forty-four-parameter collection of bespoke rules is not what an engineer designs from first principles to solve a problem. It is what the engineer arrives at when they accumulate fixes and never go back to audit. A twelve-parameter system organized around one equation is what you get when the underlying principle is correct and you let it do the work.

The biology was the principle that let the consolidation happen. That is the only claim the analogy needs to support.

A second defense is comparative. Other adaptive routing systems — Tapestry's soft-state pointers, Coral's nested clusters, Vivaldi's spring-force coordinate updates, Ghinita-Teo's adaptive stabilization — are all attacking the same problem from different angles. They are sibling solutions in the same design space (Chapter I). The neuromorphic for-

mulation is the one that comes out simplest. That is not an accident: the brain has been working on this problem with intense selective pressure for several hundred million years. We get to borrow the answer.

The Synapse and the Synaptome

THE DATA STRUCTURES that carry the analogy across. A *synapse* is the protocol's unit of routing knowledge. A *synaptome* is the collection of synapses a node maintains.

The synapse. A synapse holds a peer's identity and the weights and timestamps that govern how it is treated. The implementation fields are the following.

- `peerId` — the 64-bit identifier of the peer this synapse points to. Combined with the local node ID, this gives the synapse its XOR distance and stratum.
- `stratum` — the count of leading-zero bits of the XOR difference; equivalently, the bucket index in a Kademlia-style routing table. Used by stratified-bootstrap and stratum-diversity rules.
- `latency` — the synapse's observed round-trip latency to the peer, in milliseconds. Used directly by AP scoring (§II).
- `weight` — a real number in $[0, 1]$. Set to 0.5 on bootstrap and on hop-cache pickup; set to 0.1 on annealing; incremented by +0.05 per LTP reinforcement (capped at 1.0); decayed multiplicatively per decay tick.
- `inertia` — the simulator epoch at which this synapse was last reinforced. While $\mathit{simEpoch} - \mathit{inertia} <$ INERTIA_DURATION (default 20 ticks), the synapse is exempt from decay and from eviction. This is the synaptic-tagging analogue.
- `useCount` — the cumulative count of LTP reinforcements this synapse has received. Used by the incoming-synapse promotion rule (§II) and as a debug signal.
- `bootstrap` — a flag indicating this synapse was installed by the bootstrap process rather than by traffic. Bootstrap synapses receive a small admission preference under vitality competition[26]: when the lowest-vitality candidate would otherwise be a bootstrap entry, eviction prefers the next-lowest non-bootstrap entry instead.
- `_addedBy` — a debug field carrying the rule that added the synapse (`bootstrap`, `hopCache`, `lateralSpread`, `triadic`, `evictReplace`, `anneal`, `promote`). Not consulted by routing; used in observability (§VI).

In the simulator, `latency` is computed once at synapse creation as $\text{haversine}(A, B) \cdot c^{-1} + \delta_{\text{node}}$, where c is the speed of light in fiber and δ_{node} is a fixed per-hop processing cost. In production, `latency` would be a Vivaldi-style learned coordinate or an EMA over observed RTTs (§VIII).

[26] NX-6 parity. The preference is worth a few percentage points of routing quality at bootstrap cost vs. recovered convergence; ablation showed it was net positive at 25,000 nodes.

The synaptome. The synaptome is the JavaScript `Map` keyed by `peerId`, holding the live synapses. Default capacity is 50. When a new synapse wants to enter and the synaptome is at capacity, the vitality-based admission gate (§II, FORGET) decides which existing synapse to evict.

Incoming synapses. A second, parallel structure. When peer A uses the local node as an intermediate hop in A's routing, the local node does not necessarily already know about A. The local node logs a record in its *incoming synapses* map — "A has reached me" — with weight 0.1, in case A proves useful as a peer for outgoing traffic later. The mechanism is called *incoming-synapse promotion*: if the same incoming peer is reached for routing more than PROMOTETHRESHOLD times, the entry is upgraded to a regular synapse with weight 0.5 via vitality-based admission. This makes the network notice who is interested in a node, not just who the node is interested in.

The biological analogue is *retrograde signaling* — a post-synaptic neuron signaling back across the synapse to the pre-synaptic side, modulating its release behavior. The mechanism exists because traffic in a neural circuit is not strictly one-way, and a peer's view of who-cares-about-it is not strictly one-way either.

What This Substrate Buys

THE SUBSTRATE described in this chapter is not yet a routing protocol. It is the data structure on which a routing protocol can be built. What N-DHT does on top of the substrate constitutes the rest of Part II:

- Chapter II defines five operations (NAVIGATE, LEARN, FORGET, EXPLORE, STRUCTURE) and the rules within each. The five operations together fully specify how the synaptome is manipulated during routing.
- Chapter II adds geographic structure to the bootstrap process: node IDs encode position via S2, the initial synaptome respects strata weighted toward locality, and routes converge to short physical paths in the common case.
- Chapter II extends the substrate to support publish/subscribe. The result is the axonal-tree primitive, which inherits the same vitality logic for tree maintenance.

By the end of Part II, the substrate is operational: every mechanism the protocol uses to route lookups, deliver pub/sub, and heal under churn is fully specified. Part III then describes the specific NH-1

implementation choices that took the architecture from eighteen rules and forty-four parameters in NX-17 to twelve rules and twelve parameters in NH-1, and Part V measures what the combination delivers.

Five Operations, One Vitality Model

> *Act, learn from feedback, forget the obsolete, occasionally try something new, maintain basic structure.*
>
> THE UNIVERSAL PATTERN

THIS CHAPTER is the architectural heart of the document. Every routing decision N-DHT makes falls into one of five operations: NAVIGATE (act on present input), LEARN (update from feedback), FORGET (decay the unused), EXPLORE (occasionally try something new), and STRUCTURE (maintain the basic shape of the network). The operations together constitute the protocol. The single equation that drives admissions, retentions, and evictions is *vitality*, defined first.

The frame is universal: any adaptive system will need the same five operations in some form. The specifics of each operation are shaped by the substrate (Chapter II) and by the peer-to-peer setting. We describe each operation at the level of mechanism — what computation is performed, on what state, in what order — with formal pseudocode reserved for Chapter III once the NH-1 specifics are introduced.

The Vitality Function

Vitality is a real number in $[0, 1]$ that scores every synapse. The definition is one line:

$$v(s) \;=\; \mathit{weight}(s) \;\times\; \mathit{recency}(s) \tag{1}$$

Weight (the strengthening factor, in $[0, 1]$) accumulates under reinforcement and erodes under disuse. The dynamics are described in §II (additive reinforcement) and §II (multiplicative decay). At any instant, $\mathit{weight}(s)$ is the value the LTP and decay rules have left.

Recency (the timing modulator, in $[0, 1]$) captures how recently the synapse was reinforced. Within the *inertia window* of INERTIA_DURATION ticks (default 20) following reinforcement, recency is locked at 1.0 — this is the synaptic-tagging analogue from §II.[27] Outside the window, recency decays exponentially with half-

[27] The lock prevents the failure mode where a freshly-strengthened synapse loses to an older, more weight-saturated competitor in eviction. The fix preserves *recently learned* signals long enough to be reinforced again.

life RECENCY_HALF_LIFE ticks (default 50). The formal expression:

$$recency(s) = \begin{cases} 1 & \text{if } simEpoch - s.inertia < T_{lock} \\ \max(0.1, e^{-(simEpoch - s.inertia)/\tau}) & \text{otherwise} \end{cases} \tag{2}$$

where τ = RECENCY_HALF_LIFE. The floor of 0.1 prevents very old synapses from contributing zero vitality even when they have non-trivial weight; this matters in low-traffic regions where a synapse may be useful but used rarely.

That is the entire vitality function. It is the only score the admission gate (§II) consults.

Why this product, and not a sum? A sum would let a high-weight, very-old synapse beat a freshly-introduced one with moderate weight indefinitely. A product makes recency a first-class veto: a synapse that has not been reinforced in many ticks is automatically a weak candidate, regardless of its historical weight. The brain works the same way — consolidated synapses still need recent activity to remain useful.

NAVIGATE: Acting on Present Input

NAVIGATION is the operation by which a node selects the next routing hop for a given lookup. Five mechanisms operate in priority order; the first one to succeed determines the next hop.

Mechanism 1: Direct shortcut. If the synaptome contains a synapse pointing directly to the target ID, use it. Trivial in practice (the lookup is one hop), but matters when hop caching (below) has installed the destination earlier.

Mechanism 2: Epsilon-greedy first hop. On the very first hop of a lookup, with probability EPSILON (default 0.05), pick a candidate uniformly at random from those that make XOR progress. This is the protocol's exploration-vs-exploitation hedge: a purely greedy router would lock onto the first decent shortcut and never discover better ones.

Mechanism 3: Action-Potential (AP) scoring with two-hop lookahead. The default mechanism. AP scoring combines XOR-distance progress with observed latency into a single score per candidate; the highest-scoring candidate is chosen. The formula uses progress in bits and latency in milliseconds, with latency entering exponentially:

$$AP(s,t) \;=\; progress(s,t) \;\times\; 2^{-\ell(s)/100} \tag{3}$$

where $progress(s,t)$ is the reduction in XOR distance from the current node's distance to t to s.peer's distance to t (interpreted as an integer; can be negative for non-progress candidates), and $\ell(s)$ is the synapse's observed latency in milliseconds. The exponential gives the score a half-life of 100 ms in latency: a synapse that is mathematically a small step closer but physically a large step closer wins, by a factor proportional to how much closer.

Two-hop lookahead. A pure single-hop AP score is greedy: it picks the locally-best peer regardless of where the peer's peers sit. For each of the top LOOKAHEADALPHA candidates (default 5), N-DHT extends the search one step: it inspects the candidate's synaptome (already known to the local node from previous interactions or sponsor-chain bootstrap) for the candidate's best forward-progress synapse, and computes a joint two-hop AP score. The candidate whose two-hop projection is best becomes the actual chosen first hop. The mechanism doubles the effective routing budget at small cost (the candidate's synaptome is local data, no extra round trip required).

Mechanism 4: Iterative fallback. If no synapse offers forward XOR progress (the lookup has hit a local minimum), the fallback is to pick the closest unvisited peer in the synaptome *regardless* of strict progress. This trades optimality for reachability and is the mechanism that prevents catastrophic failures at partition boundaries. Inherited from NX-4. Tapestry called the same mechanism *surrogate routing*.

Mechanism 5: Termination. If even the iterative fallback yields no candidate (every synapse visited or every synapse dead), the lookup fails. The simulator records the failure for the success-rate calculation. In practice this happens exceedingly rarely: at 25,000 nodes under normal conditions the success rate is 100% across all our tested workloads.

The five mechanisms together handle every navigation case the protocol encounters. Chapter III gives the full pseudocode for a lookup tick, including the operation-by-operation walk-through.

LEARN: Updating from Feedback

LEARNING happens after a successful lookup. N-DHT applies five reinforcement mechanisms; the first three run on every successful path, and the last two run on observed transit patterns.

1. Long-term potentiation (LTP). Every synapse traversed in a successful path that was *faster than the running average* of recent path latencies receives a reinforcement event:

- Increment *weight* by +0.05, capped at 1.0.
- Set *inertia* to the current simulator epoch (entering the INERTIA_DURATION protection window).
- Increment *useCount* by 1.

The "faster than average" filter is essential. If LTP fired on *every* successful path, every synapse on a slow recovery path would also be reinforced, and the protocol would learn *whichever paths exist*, not *which paths are fast*. The filter is implemented as: maintain an exponential moving average of recent path latencies (per node), apply LTP only on paths with total latency at or below the current EMA. The EMA itself updates with each successful path, so the threshold is adaptive.

2. Hop caching. Every intermediate node along a successful path remembers the destination as a directly-routable peer. After traversing $A \to B \to C \to D$, both B and C install a synapse pointing to D (via vitality-based admission, weight 0.5, fresh inertia). The next lookup originating anywhere — not just from A — that needs D and reaches B or C will take the shortcut. This is the source of the dramatic hop-count reductions we see for concentrated-destination workloads.

The mechanism is the structural ancestor of Tapestry's soft-state location pointers (Chapter I); we make it active on every successful path rather than only on `PublishObject` events.

3. Lateral spread. A successful hop-cache pickup propagates to the originating node's geographic neighbors. After A caches D, A tells its $k =$ LATERAL_K $= 2$ geographically-nearest synaptome peers about D as well, biased by their existing weights. The mechanism is named after the biological observation that activated neurons influence their spatial neighbors via diffusive signaling.[28]

[28] The biological analogue is loose; the engineering function is precise. Lateral spread converts a one-to-one cache hit into a one-to-three (or k+1) cache deposit, dramatically accelerating shortcut formation in dense regional clusters.

4. Triadic closure. A node observing the same transit pair $A \to$ (this node) $\to C$ on multiple lookups introduces A directly to C. The transit count is held in a local cache; when it crosses TRIADIC_THRESHOLD $= 2$, the node sends an introduction hint to A pointing at C. The name is from social-network theory: dense triangles emerge in any network shaped by repeated interaction.

The biological analogue is the network-level coincidence counterpart to NMDA receptors. NMDA detects coincidence at *single synapses*; triadic closure detects coincidence at the *network* level, between groups of peers that keep flowing through the same intermediate.

5. Incoming-synapse promotion. An incoming synapse (§II) records that another peer reached this node. If the same incoming peer is reached for routing more than PROMOTE_THRESHOLD $= 2$ times, the protocol promotes the incoming entry to a first-class outgoing synapse with weight 0.5, via vitality-based admission. The

mechanism turns observed inbound interest into established routing capacity.

FORGET: Decaying the Unused

FORGETTING is the operation that keeps the synaptome's content responsive to current traffic rather than historical accumulation. Three mechanisms.

1. Periodic decay. Every $\mathrm{DECAY\_INTERVAL} = 100$ lookups in which the local node participates, every synapse not under LTP-lock has its weight multiplied by $\gamma = 0.995$:

$$\mathit{weight}(s) \leftarrow \mathit{weight}(s) \cdot \gamma \tag{4}$$

After fifty unrefreshed decay ticks (5,000 lookups), a synapse that started at weight 1.0 has fallen to 0.78; after a hundred, 0.61; after two hundred, 0.37. The shape is gentle enough that occasional-but-real synapses retain useful weight, sharp enough that genuinely-unused synapses fade to eviction-eligible territory within a few decay periods.

2. Vitality-based admission. When a new candidate synapse wants to enter a full synaptome, the rule is:

1. Compute the vitality of every existing synapse not under LTP-lock.
2. Among the non-bootstrap synapses, identify the lowest-vitality candidate $v_{\min}$.
3. If $v_{\min}$ is below the new candidate's projected vitality, evict it and admit the new synapse. Otherwise, refuse admission.

The bootstrap-synapse preference is a safety hedge: bootstrap entries are never first-evicted as long as a non-bootstrap candidate is available, because they encode the structural backbone of the routing table. If *every* candidate is a bootstrap synapse, the gate falls back to the lowest-vitality bootstrap entry.

3. LTP-lock exemption. Synapses within the inertia window ($\mathit{simEpoch} - \mathit{inertia} < \mathrm{INERTIA\_DURATION}$) are exempt from both decay and eviction. This is the synaptic-tagging mechanism in action: recently-strengthened synapses are protected from competing plasticity for a brief window.

The combination of the three mechanisms gives N-DHT its characteristic behavior: *the synaptome's content drifts toward whatever the current traffic is using*, with enough hysteresis that isolated bursts don't permanently disrupt the routing table.

EXPLORE: Avoiding Local Optima

A PURELY GREEDY adaptive system locks onto the first decent solution it finds and stops looking. N-DHT injects randomness through three mechanisms.

1. Temperature. Each node carries a scalar $T \in [T_{\min} = 0.05, T_{\text{init}} = 1.0]$ (the named constants are T_MIN and T_INIT). After each hop the node participates in, T is multiplied by ANNEAL_COOLING $= 0.9997$ down to the floor. After several thousand hops a typical node sits at the floor, $T = 0.05$.

2. Probabilistic annealing. On each hop, with probability $T \cdot r$ (where r is ANNEALRATESCALE), the node attempts an annealing step: pick a weak existing synapse, scan the 2-hop neighborhood for an under-represented stratum, find a candidate, and replace the weak synapse with a synapse to the candidate. The ANNEALRATESCALE parameter (default 1.0) is the master volume control on annealing rate; Chapter V describes how it serves as the bandwidth-vs-distribution tuning knob.

The biological analogue is exploratory dendritic outgrowth: a neuron that has spare capacity occasionally extends a new branch toward an under-represented region. The engineering rationale is the same: keep some uncommitted slots available so that emerging patterns of useful traffic can be picked up.

3. Reheat on dead-peer detection. When a routing decision encounters a dead peer (a synapse pointing at a node that no longer responds), the protocol *reheats*: the node's temperature is raised to T_REHEAT $= 0.5$. Subsequent hops will fire annealing more aggressively until the temperature cools again. This is the mechanism by which N-DHT recovers synaptome composition under churn without needing a periodic maintenance schedule (Ghinita-Teo's adaptive-stabilization problem solved differently; Chapter I).

4. Epsilon-greedy first hop. Already mentioned in §II, but listed here too because it is also an exploration mechanism. The epsilon hedge is small (5% of first hops) but persistent, ensuring that the protocol always explores *some* fraction of off-greedy first-hop choices regardless of synaptome state.

STRUCTURE: Maintaining Network Shape

STRUCTURE is the operation that maintains the underlying graph topology: new nodes joining, the bootstrap process, and the maintenance of basic connectivity invariants.

1. Geographic node IDs. Every node ID encodes the node's S2 cell prefix in the high bits (Chapter II). This is a structural seed: it makes XOR distance correlate with physical distance from the moment the node joins, before any traffic-driven learning has happened.

2. Stratified bootstrap. A new node's initial synaptome is populated by a stratified algorithm: enough entries in each XOR-distance bucket to satisfy the routing-table connectivity guarantee, with explicit fill on the geographic-prefix strata (top GEOBITS) so locality is operational immediately. The budget allocation is described in detail in Chapter II; the principle is that the synaptome should be *usable for routing* from the first lookup, not require a learning warmup.

3. Sponsor-chain join. A new node joins through a sponsor: it routes a self-lookup query through the sponsor toward its own ID. Each node along the resulting path contributes a row of its synaptome to the joining node, in the same locality-preserving manner as Pastry's join (Chapter I). The result is that the new node arrives with synaptome entries that are RTT-close to it, courtesy of the path's locality.

4. Bilateral cap enforcement. Every connection edge is enforced bilaterally: peer A can want B in its synaptome, but the connection only forms if B's connection slots also have room. The cap is the simulator's first-order analogue of WebRTC connection limits and the structural constraint behind every admission decision.

5. Axon-tree construction. Pub/sub topics build delivery trees through the same mechanism: a routed subscribe message follows the routing layer, and every intermediate node along the path becomes a potential axon-tree branch point. The full treatment is in Chapter II; here we note that the axon tree is structurally part of the protocol, not a layered add-on.

The Universality of the Five-Operation Frame

THE FRAME IS UNIVERSAL. Any system that adapts to its environment must implement all five operations in some form. NAVIGATE is the act-on-input operation: every system must produce *some* output for current input. LEARN is the credit-assignment operation: every system must update its internal state from feedback or it will not adapt. FORGET is the entropy-resistance operation: every system must shed obsolete state or it will saturate. EXPLORE is the local-optima-avoidance operation: without it, a system locks onto the first usable solution and stops improving. STRUCTURE is the topology-maintenance operation: every system needs some account of how its parts are connected.

The same frame describes how reinforcement learning systems work

(action selection, reward update, value-function decay, ϵ-greedy, environment model), how immune systems work (antigen recognition, clonal expansion, naive-cell turnover, hypermutation, repertoire diversification), and how social networks form (contact, friendship reinforcement, drift, novelty seeking, structural maintenance). N-DHT is one realization of this pattern with biology-inspired specifics tuned to the peer-to-peer routing problem.

The unification is not just an organizational convenience. Earlier generations (NX-1 through NX-17) carried eighteen rules across ad-hoc categories; the same rules organized into five operations made it visible which mechanisms did the same job at different abstraction levels, which made consolidation possible. Twelve rules and twelve parameters, with one equation (vitality) governing every admission decision, is the consolidation that became NH-1 (Chapter III).

Geographic Locality

LOCALITY in a peer-to-peer network is the property that two nodes physically close on the Earth's surface can reach each other with few hops and small per-hop latency. The property is essential because real workloads are dominated by local traffic. Kademlia does not have it; locality is its single largest performance gap. G-DHT introduces it via a geographic prefix in node identifiers, and N-DHT inherits the construction unchanged. This chapter describes the construction, its tradeoffs, and the way it interacts with the learning mechanisms of Chapter II.

The principle, stated up front: **locality is a seed, not a teacher.** The geographic prefix gives the routing structure *starting* affinity for short physical paths; the learning mechanisms then *strengthen* the affinity by reinforcing shortcuts that traffic actually uses. Without the seed, learning would have to discover regional structure from scratch (slow); without the learning, the prefix alone would provide static locality that doesn't respond to traffic (rigid). Together, they produce a routing table that is regional from the first lookup and increasingly traffic-shaped from then on.

The S2 Cell Prefix

GOOGLE'S S2[29] divides the Earth's surface into a hierarchy of cells. The construction projects the surface of a sphere onto the six faces of a circumscribed cube, partitions each face along a Hilbert space-filling curve, and assigns each cell a 64-bit integer identifier called the *cell ID*.

The Hilbert curve has a useful property: *geographically adjacent points have numerically close cell IDs.* Two cells that share a 4-bit prefix are guaranteed to be on the same quadrant of the same cube face; two cells that share an 8-bit prefix are within roughly continental scale of each other. Refining the prefix one bit further halves the cell area along the curve.

[29] S2 documentation: `https://s2geometry.io/`. The library is open source under Apache 2.0; we use only the Hilbert-curve cell-ID computation, which is a few hundred lines of arithmetic.

Top 8 bits. N-DHT uses the top 8 bits of the S2 cell ID as the geographic prefix. Three bits encode the cube face (six faces, two unused encodings); five bits subdivide that face. $6 \times 32 = 192$ tiles worldwide,[30] each on the order of continent-scale (e.g. "western North America," "South-East Asia," "northern Europe").

[30] Some tiles are unused or sparsely populated (oceans, polar regions); the live node distribution settles into roughly 80–120 active tiles on a uniform-land deployment.

Sub-cell hierarchy. Refining the prefix bit-by-bit subdivides the tile in half along the Hilbert curve. At 16 bits the cell is roughly state-sized. At 30 bits it is roughly city-block-sized. At 40+ bits it is metres. N-DHT uses 8 bits because that is the right granularity for routing decisions: fine enough to distinguish continents, coarse enough that the remaining 56 bits of node ID give plenty of intra-cell diversity.

Earlier experiments tried 4 and 16. Four gave too few cells (only 24 live tiles); routing decisions within a tile were near-random. Sixteen gave too many tiny cells (tens of thousands), most empty; routing within a tile collapsed to a single peer. Eight is the empirical knee.

Node ID Construction

The node ID is a 64-bit integer assembled from two fields:

$$nodeId = \underbrace{\textit{S2 cell prefix}}_{8 \text{ bits}} \parallel \underbrace{H_{56}(\textit{publicKey})}_{56 \text{ bits}}$$

The high 8 bits come from the S2 cell containing the node's claimed lat/lng coordinates (computed by the S2 library's `cellIDFromLatLng()` routine, truncated to 8 bits). The low 56 bits are a hash of the node's public key, providing intra-cell uniqueness while keeping the ID bound to a stable cryptographic identity.

Why XOR distance approximates physical distance. The XOR metric Kademlia uses (Chapter I) is bit-by-bit: two IDs differ by an integer whose bit pattern reflects which bits disagree. Two node IDs that share their top 8 bits XOR to a value whose top 8 bits are zero — a small integer in absolute terms. Two node IDs that disagree in the top 8 bits XOR to a value whose top 8 bits are nonzero — a large integer.

Because the top 8 bits encode geographic position via the Hilbert curve, "share the top 8 bits" translates to "geographically close," and "differ in the top 8 bits" translates to "in different continents." XOR distance, applied to S2-prefixed IDs, *approximately tracks* physical distance.

The approximation is loose at fine granularity (intra-cell, the 56-bit hash dominates and XOR distance is essentially random) and tight at coarse granularity (cross-cell, XOR distance and physical distance are very tightly correlated). This is exactly the property a routing table wants: routes within a region need all the intra-region diversity they can get; routes between regions need to find each other quickly.

Stratified Bootstrap

THE BOOTSTRAP populates a new node's synaptome with 50 initial entries distributed across XOR strata such that:

Inherited from NX-5 with minor refinements. The motivating observation: a uniformly random synaptome at bootstrap fails to give good locality immediately, because the geographic-prefix strata are statistically underrepresented. A pure-locality bootstrap fails the other way, collapsing reachability when local traffic doesn't dominate. Stratified bootstrap is the empirically-tuned middle ground.

- Every stratum from 0 (closest to local) up to 63 (most distant) has at least one entry, so the routing-table connectivity guarantee is satisfied. This consumes ≤ 64 entries (one per stratum), well within the 50 budget for sparse strata.
- The geographic-prefix strata (top GEOBITS) are explicitly filled: the bootstrap pulls multiple entries from each of these strata up to a per-stratum cap, so that intra-region routing has multiple peers to choose from from the start.
- Remaining budget is allocated by sampling. The result is that the synaptome shape is *biased* toward locality without being *exclusive* to it.

The mechanism is deterministic given the bootstrap candidate list, so we describe it as a budget-allocation algorithm:

Algorithm 1: Stratified bootstrap (sketch)

1: $budget \leftarrow 50$
2: $geoStrata \leftarrow \{0, 1, \ldots, g-1\}$ ▷ $g =$ GEOBITS; the locality strata
3: Allocate k slots per geo stratum from *budget*
4: Allocate 1 slot per non-geo stratum (skeleton)
5: Allocate remaining slots uniformly at random across all strata
6: Materialize each slot by sampling a peer from the candidate list at that stratum

The candidate list is built by a sponsor-chain join (§II): the new node routes a self-lookup through its sponsor; each node on the path returns a row of its synaptome; the union becomes the candidate list. This piggybacks on Pastry's locality-preserving join (Chapter I) to ensure that the candidates are themselves RTT-close to the sponsor's region.

Locality is a Seed, Not a Teacher

THE S2 PREFIX gives N-DHT *starting* locality. The learning mechanisms of Chapter II give it *evolving* locality. The interaction between the two is the protocol's locality story.

The seed. The bootstrap synaptome is geographically biased; AP scoring's latency penalty means that, even before any LTP has fired,

the protocol prefers nearby peers; XOR distance in S2-prefixed ID space approximately tracks physical distance, so $\log N$-bound iterative searches naturally converge on regional peers in the common case. From the first lookup, regional traffic already routes regionally.

The teacher. As traffic flows, LTP reinforces the synapses that the actual workload uses. In a region where most traffic is regional, LTP-strengthened synapses are nearby; in a region with sustained cross-continental flows, LTP strengthens some non-local synapses too. Hop caching installs targets that were the destination of recent successful paths; lateral spread propagates these to geographic neighbors. Over enough lookups, the synaptome's content reflects the traffic pattern in the node's region.

The non-locality story. The same protocol routes non-local traffic perfectly well. Cross-continental lookups follow the iterative-fallback path: first, route within the local region toward a peer whose strata cross the geographic boundary; then, the cross-boundary peer's synaptome (which carries some peers in the destination region by virtue of its under-represented-stratum bootstrap fill) provides the next hop. The geometric-cost analysis (3δ floor, Chapter V) is unchanged by the locality bias; locality is purely additive.

The combination is what gives N-DHT its scaling shape. Regional latency is short and *decreases* with N as more nodes give more shortcut opportunities. Global latency is bounded near the 3δ floor and grows logarithmically. Both regimes are captured by the same routing protocol with no special-cased modes.

Cooperative Trust

The cost of structural locality is a trust assumption. The S2 prefix is *self-declared*: a node can claim any prefix it wants, regardless of where it actually is. The protocol does not verify.

The benign failure mode is degraded routing. A node that lies about its location ends up in a synaptome stratum that does not match its actual physical position; routing through it costs more latency than the synaptome's latency estimates would suggest. The node still works as a routing primitive — it can still receive and forward messages — but it does not contribute to locality.

The adversarial failure mode is a *cell eclipse*: an attacker generates many forged identities all claiming to land in the same target cell, flooding that cell with attacker-controlled peers. Subsequent honest peers in the cell are likely to bootstrap from the attacker's nodes; the attacker can then drop traffic, inject false peers, or redirect lookups within the cell. Chapter VI discusses this attack class and the mitiga-

tions.

Mitigations. Three non-exclusive directions are visible in the literature:

- *Verifiable RTT* via Vivaldi-style coordinates (Chapter I): a peer's claimed location must be consistent with measured RTT to other peers. An attacker that misclaims must also fake RTTs, which is expensive at scale.
- *Geographic proof-of-work*: bind the cell prefix to a cryptographic challenge that takes more time on a node not physically in the cell (e.g., a TURN-server-of-record handshake, an IP–ASN binding check).
- *Trusted-witness schemes*: have a small number of well-known peers attest to the locations of other peers, similar to certificate-authority models.

The current protocol implements none of these; the prefix is cooperative trust. *The protocol does not depend on the prefix being honest, but its locality benefits do.* Chapter VI ranks the eclipse-attack risk as a Tier-2 issue: real, but not blocking initial deployment, and addressable through staged hardening.

What this means for performance claims. Every quantitative claim in this document about regional latency assumes honest prefixes. The simulator's nodes have correct prefixes by construction. A real-world deployment with 5% adversarial prefixes would experience proportionally degraded locality, not catastrophically degraded routing. The protocol's correctness is unchanged; only the locality benefit is.

Where Vivaldi Could Replace S2

A FUTURE DIRECTION[31] is to replace the structural S2 prefix with a *learned* coordinate, in the spirit of Vivaldi (Chapter I). The trade is concrete: structural locality is fast to bootstrap (no convergence period required, prefix is known at addNode-time), trust-bound (the prefix is self-declared), and parameter-free (no learning rate, no coordinate dimension to choose). Learned locality is slow to bootstrap (requires sustained traffic for convergence), trust-free (coordinates can't be falsified without also falsifying RTTs), and has a few tuning parameters.

For initial deployment, structural locality is the right choice: it gives most of the benefit at a small fraction of the implementation and operational cost. For mature deployment with adversarial concerns, the migration path to learned locality is clean — the AP scoring's

[31] See §VIII for the full discussion.

latency factor (§II) already uses observed latency, so a Vivaldi-style coordinate system would slot in under the existing scoring formula without disturbing the rest of the protocol.

Axonal Publish/Subscribe

Pub/sub belongs in the network, not on top of it.

PUBLISH/SUBSCRIBE is the second primitive a real peer-to-peer substrate must provide. A publisher sends a message to a topic; every subscriber to that topic receives it. The publisher does not know the subscribers; the subscribers do not know each other; the network handles the fan-out.

Most contemporary peer-to-peer systems either layer pub/sub on top of the routing primitive (Pastry's SCRIBE, IPFS's gossipsub) or punt the problem to a central component (essentially every production messaging app, however "decentralized" the addressing layer). N-DHT builds pub/sub *into* the protocol via a structure called the *axonal tree*, named for the output branching of a biological neuron. The tree grows by routed subscribe messages, heals continuously by routed re-subscribe under churn, and delivers 100% of messages at baseline and 100% of messages within three refresh intervals under 5% instantaneous churn (Chapter V).

This chapter describes the construction. Chapter III gives the per-tick pseudocode.

Five Requirements

WHAT PUB/SUB must do, stated as five requirements that any working implementation must meet.

1. Topic addressing. A publisher and a subscriber must agree on a topic *address* without coordinating. Both must be able to derive the address from the topic name alone, with no central registry.

2. Subscriber discovery. The publisher must be able to reach all subscribers without knowing them individually. The publisher's only inputs should be the topic address and the message payload.

3. Bounded fan-out. A single node must not be required to deliver to a number of peers proportional to the total subscriber count of a popular topic. A topic with 10,000 subscribers cannot saturate

the publisher's outbound bandwidth.

4. Resilience to churn. A subscriber that joins should start receiving messages quickly. A node in the delivery path that leaves should not silently break the topic for downstream subscribers; recovery should be automatic.

5. Replay for late joiners. A subscriber that joins *slightly* after a publish should be able to receive the recent past of the topic — not the entire history, but enough to fill the gap from missed messages.

The five requirements together are what lets pub/sub be *useful* as a real-time primitive: discovery is automatic, fan-out is bounded, churn doesn't break anything, late joiners recover.

Topic Addressing: The Publisher Prefix

THE ADDRESS of a topic is a 64-bit integer. N-DHT constructs it as:

$$topicId = \underbrace{publisher's\ S2\ prefix}_{8\ \text{bits}} \parallel \underbrace{H_{56}(topicName)}_{56\ \text{bits}}$$

The high 8 bits come from the publisher's S2 cell prefix; the low 56 bits are a hash of the topic name. The address is deterministic: given the topic name and the publisher's claimed location, both publisher and subscribers can compute the same topic ID without coordinating.

Why publisher-prefix instead of plain hash. If the topic ID were a uniform hash of the topic name (NX-15 era), the topic's delivery tree would root at a random cell on the planet. A chat group of US-east subscribers might find its tree pinned to central Asia. By embedding the publisher's S2 prefix in the topic ID, the tree naturally roots near the publisher, and the publisher's neighborhood (in S2) is also typically the subscribers' neighborhood (real-world topic affinity is geographically clustered). Inherited from NX-17.

Naming convention. Publishers and subscribers must agree on the topic ID, which means agreeing on the publisher's prefix. The convention we use is the topic-name form `@CC/domain/event` where `CC` is the two-character S2-cell shortcode for the publisher's region. The convention is out-of-band; the protocol does not enforce it but treats topic IDs as opaque 64-bit integers.

Tree Construction

THE AXONAL TREE is built by routed subscribe messages.

Subscribe. A subscriber wishing to follow a topic constructs the topic ID and routes a `pubsub:subscribe` message toward it via the

underlying N-DHT routing layer. The message is processed at every intermediate node along the path: each node checks whether it is already an axon for this topic.

- If the node is already an axon for the topic (it has open children for the topic), it accepts the new subscriber as one of its children and forwards no further. The subscribe has been intercepted; the subscriber is in the tree.
- If the node is not already an axon and the message has further to route, the node forwards. The message continues toward the topic ID.
- If the node is not already an axon and the message cannot route further (this is the topic's local root), the node opens a new role for the topic, becomes the *axon root*, and accepts the subscriber as its first child.

Publish. A publisher constructs the same topic ID and routes a `pubsub:publish` message toward it. The message is intercepted by the same first-axon-along-the-path; from there, it flows down the tree to all subscribers via the children-of-children recursion.

Branching on overflow. When an axon's direct-child count exceeds the configured `maxDirectSubs` threshold, the overflow subscriber is delegated to a sub-axon (*recruited* from one of the existing children, or from an external synaptome peer if the children are in the wrong direction for the new subscriber). The mechanism keeps the maximum fan-out at any node bounded by `maxDirectSubs`, regardless of total subscriber count. A topic with 10,000 subscribers might have 40 direct-child slots filled at the root, and the rest of the subscribers handed off recursively to depth-2 and depth-3 sub-axons.

Self-Healing Under Churn

THE MECHANISM that heals the tree under churn is the same routed re-subscribe that built it.

Every member of the tree — subscribers and axons alike — periodically (every `refreshIntervalMs` milliseconds) re-issues its subscribe message upstream. The re-subscribe is exactly the same operation as the initial subscribe: a routed message toward the topic ID.

If the tree is intact, the re-subscribe is intercepted by the same axon as before. The tree state is unchanged; the re-subscribe is a heartbeat.

If a parent died, the re-subscribe routes *around* the dead parent (the routing layer detects the dead peer and takes an alternate hop,

via the normal mechanism described in Chapter II). The re-subscribe lands at whichever live ancestor is now closest to the topic ID, which becomes the subscriber's new parent. The orphan branch reattaches automatically.

If the entire ancestor chain died, the re-subscribe routes all the way to the topic root. If the root is also dead, the re-subscribe terminates at the closest-surviving node, which opens a new role for the topic and becomes the new root. The tree has been rebuilt from the re-subscriber's perspective.

The mechanism uses no special machinery. There are no heartbeats beyond the re-subscribe itself; no failure detection separate from the routing layer's dead-peer detection; no parent-tracking state beyond the local children list. *The same operation that builds the tree repairs it.* This is the property that gives N-DHT 100% pub/sub delivery under 5% churn at 25,000 nodes (Chapter V).

TTL sweep. As a complement to re-subscribe, axons *also* TTL-sweep their children: any child that has not re-subscribed within `maxSubscriptionAgeMs` is dropped from the children list. The TTL sweep cleans up after subscribers that left without un-subscribing (e.g., the user closed the browser).

The Replay Cache

Late-joining subscribers are subscribers who joined *after* a publish. The naive behavior — the late joiner just misses the publish — is acceptable for some workloads but not for many.

N-DHT addresses this with a *replay cache*: each axon node maintains, for each topic it serves, a bounded ring buffer of recently-published messages. The default size is 100 messages. When a new subscriber joins, the subscribe message can carry a `lastSeenTs` parameter; the receiving axon then sends back all messages in its replay cache more recent than that timestamp. The new subscriber receives the publishes it missed during its join transition.

Bounded recovery. The replay cache is a bounded recovery, not infinite history. If a subscriber has been offline for longer than the cache holds, the gap is irrecoverable. This is acceptable for the operational target: "a subscriber missing a few seconds of publishes during a flaky reconnect should recover" is a much weaker requirement than "a subscriber that joins arbitrarily late should see all history" — the latter needs a different system (a log, not a pub/sub bus).

Per-topic sizing. For high-rate topics, the cache size in messages may correspond to a very short time window. The cache can be configured to size by time rather than message count, in which case the

cache holds whatever fits in `cacheLifetime` seconds. The protocol exposes both modes; the default is message-count for predictable memory.

Hot-Topic Migration (Future)

One known limitation[32] of the axonal-tree primitive as described above is that a single popular topic with high publish rate places its bandwidth cost on the topic root. The publisher sends to the root; the root fans out to its `maxDirectSubs` children; each child fans out further. The root's bandwidth scales with the number of direct children, which is bounded, but its compute cost (managing the children, sweeping TTLs, processing re-subscribes) scales with the topic's churn rate in subscribers. For very-high-rate Zipf-distributed pub/sub workloads, this can saturate the root's CPU.

[32] Discussed in Chapter VI as a residual of the bandwidth-saturation issue.

The mitigation is *topic migration*: an axon root that exceeds a load threshold can hand off the topic to a less-loaded peer in the same S2 cell, leaving a redirect at the original location. Subscribers re-subscribing to the original ID find the redirect and follow it to the new root; the migration completes when the redirect TTL expires. The mechanism is described in detail in Chapter VI as part of the production pathway. It is not implemented in the current N-DHT; the foundational evidence (Chapter V) suggests it is needed only for skewed pub/sub workloads, not for routing load in general.

Why "Axonal"?

The naming is exact. The output of a biological neuron is the *axon*: a long, branching cable that delivers the neuron's spike output to many downstream targets. The branching is structural — new branches grow when the neuron's outputs need to reach more cells; old branches retract when they fall into disuse.

The axonal tree in N-DHT plays the same role in a peer-to-peer network. A publishing node has an output (the topic message) that must reach many downstream targets (the subscribers). The tree branches are recruited as subscribers attach; sub-axons grow when fan-out exceeds the local capacity; branches retract when subscribers drop without renewing. The topology is not fixed at construction; it evolves with the topic's load.

The translation, paralleling the table from Chapter II:

The naming is not just decoration. The mechanism it describes is structurally what biological axons do: deliver an output to many

In the brain	In N-DHT
Axon hillock (where the spike originates)	Publisher
Axon proper (the long delivering cable)	The route from publisher to topic root
Axonal arbor (the branching output structure)	The axon tree spanning the topic's subscribers
Synaptic terminal (where the axon meets a target dendrite)	An axon's child link to a subscriber
Branch growth on activity	Recruit-on-overflow
Branch retraction on disuse	TTL sweep of stale children

targets through a branching cable that adapts its structure to the demands of the workload it carries.

Part III

NH-1: The Consolidated Generation

NH-1: Rules and Consolidation

Eighteen rules and forty-four parameters became twelve rules and twelve parameters. The behavior did not change.

PART II described the architecture in generation-independent terms: the substrate, the five operations, geographic locality, axonal pub/sub. This chapter describes the specific implementation of that architecture in the current generation, NH-1, and the consolidation work that produced it.

NH-1 is not a redesign of NX-17. It is the same protocol with the rule set audited, deduplicated, and reorganized around the vitality function. Every benchmark measurement in this volume was produced by NH-1; every architectural claim in Part II is a description of what NH-1 does. The reason we treat the generation as its own chapter is that the audit was the moment when "N-DHT" became a coherent thing rather than a pile of mechanisms each justified individually.

The Twelve Parameters

NH-1 carries twelve tunable parameters. Each maps to one of the five operations and controls a specific behavioral axis. Table 2 lists them in operational order.

The point of the tabulation is that the twelve parameters span the entire behavioral envelope of the protocol. There is no "hidden" state that depends on a parameter not in this table. Every other constant in the implementation (T_INIT, T_MIN, T_REHEAT, DECAY_INTERVAL, ANNEAL_LOCAL_SAMPLE, GEO_BITS, STRATA_GROUPS, LATERAL_K) is a fixed implementation choice not intended for routine tuning; the values are documented in Appendix VIII.

A reader who has internalized Chapter II can read Table 2 as a complete behavioral specification: tell me the twelve numbers and I can predict the protocol's behavior. That is the contribution of the consolidation.

Parameter	Default	Operation	Controls
`maxSynaptome`	50	STRUCTURE	Routing-table capacity (matches WebRTC connection budget)
`lookaheadAlpha`	5	NAVIGATE	Two-hop probe breadth in AP scoring
`maxHops`	40	NAVIGATE	Hard ceiling on lookup depth before failure
`epsilon`	0.05	EXPLORE	First-hop random-choice probability
`annealCooling`	0.9997	EXPLORE	Per-hop temperature decay multiplier
`annealRateScale`	1.0	EXPLORE	Multiplier on per-hop annealing trigger probability
`decayGamma`	0.995	FORGET	Per-decay-tick weight multiplier
`vitalityFloor`	0.05	FORGET	Below this, a synapse is replaceable regardless of recency
`inertiaDuration`	20	LEARN	LTP-lock window protecting freshly-reinforced synapses
`promoteThreshold`	2	LEARN	Incoming-synapse use count needed to promote to first-class
`triadicThreshold`	2	LEARN	Transit-pair count needed to fire triadic introduction
`recencyHalfLife`	50	FORGET	Tick half-life of the post-inertia recency decay

Table 2: The twelve tunable parameters of NH-1, with default values, the operation each governs, and a one-line description of what it controls.

The NX-17 → NH-1 Mapping

NX-17 carried eighteen rules across forty-four parameters (Appendix VIII traces the lineage). The audit that produced NH-1 categorized each NX-17 rule under one of three dispositions: *retained* (the rule survives, possibly with a parameter rename or a small simplification), *absorbed* (the rule's function is performed by another rule plus the vitality function, and the explicit rule can be retired without behavioral loss), or *retired* (the rule is removed; ablation showed it was either harmful or ineffective at the scales we benchmark).

Table 3 is the audit result.

The two-tier collapse is the largest single piece of the consolidation. NX-15 and NX-17 carried both a synaptome (the local-tier routing table, capacity ~ 50) and a separate *highway* tier (a small set of long-haul peers, capacity ~ 8). The highway was managed by its own admission and eviction rules, its own decay constants, and its own refresh schedule. The audit observed that the highway-tier function — "a small number of long-haul peers maintained against churn" — is exactly what the high-vitality entries of the synaptome do when traffic is dominated by long-haul lookups. The two-tier machinery was producing a structural separation that the underlying mechanism didn't actually need. Collapsing the two tiers into one removed eight parameters and the entire `_refreshHighway` pathway.

The stratum-floor absorption is the second-largest. NX-17 carried an explicit rule that prevented any XOR stratum from falling below a minimum population (default 2 entries). The rationale was reachability: if a stratum becomes empty, lookups that need to traverse it have no candidate. The absorption observation was that the

connection-layer bilateral cap (every candidate must pass `tryConnect`) plus the bootstrap's stratified fill (every stratum gets at least one entry at bootstrap) together provide the same guarantee. The explicit floor rule was a safety net that the underlying mechanisms had already made redundant.

The Markov retirement is the most interesting because it documents a mechanism that worked but didn't justify its complexity. NX-7 introduced *Markov pre-learning*: each node maintained a per-destination frequency table; destinations above a frequency threshold received a pre-emptive synapse before the first lookup actually used them. The mechanism produced measurable improvement on hot-destination workloads, but only when the same destinations were repeatedly queried within the Markov window. The ablation showed that the same workload pattern was captured by the triadic closure rule, with strictly less local state. NX-17 still carried Markov as a separate rule for backward compatibility; NH-1 retired it.

What the Consolidation Did Not Do

The consolidation did not change the protocol's behavior. The benchmark suite at 25,000 nodes shows NH-1 and NX-17 within a few percent of each other on every metric: 1.18× vs 1.27× the 3δ floor on global lookups; 32 ms vs 42 ms on 10% → 10% concentrated workload; 94.4% vs 94.8% Slice World success rate; 100% pub/sub baseline delivery for both. The full comparison is in Chapter V.

The point is that the audit was a *simplification* not an *optimization*. NX-17 was already at the state-of-the-art; the question was whether the eighteen rules were each genuinely doing different work or whether the same behavior could be expressed more economically. The answer was the latter, by a margin of six rules and thirty-two parameters.

NH-1 is not faster than NX-17. NH-1 is the version of NX-17 that *can be reasoned about* — twelve parameters spanning the behavioral envelope, one equation governing every admission decision, five operations under which every rule is classified. That clarity is the contribution.

Why NH-1, Specifically, for Production

We selected NH-1 as the deployment target for two reasons: *maintainability* and *tunability*.

Maintainability. An eighteen-rule, forty-four-parameter protocol is feasible to implement once and very hard to maintain. A twelve-

rule, twelve-parameter protocol fits in a single developer's head; the rule-to-mechanism map is small enough to verify by inspection; ablation studies on a single parameter no longer have to control for forty-three other interacting parameters. The simulator and the rest of the codebase are easier to reason about as a result. (NH-1's `NeuromorphicDHTNH1.js` is approximately 1,500 lines; NX-17's was approximately 2,400.)

Tunability. The bandwidth-distribution analysis in Chapter V identified `annealRateScale` as a single tunable knob with a clean knee — 5× volume reduction at no routing-quality cost. The same analysis was much harder to perform on NX-17 because the equivalent "annealing rate" was determined by an interaction between three different parameters (`tInit`, `annealCooling`, and the two-tier-specific `hubRefreshInterval`). Tuning one without disturbing the others required a multi-dimensional sweep. NH-1's twelve parameters are deliberately as orthogonal as we could make them; sweeping a single axis produces an interpretable result.

We continue to use NX-17 as the reference benchmark — the bar that NH-1 is asked to approach, and that future work should match or surpass. The two protocols sit side-by-side in the test harness; every benchmark in this volume reports both. The eighteen-rule version remains as the empirical ceiling of the "what can be done with this much machinery" question; the twelve-rule version is what we deploy.

The State Each Node Carries

For completeness, here is the full per-node state of an NH-1 node. Eight fields, all derived directly from the operations described in Chapter II.

- `id` — the 64-bit identifier (Chapter II).
- `lat, lng` — claimed geographic coordinates, used only at bootstrap to compute the S2 prefix and at runtime for haversine latency estimation.
- `synaptome` — a JavaScript `Map` keyed by `peerId`, holding live `Synapse` objects. Default capacity 50.
- `incomingSynapses` — a parallel `Map` of incoming-only synapses (peers that reached this node but that this node hasn't actively learned). Default capacity unbounded in practice; in *de facto* bounded by the bilateral cap.
- `transitCache` — a `Map` keyed by `(originId, nextId)` pairs, holding transit counts for triadic closure. Pruned on triadic firing.

- `temperature` — a real in $[T_{\min}, T_{\text{init}}]$, governing annealing probability.
- `simEpoch` — the local count of lookups this node has participated in. Used as the time index for inertia and recency.
- `maxConnections` — the bilateral cap, used by `tryConnect`.

That is everything an NH-1 node needs to operate. There is no global state, no configuration that varies per node beyond the identity and capacity fields, no specialized roles. Every node runs the same code with the same parameters; specialization emerges from the traffic pattern.

NX-17 rule	Disposition	NH-1 status
1. AP scoring with two-hop lookahead	retained	Same mechanism; the latency penalty is the dominant scoring signal
2. LTP reinforcement on fast paths	retained	Same mechanism; applied only on paths under the EMA threshold
3. Hop caching on successful path	retained	Same mechanism; uses vitality-based admission
4. Lateral spread to geographic neighbors	retained	Same; $k = 2$ propagation depth
5. Triadic closure on transit pairs	retained	Same; threshold = 2 pairs
6. Iterative fallback on local minimum	retained	Same; takes closest unvisited regardless of XOR progress
7. Vitality-based admission gate	retained	The single equation governing all admissions
8. Vitality-based eviction with bootstrap preference	retained	Same hedge: bootstrap synapses preferred to non-bootstrap when vitality is tied
9. Periodic decay	retained	Decay gamma = 0.995 at intervals of 100 lookups
10. Temperature annealing with reheat	retained	Reheat to 0.5 on dead-peer detection
11. Stratified bootstrap	retained	Same construction; geographic strata explicitly filled
12. Sponsor-chain join	retained	Same; piggybacks on locality-preserving routing
13. Two-tier (highway) synaptome	absorbed	The vitality-scored single tier subsumes both "leaf set" (high-vitality) and "routing table" (mid-vitality) roles. No need for an explicit highway tier
14. Stratum-floor enforcement	absorbed	Stratum diversity is preserved by the connection layer's `tryConnect` bilateral cap and the bootstrap's stratified fill. The explicit floor was redundant
15. Bespoke relay-pinning logic for pub/sub	absorbed	The standard axon-tree recruit-policy (synaptome-aware child selection) does the same work without a special-cased rule
16. `weightScale` parameter on AP scoring	absorbed	Ablation at 25,000 nodes showed no measurable effect on routing or pub/sub. Removed; AP scoring is now pure progress/latency without LTP-weight bias
17. Markov pre-learning of hot destinations	retired	The triadic-closure mechanism handles the same pattern at the network level. Removing Markov simplified the implementation without measurable performance loss
18. Load-tracked AP scoring (NX-12 experimental)	retired	Deferred to the production pathway (§VI); not present in NH-1. The bandwidth-distribution analysis (Chapter V) showed it was not needed for the routing layer at our scales

Table 3: The disposition of each NX-17 rule in NH-1. Twelve retained, four absorbed into vitality, two retired. The behavioral delta on the benchmark suite is within measurement noise; the parameter count drops from forty-four to twelve.

The Full Routing Tick

THIS CHAPTER walks through the complete behavior of a single lookup in NH-1, hop by hop, with each of the five operations of Chapter II called out at the moment it fires. The treatment is at the level of pseudocode: detailed enough that a reader can implement it from this chapter alone, abstract enough that the implementation language is not assumed.

The reference implementation in JavaScript (`src/dht/neuromorphic/NeuromorphicDHTNH1.js`) follows this pseudocode line-for-line; cross-references in the margins point to specific source-code symbols where useful.

The Lookup Entry Point

A lookup is invoked as `lookup(sourceId, targetKey)`, where `sourceId` is the originating node and `targetKey` is the 64-bit key being sought. The lookup returns a record `{path, hops, time, found}` where `path` is the sequence of node IDs traversed, `hops` is the path length minus one, `time` is the cumulative latency, and `found` is true if the target was reached.

The simEpoch advance happens once per lookup. The per-decay-tick weight multiplication happens every 100 lookups (the DECAY_INTERVAL constant); the rationale is that decay is cheap to amortize but doesn't need to fire every hop. The trace structure — a list of `(fromId, synapse)` pairs — is what the LTP reinforcement walk uses post-lookup (LEARN); we accumulate it as we go.

The Hop Body

The body of the loop is where every operation other than periodic decay actually fires. The structure is in five phases.[33]

The hop body is deliberately structured so that each of the five operations is identifiable as its own phase or sub-phase. A reader who has internalized Chapter II can read the algorithm as: *at every hop, the protocol acts on present input (Phase 4 chooses the hop), learns*

[33] In the source, the phases are not always presented in this exact order — some interleave for performance reasons — but the pseudocode order is the conceptual one, and the behavior is identical.

Algorithm 2: lookup(sourceId, targetKey) — top level

```
source ← nodeMap.get(sourceId)
if source = null or not source.alive then
    return null
end if
simEpoch += 1                                   ▷ advance the local clock
if lookupsSinceDecay ≥ D_interval then          ▷ D_interval = DECAY_INTERVAL
    tickDecay()                                 ▷ periodic weight decay (FORGET)
    lookupsSinceDecay ← 0
end if
path ← [sourceId]; trace ← [ ]; queried ← {sourceId}
currentId ← sourceId; totalTimeMs ← 0; reached ← false
for hop ← 0 to maxHops −1 do
    hop body — see Algorithm 3
end for
if reached then
    post-lookup learning — see Algorithm 4
end if
return {path, hops = |path| − 1, time = totalTimeMs, reached}
```

from the choice (Phase 5a–c), forgets along the way (Phase 2 evicts dead, periodic decay), and explores occasionally (Phase 5d, plus the epsilon-greedy in Phase 4). STRUCTURE doesn't fire per hop; it fires at bootstrap (§III).

Selecting the Next Hop in Detail

Phase 4 is the protocol's main act-on-input decision. The selection runs through five mechanisms in priority order:

1. *Direct shortcut.* If *current.synaptome* contains a synapse for *targetKey*, use it. The lookup completes in one more hop.
2. *Epsilon-greedy first hop.* If *hop* = 0 and Random() < `epsilon`, pick a candidate uniformly at random.
3. *AP scoring with two-hop lookahead.* For each of the top `lookaheadAlpha` candidates by single-hop AP score, extend the search one step into the candidate's known synaptome and compute a joint two-hop AP. Pick the candidate whose two-hop projection scores best.
4. *(Iterative fallback handled in Phase 3 — invoked if Phase 1 produces no candidates.)*
5. *Termination.* If no candidate exists even after fallback, break the loop. The lookup returns *found* = false.

Algorithm 3: hop body, run for each hop $\in [0, \texttt{maxHops})$

current ← nodeMap.get(*currentId*)
if *current* = null **or not** *current.alive* **then break**
end if
currentDist ← *current.id* ⊕ *targetKey*
if *currentDist* = 0 **then** *reached* ← true; **break**
end if
Phase 1: NAVIGATE *— collect candidates*
candidates ← [ ]; *deadSynapses* ← [ ]
for each *s* **in** *current.synaptome* **do**
    if (*s.peerId* ⊕ *targetKey*) ≥ *currentDist* **then continue** ▷ no XOR progress
    end if
    peer ← nodeMap.get(*s.peerId*)
    if *peer* alive **then** *candidates*.push(*s*)
    else*deadSynapses*.push(*s*); *s.weight* ← 0
    end if
end for
for each *s* **in** *current.incomingSynapses* **do**
    if (*s.peerId* ⊕ *targetKey*) < *currentDist* **and** alive **then** *candidates*.push(*s*)
    end if
end for
Phase 2: FORGET/EXPLORE *— dead-peer reheat & evict-replace*
if |*deadSynapses*| > 0 **then**
    current.temperature ← max(*current.temperature*, T_{reheat})
    for each *deadSyn* **do**
        repl ← evictAndReplace(*current*, *deadSyn*)
        if *repl* ≠ null **and** progress **then**
            candidates.push(*repl*)
        end if
    end for
end if
Phase 3: NAVIGATE *— iterative fallback*
if |*candidates*| = 0 **then**
    candidates ← [closestUnvisitedPeer()]
    if |*candidates*| = 0 **then break**
    end if
end if
Phase 4: NAVIGATE *— select next hop*
nextSyn ← choose by direct-shortcut → epsilon-greedy → AP score with two-hop lookahead
nextId ← *nextSyn.peerId*
nextNode ← nodeMap.get(*nextId*)
Phase 5: LEARN & EXPLORE *— on-the-fly updates*
*(a) Incoming-synapse promotion if applicable (*LEARN*)*
*(b) Hop caching: install target in current (*LEARN*)*
*(c) Triadic-transit recording (*LEARN*)*
*(d) Anneal cooling & probabilistic anneal (*EXPLORE*)*
(e) Update path / trace / totalTimeMs
currentId ← *nextId*

The two-hop AP formula is given in equation 3; the extension to two hops uses the candidate's known synaptome entries (no extra round trip) to project the second-hop position.

Phase 5 in Detail: On-the-Fly Updates

The five sub-steps of Phase 5 are where most of the protocol's *adaptation* happens. We elaborate each.

(a) Incoming-synapse promotion. If *nextSyn* is an incoming synapse (came from *current.incomingSynapses*), increment its `useCount`. If the count crosses `promoteThreshold`, promote it to a first-class synapse in *current.synaptome* via addByVitality(), weight 0.5, fresh inertia.

(b) Hop caching. If *currentId* $\neq$ *targetKey*, the current node installs a synapse pointing to *targetKey* via addByVitality() with weight 0.5 and fresh inertia. Lateral spread also fires here: the top `LATERAL_K` geographically-nearest peers in *current.synaptome* each get a recursive hop-cache invocation (depth-1 only; no further recursion).

(c) Triadic-transit recording. If *currentId* $\neq$ *sourceId*, increment the transit count for the pair (*sourceId*, *nextId*) in *current.transitCache*. If the count crosses `triadicThreshold`, fire the introduction: introduce(*sourceId*, *nextId*) — which adds a synapse from *source* to *next*, weight 0.5, via addByVitality() at the source's synaptome.

(d) Anneal cooling and probabilistic anneal. Multiply *current.temperature* by `annealCooling`, floored at $T_{\min}$. With probability *current.temperature*· `annealRateScale`, invoke tryAnneal(*current*). The inner mechanism of tryAnneal is described below.

(e) Path / trace / time updates. Append *nextId* to *path*. Append (*currentId*, *nextSyn*) to *trace*. Increment *totalTimeMs* by *nextSyn.latency*. Mark *nextId* as queried.

The Vitality-Based Admission Gate

Many of the operations above call addByVitality(). This is the single function that admits new synapses to a synaptome under the vitality rule.

The bootstrap-preference hedge (line 12 of Algorithm 5) is worth dwelling on. Bootstrap synapses are the structural backbone of the routing table; evicting them too aggressively breaks the stratum-coverage guarantee that the bootstrap process worked to establish. The hedge says: prefer to evict a non-bootstrap synapse if one is available. Only if every candidate is a bootstrap entry does the gate fall back to evicting the lowest-vitality bootstrap entry. In practice this rarely fires; new entries from triadic / hop-cache / lateral-spread

Algorithm 4: addByVitality(node, newSyn)

```
peer ← nodeMap.get(newSyn.peerId)
if peer ≠ null and not node.tryConnect(peer) then
    return false                          ▷ bilateral cap refused; STRUCTURE
end if
if |node.synaptome| < maxSynaptome then
    node.addSynapse(newSyn)
    return true
end if
synaptome at capacity — vitality-based eviction
victim ← null; minV ← ∞
victimAny ← null; minVAny ← ∞
for each s in node.synaptome do
    if s.inertia > simEpoch then continue          ▷ LTP-locked; protected
    end if
    v ← vitality(node, s)
    if v < minVAny then minVAny ← v; victimAny ← s
    end if
    if not s.bootstrap and v < minV then minV ← v; victim ← s
    end if
end for
evictTarget ← victim ≠ null ? victim : victimAny
if evictTarget = null then return false
end if
node.synaptome.delete(evictTarget.peerId)
node.addSynapse(newSyn)
return true
```

quickly populate the synaptome with non-bootstrap material, and bootstrap entries become the longstanding anchors that never get evicted.

The Anneal Step

The anneal step is the protocol's main exploration mechanism. It fires probabilistically, governed by the local temperature and the `annealRateScale` master volume control.

Algorithm 5: tryAnneal(node)

```
1: Find weakest non-LTP-locked synapse in node.synaptome
2: victim ← null; minW ← ∞
3: for each s in node.synaptome do
4:     if s.inertia > simEpoch then continue
5:     end if
6:     if s.weight < minW then minW ← s.weight; victim ← s
7:     end if
8: end for
9: if victim = null then return
10: end if
11: Identify under-represented stratum group
12: (count strata occupancy; pick the lo-hi range with min count)
13: candidate ← localCandidate(node, lo, hi)
14: if candidate = null then return
15: end if
16: Atomic 1-for-1 exchange under bilateral cap
17: node.synaptome.delete(victim.peerId)
18: node.connections.delete(victim.peerId)
19: if not node.tryConnect(candidate) then return ▷ candidate's cap
    refused; abort
20: end if
21: node.addSynapse(Synapse(candidate, weight = 0.1))
```

The 1-for-1 exchange is intentional: anneal does not grow the synaptome, it *rotates* an entry. The peer count stays constant; the under-represented-stratum bias gives the rotation its diversifying effect over time. The starting weight of 0.1 is low enough that the new entry is itself a candidate for eviction by the next anneal — only entries that the actual traffic reinforces accumulate weight beyond the floor.

The `localCandidate`() function scans the 2-hop neighborhood (the union of *node*'s synaptome peers' synaptomes) for a peer in the target stratum range. It samples up to ANNEAL_LOCAL_SAMPLE candidates and returns one uniformly. This is the `local_probe` traffic source that dominates the bandwidth analysis in Chapter V.

Post-Lookup Learning

If the lookup reached the target, NH-1 walks the trace and applies LTP to every synapse on the path that beat the running EMA of recent path latencies.

Algorithm 6: post-lookup learning, fires only if reached

```
hopCount ← |path| − 1
Update EMA of recent path latencies
emaTime ← 0.9 · emaTime + 0.1 · totalTimeMs
if totalTimeMs ≤ emaTime then
    for each (fromId, synapse) in trace from end to start do
        node ← nodeMap.get(fromId)
        syn ← node.synaptome.get(synapse.peerId)
        if syn ≠ null then
            syn.weight ← min(1.0, syn.weight + 0.05)
            syn.inertia ← simEpoch            ▷ enter LTP-lock window
            syn.useCount += 1
        end if
    end for
end if
```

The trace is walked end-to-start so that the synaptic-tagging mechanism behaves correctly: the more recent reinforcements (toward the destination) fire first, establishing their LTP-lock, before earlier ones. The EMA threshold ensures that LTP fires only on paths that beat the running average; if every successful path triggered reinforcement, slow paths would fight fast paths in the synaptome, and the learning would not converge on the fast routes.

The Bootstrap Tick

A new node's bootstrap is the only time STRUCTURE operations dominate. The sequence:

1. Compute the node's S2 cell from its claimed lat/lng.
2. Construct the node ID: $nodeId = S2prefix \parallel H_{56}(publicKey)$.
3. Pick a sponsor (a known live peer; in production this is typically a published rendezvous list).
4. Route a self-lookup through the sponsor toward *nodeId*. Each node along the path returns a row of its synaptome.
5. Run the stratified-bootstrap allocator (§II) over the union of returned candidates. Allocate 50 slots: k per geographic stratum, 1 per non-geographic stratum, remainder uniformly random.

6. Materialize each slot by calling addByVitality() with weight 0.5, fresh inertia, and the bootstrap flag set.
7. Propagate the bidirectional handshake: each peer added learns of the new node as an incoming synapse.

After this completes, the node is operational. Its very first lookup will route through a synaptome shaped by its sponsor's locality, with explicit fill on the geographic strata so that intra-region traffic finds nearby peers immediately. The learning mechanisms then take over on subsequent traffic.

The bootstrap pseudocode is more involved than the steady-state hop body; the full algorithm is in Appendix VIII.

The Full Pub/Sub Tick

The pub/sub layer sits on top of the routing layer of Chapter III and uses two new primitives: `routeMessage` (route a typed message toward a topic ID, intercepting at the first axon along the path) and `sendDirect` (deliver a message to a specific known peer). The axonal tree's subscribe / publish / refresh behaviors are implemented entirely in terms of these two primitives plus a per-node *role* table that tracks topic membership.

Per-Node Pub/Sub State

NH-1 augments the per-node state of §III with the following pub/sub-specific fields. Each is initialized empty when the node joins.

- `routedHandlers` — a map from message type to handler function for routed messages received at this node. AxonManager populates this at axonFor() time.
- `directHandlers` — the same for direct messages.
- `axonsByNode` — a map from `NeuronNode` to its `AxonManager` instance. Lazy: an entry is created the first time this node calls `axonFor()`.
- Per topic, in the AxonManager: a *role* record holding `children` (a map of subscriber IDs to metadata), `lastSeenTs` (the timestamp of the most recent publish), and the `replayCache` ring buffer.

The AxonManager is the per-node object that owns all pub/sub state for that node. A single node can serve as an axon for many topics simultaneously; each topic has its own role record inside the same AxonManager.

Topic ID Construction

The topic ID is constructed from the topic name and the publisher's S2 prefix, as described in Chapter II. The `usesPublisherPrefix` flag

on the protocol activates the `topicIdForPrefixed` variant of the construction: the topic name carries an explicit `@CC/` prefix, and the protocol uses the first eight bits of the publisher's claimed cell as the high bits of the topic ID.

Algorithm 7: topicId = topicIdForPrefixed(name)

```
Parse @CC/domain/event into the cell shortcode and the rest
cellPrefix ← S2.cellFromShortcode(CC)
nameHash ← H_56(restofname)
return (cellPrefix ≪ 56) | nameHash
```

The construction is deterministic. Both publisher and subscriber compute the same topic ID without coordination, given only the topic name string. The publisher's S2 prefix encoded in the high bits causes routed subscribes to converge on a node geographically close to the publisher — typically also close to the subscribers, since real-world topic memberships are geographically clustered.

Subscribe

A subscribe operation routes a typed message `pubsub:subscribe` toward the topic ID. The `routeMessage` primitive does the actual routing; an intermediate node intercepts when it is already an axon for the topic.

Algorithm 8: subscribe(node, topicId, subscriberMeta)

```
result ← routeMessage(node, topicId, `pubsub:subscribe', subscriberMeta)
if result.consumed then
    some axon along the path accepted; subscriber is in the tree
else if result.terminal then
    the routed walk reached a local terminal with no existing axon
  for this topic — the terminal node becomes the new root
    terminalNode ← nodeMap.get(result.atNode)
    terminalNode.axonFor(topicId).openRole()
    terminalNode.axonFor(topicId).addChild(subscriberMeta)
else
    exhausted — failure mode; subscriber retries
end if
```

The interception logic. When a routed message visits an intermediate node, `routeMessage` delivers the message to the node's registered routed-message handler for the message type. The handler returns either `` `forward' `` (continue routing) or `` `consumed' `` (stop here). The `pubsub:subscribe` handler checks: *is this node already an axon for this topic?* If yes, it adds the subscriber to its children, returns `` `consumed' ``, and routing terminates. If no, it returns `` `forward' `` and the message continues toward the topic ID.

The handler is registered via `onRoutedMessage` when the node's AxonManager is initialized.

Publish

A publish operation routes a `pubsub:publish` message toward the topic ID. The first axon along the path intercepts and fans out.

Algorithm 9: publish(node, topicId, payload)

```
result ← routeMessage(node, topicId, `pubsub:publish', payload)
    ▷ first axon intercepts; fan-out happens inside the handler
```

The fan-out logic, inside the handler at the intercepting axon:

Algorithm 10: publish handler at an axon

```
role ← this.rolesFor(topicId)
role.replayCache.push(payload)
role.lastSeenTs ← now()
for each (childId, childMeta) in role.children do
    sendDirect(this, childId, `pubsub:deliver', payload)
end for
return `consumed'
```

The fan-out uses `sendDirect` (point-to-point), not `routeMessage` (routed). The axon already knows the children's identities, so direct delivery is correct.

Children that are sub-axons recurse. If a child is itself an axon (registered to handle the topic), its `pubsub:deliver` handler does the same fan-out as above. The result is a tree-structured fan-out where each level handles up to `maxDirectSubs` children before delegating further.

Refresh: The Self-Healing Mechanism

The refresh tick fires on every member of the tree (subscribers and axons alike) every `refreshIntervalMs` milliseconds. It is the mechanism that heals the tree under churn.

Why this is self-healing. The re-subscribe is exactly the same routed message that built the tree initially. If the upstream parent is alive, the re-subscribe is intercepted by the parent, which already has this child in its children list — the child's `lastRefreshTs` is updated, no other state changes. If the parent is *dead*, the re-subscribe routes *around* the dead parent (the routing layer's dead-peer detection takes care of this), lands at whichever live ancestor is now closest to the topic ID, and that ancestor accepts the re-subscribe. The orphaned child is now connected to its new parent.

Algorithm 11: refreshTick(node)

```
for each topic this node is subscribed to or serves as axon for do
    Send a re-subscribe to keep the upstream link alive
    subscribe(node, topicId, thisnode′smeta)
    TTL-sweep our own children
    for each child c in our role for topicId do
        if now() − c.lastRefreshTs > maxSubscriptionAgeMs then
            drop c from the children list
        end if
    end for
end for
```

The TTL sweep is the corresponding garbage-collection: a child that has not refreshed within `maxSubscriptionAgeMs` is assumed to have left without unsubscribing, and is dropped.

There is no separate failure-detection machinery. There are no heartbeats independent of the re-subscribe. There is no parent-pointer tracking that could become inconsistent. *The same operation that builds the tree repairs it.* This is the architectural invariant that produces 100% pub/sub recovery under 5% instantaneous churn.

Branching on Overflow: Recruit and Relay

When an axon's direct-child count exceeds `maxDirectSubs`, the overflow subscriber is delegated to a *sub-axon*. NH-1 inherits two policies for picking the sub-axon, both implemented as callbacks on the AxonManager.

Recruit policy (existing-child promotion). If at least one of the existing children is a synaptome peer of this axon with high LTP weight, promote it to a sub-axon and have it adopt the overflow subscriber. The high-vitality synaptome peer is the most reliable choice for sustained delivery. Implementation:

Algorithm 12: pickRecruitPeer(role, meta, subscriberId)

```
for each child c in role.children do
    if c is in our synaptome and c has high LTP weight then
        return c                          ▷ recruit existing child as sub-axon
    end if
end for
return XOR-closest existing child to subscriber's ID
```

Relay policy (external promotion). If no existing child is suitable for promotion, pick an external synaptome peer (XOR-closest to the new subscriber's ID, not yet in the children) and recruit it. The recruited peer may then adopt multiple existing children whose IDs point in the same direction.

Algorithm 13: pickRelayPeer(role, subscriberId, forwarderId)

```
1: best ← null; bestDist ← ∞
2: for each synapse s in this node's synaptome do
3:     if s.peerId already in role's children or = forwarderId or
   = subscriberId then continue
4:     end if
5:     d ← s.peerId ⊕ topicToBigInt(subscriberId)
6:     if d < bestDist then bestDist ← d; best ← s.peerId
7:     end if
8: end for
9: return best
```

The two policies together keep the tree's branching factor bounded at any single node by `maxDirectSubs`, with the new sub-axon chosen so that geographic / topological clustering of subscribers ends up handled by the same sub-axon. Inherited from NX-15.

The Replay Cache

Each axon's role for each topic carries a bounded ring buffer of recently-published messages. Default capacity is 100 messages. The buffer is updated on every publish (line 2 of the publish handler above) and consulted on every subscribe whose `lastSeenTs` is set.

Algorithm 14: subscribe handler with replay request

```
1: Standard subscribe handling — add child, etc.
2: if subscriberMeta contains a lastSeenTs then
3:     for each cached message m in role.replayCache do
4:         if m.ts > subscriberMeta.lastSeenTs then
5:             sendDirect(this, subscriberId, `pubsub:replay', m)
6:         end if
7:     end for
8: end if
9: return `consumed'
```

The mechanism gives a late-joining subscriber a bounded recovery of the recent past of the topic. "Bounded" is the operative word: the ring buffer holds the last 100 messages (or whatever fits in `cacheLifetime` seconds), not the entire history. A subscriber that has been offline for longer than the cache holds is told nothing about the lost interval — the gap is irrecoverable from the pub/sub layer alone.[34]

[34] This is intentional. Pub/sub is a real-time message bus, not a log. If clients need durable history they should use a different primitive layered on top — one that either pulls from a log-backed source on resume, or carries a per-message ack protocol that reports gaps.

The Two Underlying Primitives

The entire chapter to this point has used two primitives: `routeMessage` and `sendDirect`. We give the sketches here for completeness; the full

implementations are in `NeuromorphicDHTNH1.js`.

routeMessage(originNode, targetId, type, payload). Walks a typed message hop-by-hop toward `targetId` using greedy single-hop routing (no two-hop lookahead, since pub/sub doesn't benefit from it). At each hop, the node's registered routed-message handler for `type` can intercept (`` `consumed' ``) or forward (`` `forward' ``). On reaching a local terminal (no hop offers further XOR progress), runs a 2-hop terminal-globality check[35] — if a 2-hop peer is closer to the target than the current node, take it; otherwise this node is the global terminal. Returns `{consumed, atNode, hops, terminal?, exhausted?}`.

sendDirect(fromNode, peerId, type, payload). Delivers `payload` directly to a known peer's registered direct-message handler. Used for fan-out from axons to children. NH-1 implements an iterative drain queue under `sendDirect`: a handler triggered by one `sendDirect` can itself call `sendDirect`, and so on; rather than building up a recursive call stack (which blows past Node's $\sim$ 10K stack limit on deep trees), the nested calls enqueue items in a FIFO drain that the outer `sendDirect` processes iteratively. From the caller's perspective, all handlers triggered by a top-level publish complete before `sendDirect` returns; the drain queue is an implementation detail.

[35] The terminal-globality check is necessary because greedy routing can converge on different local minima from different sources, fragmenting the tree. The 2-hop scan asks: "is there *any* peer reachable in two hops that is closer to the target than I am?" If yes, take that path. If no, this is a *global* terminal, and an axon role can be opened here.

Operational Tunables

The pub/sub layer carries five operational parameters in addition to the twelve routing parameters of §III:

Parameter	Default	Controls
`maxDirectSubs`	32	Max direct children per axon before recruit-on-overflow
`minDirectSubs`	4	Min direct children per axon before relinquishing role
`refreshIntervalMs`	30000	Per-member refresh interval (re-subscribe)
`maxSubscriptionAgeMs`	90000	TTL for child entries before sweep
`replayCacheSize`	100	Ring buffer messages per axon-role

These are operational tunables, not behavioral parameters in the same sense as the routing parameters. They control timing and capacity choices that depend on the deployed workload (publish rate, expected churn, message size); they do not change what the protocol does, only how often and how much.

Part IV

The Simulator

Philosophy: Accurate Analogy + Adversarial Laboratory

The simulator is essential to the design. Its task is two-fold: provide an accurate analogy to the real world, and try to break the protocol.

THE SIMULATOR is not a thing we built and put away once we had measurements. It is part of the design itself — the engineering bible's lab notebook — and it has two jobs, both ongoing.

Its first job is to provide an *accurate analogy* of the real-world conditions the protocol must operate under. Real internet routing, real geographic latency, real WebRTC connection caps, real churn. Where the analogy is faithful, every quantitative claim in this volume rests on it; where the analogy is incomplete, the gap is itself an artifact we describe (Chapter IV). The simulator is honest by construction — it does what it does, and we report what it does.

Its second job is to *try to break the protocol*. The benign tests are not enough. Uniform global lookups against a uniform global node distribution will succeed at any reasonable DHT design; success on those tests is necessary but not sufficient. The simulator's adversarial workload — Slice World partitions, sustained churn, Zipf-distributed pub/sub, the bandwidth-instrumentation reality check — is what gives the protocol's claims their force. We pass tests that we *designed* to make the protocol fail.

This chapter is the philosophy. The next two chapters are the specifics: what the simulator models with what fidelity (Chapter IV) and what tests we run to break it (Chapter IV).

Why a Simulator and Not a Testbed

THE TWO MAIN ALTERNATIVES to a simulator for distributed-systems research are a real testbed (PlanetLab, EmuLab, or a constellation of cloud-rented VMs) and a closed-form analytical model. Both have been used productively in this field. We chose a simulator for a

specific reason: it lets us run experiments at scales where real testbeds become impractical *and* test conditions where analysis becomes intractable.

Scale. A real testbed at 50,000 nodes is a non-trivial undertaking: not just the cost of the machines, but the operational overhead of orchestrating that many participants, collecting metrics from each, and resetting state between runs. Most published DHT research has been measured at the $\sim$ 100–1,000 node scale, with extrapolation to larger networks. Our simulator runs the full 50,000-node benchmark suite to convergence in a few hours on a laptop, with full state observability and exact reproducibility. That is the scale at which the protocol's claims actually need to hold.

Adversarial conditions. On a real testbed, you cannot ask: "what if half the nodes vanish at once?" — not without disrupting other tenants, or paying for 50,000 machines you will discard immediately. On a simulator, you can. The Slice World test (Chapter IV) takes a 25,000-node network and partitions it into hemispheres connected only by a single bridge node, then asks the protocol to route between hemispheres. This kind of test is essentially impossible on a testbed; on a simulator it is one configuration flag.

Reproducibility. The simulator is deterministic given a seed. Any measurement in this document can be reproduced exactly from the open-source code, the parameters listed in the run, and the seed. Real-testbed measurements have natural variance from cross-traffic, scheduling jitter, and network conditions that no researcher can fully control; on a simulator, we report the mean over hundreds of runs and the variance is small enough to ignore. *The price is that the simulator only models what we explicitly tell it to model, and the rest of this Part is honest about that price.*

The Engineering Bible's Lab Notebook

Every quantitative claim in this volume comes from the simulator. Average lookup latency, hop counts, success rates, Gini coefficients, bandwidth distribution, churn recovery times — all measured by the same code base, against the same N-DHT implementation, run with the parameters listed in each chapter.

This puts the simulator at the heart of the document's credibility. The whitepaper's accuracy is bounded by the simulator's faithfulness. So Chapter IV has to be brutal about what we *do* model and what we *don't*: the missing models are part of the document, not a footnote in the conclusion.

Three categories:

Modeled accurately. Geographic latency from haversine distance plus per-hop processing cost; XOR routing; bilateral WebRTC connection caps; topology effects from S2-prefix node IDs; simulator-time advancement in lookup ticks; LTP / decay / inertia at full resolution.

Modeled approximately. Churn (instantaneous removal and re-insertion, no graceful-exit signaling); pub/sub topic membership (uniformly-sampled subscriber populations, not Zipf-distributed); message delivery (reliable; no packet loss beyond explicit dead-peer detection).

Not modeled. WebRTC connection setup time (ICE/STUN/TURN/DTLS overhead); RPC timeouts and asynchronous black holes; latency jitter from queueing and bufferbloat; bandwidth saturation and congestion (until v0.70.04 added per-node send/receive instrumentation, this was the most critical omission — Chapter V); Byzantine peer behavior (Sybil attacks, eclipse attacks, prefix forgery).

Each of the "not modeled" items is a deferred concern. They are catalogued in Chapter VI (the red team) and have explicit pathways for closure in Chapter VI (the production pathway). The point is that they are *labeled*: we do not claim measurements that depend on them, and where claims of behavior under such conditions appear (e.g., "the protocol will recover from a 30-second connection-setup delay"), we mark them as projections, not measurements.

Try to Break It as a Design Discipline

The second job of the simulator is the harder one to do right. A simulator that only runs benign tests will produce optimistic numbers; a research field full of such simulators (and we have been in such a field) will collectively publish protocols that don't survive contact with reality. The discipline that protects against this is to design tests specifically intended to make the protocol *fail*.

The test suite (Chapter IV) is organized around this discipline. For each test we ask: *what hypothesis am I trying to falsify?*

- Slice World tests the hypothesis that the learning mechanisms cannot recover from a network partition. (The test passes; the hypothesis is falsified at $\sim 94\%$ cross-partition success.)
- Sustained-churn tests the hypothesis that the protocol cannot maintain 100% pub/sub delivery under continuous node turnover. (The test passes at 5% instantaneous churn; fails gracefully past 20% sustained.)
- Bandwidth instrumentation tests the hypothesis that adaptive routing concentrates load onto a small number of nodes (the success-

disaster failure mode). (The test falsifies it: N-DHT distributes more broadly than Kademlia at every tested scale.)

- The `geoBits = 0` ablation tests the hypothesis that all of N-DHT's performance comes from the geographic prefix and not from the learning. (The test falsifies it: with zero geographic information, NX-17 still routes 26% faster than Kademlia.)

Each test was added to the suite after some specific moment in the design process where we wanted to break the protocol. The discipline is generative: every time we make a new claim, we should ask what test would falsify it, and if no such test exists, we should design one. This is the inverse of "measurement-driven development"; we call it *falsification-driven development.*

The bandwidth-concentration episode is the canonical example. For most of the protocol's development, the simulator treated all nodes as having infinite bandwidth: every hop cost 10 ms regardless of how many concurrent messages a relay was processing. This was a known omission, and the red-team analysis (Chapter VI) ranked "bandwidth saturation" as a deploy blocker. The hypothesis the red team raised was that N-DHT's adaptive routing would *amplify* hotspot formation: LTP would reinforce a fast peer until it saturated; AP scoring would route around it when it slowed; LTP would re-reinforce it when it recovered, creating oscillation. We had no way to test that hypothesis until we instrumented the simulator with per-node send/receive counters.

The instrumentation took two days to write (§V describes it in detail). The first test it produced *empirically falsified the red-team's hypothesis*: N-DHT distributes load more broadly than Kademlia at every tested scale. The simulator's job was to break the protocol; in this case, it broke the assumption that we needed new mechanism to address Issue #3. The deploy blocker was reclassified.

This is what we mean when we call the simulator part of the design. It is not a tool we use after the protocol is finished; it is the iterative loop in which the protocol is shaped. Every claim in this volume passed through the loop at least once; many of the rules in Chapter III survived the consolidation specifically because the simulator showed they mattered, and others were retired because the simulator showed they didn't.

The Open-Source Simulator

THE SIMULATOR[36] is part of the N-DHT codebase. Anyone can clone the repository, run the test suite, and reproduce every measure-

[36] Source available at the URL listed in the References. Approximately 25,000 lines of JavaScript; runs in the browser via Vite-served Web Workers, or as a Node.js process. Same code, both targets.

ment in this document. The parameters used for each measurement are recorded in the benchmark CSV alongside the numbers, so a third party can verify both the result and the conditions under which it was obtained.

This commits us to a discipline that is easy to forget about: every claim in this document is, in principle, falsifiable. A reader who suspects a measurement is wrong can run the same test on the same code with the same parameters and either confirm or contradict the number. We have, on multiple occasions during the development of this protocol, retracted findings because of exactly this kind of self-audit. The bandwidth analysis is the most recent example; Appendix VIII catalogues the full sequence.

The simulator is also a research tool independent of N-DHT. The code base supports running comparison protocols (Kademlia, G-DHT, NX-17) against the same workloads with the same instrumentation, and adding a new protocol is a matter of implementing the five `DHT` interface methods (`addNode`, `removeNode`, `buildRoutingTables`, `lookup`, `bootstrapJoin`) plus optional pub/sub hooks. We have used this affordance to run Vivaldi-style ablations and Coral-style nested-cluster experiments without disturbing the main protocol.

The next two chapters describe the simulator concretely: what it models (§IV) and what tests it runs (§IV).

What the Simulator Models, and What It Does Not

THIS CHAPTER is the methodological honesty chapter. Every quantitative claim in this volume rests on the simulator; the simulator's faithfulness bounds the claims' credibility. For each modeled phenomenon we describe the model; for each *not*-modeled phenomenon we describe the omission, the deferred concern, and the pathway to closing the gap.

The categories we walk through, in order: latency model, topology model, churn model, message-delivery model, connection model, bandwidth model, and adversarial-behavior model. Each gets a section.

Latency: Haversine Plus Per-Hop Cost

What we model. Per-hop latency is computed deterministically from the haversine great-circle distance between the two nodes' claimed coordinates, using a fiber-optic propagation constant ($c \approx$ 200,000 km/s, accounting for refractive index) plus a fixed per-hop processing cost.[37]

$$latency(A, B) \;=\; 2 \cdot \left(\frac{haversine(A, B)}{c} + \delta_{\text{node}} \right)$$

The factor of 2 accounts for round trip; δ_{node} defaults to 10 ms.

What we do not model. *Latency jitter.* Real internet round-trip times fluctuate ±30% from queueing, bufferbloat, and asymmetric paths. Our latency is monotone and clean. The protocol's exponential moving average of recent path latencies (used as the LTP reinforcement threshold) has an easy job in the simulator; high-frequency noise could prematurely decay good synapses or promote lucky-but-unstable ones. The deferred concern is documented in Chapter VI as Tier 1 Issue #4 and addressed in Chapter VI as the jitter-injection extension to the simulator.

What we do not model. *Asymmetric latency.* A real internet path from A to B may take a different route than the path from B to A, with different queueing characteristics on each. Our model is

[37] The 10 ms-per-hop processing constant is calibrated from published WebRTC RTT measurements at small distance — two nodes in the same data center exhibit roughly 10 ms RTT before great-circle distance becomes the dominant factor. The constant is conservative; real-world processing costs vary from $\sim$ 5 ms (server-class) to $\sim$ 30 ms (mobile under poor radio conditions).

symmetric. The implication is that the `forward` and `reverse` synapse fields collapse to the same value; no protocol mechanism distinguishes them. Tier 2 Issue #8 in the red-team analysis.

Topology: Uniform Land Distribution

What we model. Nodes are placed at uniformly-distributed random coordinates over the Earth's land surface (a land-detection function rejects ocean coordinates). The S2 cell prefix is computed from the coordinates and stamped into the high bits of the node ID. The total number of nodes is a configuration parameter; experiments in this volume use 5,000, 10,000, 25,000, and 50,000.

What we do not model. *Population-weighted node distribution.* Real-world peer-to-peer node distribution correlates with population centers and broadband access; nodes are not uniformly distributed across land. We expect the qualitative behavior of the protocol to be insensitive to this (locality benefits accrue regardless of the population distribution), but the quantitative gap between regional and global latency may differ. We have not measured this directly.

What we do not model. *Node-class heterogeneity at high resolution.* The simulator supports a single "highway-percent" parameter to bias a fraction of nodes toward unrestricted connection caps (server-class deployment); this is binary (browser-class vs. server-class). Real deployments will exhibit a continuous spectrum of capacity and reliability tiers. The current ablation (Chapter V) is sufficient for first-order conclusions but does not capture the full heterogeneity.

Churn: Instantaneous Removal and Re-Insertion

What we model. A churn event removes a percentage of live nodes and replaces them with fresh nodes that bootstrap through random sponsors. "Removed" means: `alive` is set to false, all incoming references are observable to the remaining network as dead synapses on first attempt. "Inserted" means: a new node is created with fresh ID, the bootstrap operation runs as in §III.

The simulator supports two churn modes:

- *Instantaneous churn* — a single event at a specific simulation time. "5% instantaneous churn" means one event removes and replaces 5% of the live nodes, then the test continues against the new population.
- *Continuous churn* — a churn event fires every 5 simulation ticks at a configurable rate. "1% continuous churn" means each event replaces 1% of the live nodes, so the cumulative turnover grows

linearly with time.

What we do not model. *Graceful exit signaling.* Real clients leaving a network might announce their intent ("I am disconnecting"); our churn is silent removal. The protocol's recovery mechanism (dead-peer detection) covers both cases — a graceful exit and an abrupt disappearance look the same from the routing layer's perspective — but a real protocol can exploit the graceful case for proactive handoff. We do not.

What we do not model. *Clustered churn.* A real network outage typically affects a geographic region (a power failure, a transit cable cut, a regional ISP problem) rather than a uniformly-sampled subset of nodes. Our churn is uniformly sampled. We have run informal experiments with regional churn (removing all nodes in a specific S2 cell) and the protocol recovers — the partition-recovery mechanism in Slice World generalizes — but the formal benchmark uses the uniform-sample version.

Message Delivery: Reliable, with Explicit Dead-Peer Detection

What we model. A message sent from A to B is delivered if B is alive at the moment of delivery. The `SimulatedNetwork.send` primitive is synchronous and reliable: it returns immediately with the response, accounting for the latency from the model in §5.1.

What we do not model. *Packet loss.* Real internet delivery has a small but nonzero loss rate, typically 0.1–1%. We do not lose packets except in the explicit "the peer is dead" case. The protocol's response to single-message loss is the same as its response to a dead peer (timeout + iterative fallback); we do not test the regime where the loss rate is high but bounded.

What we do not model. *Asynchronous black holes — RPC timeouts.* A real RPC to a peer that has gone silent can stall for seconds before the caller knows the peer is dead. Our simulator detects death instantaneously: `network.send` to a dead peer returns immediately with a "dead" indication. This is a fundamental simplification. The protocol's churn-recovery measurements in this document assume instant detection; a real deployment will see multi-second timeout windows during which messages are lost. The deferred concern is Tier 1 Issue #2.

What we do not model. *Message corruption.* We assume bytes delivered intact. Real-world deployments must use authenticated and integrity-protected message formats; the simulator does not exercise this layer.

Connection Model: Bilateral Cap, Instant Setup

What we model. Each node has a configurable `maxConnections` cap (default 100, with the synaptome itself capped at 50). When node A wants to add B as a synapse, the simulator's `tryConnect`(A, B) checks whether *both* sides have an open connection slot. If either side is full, the connection is silently refused (the caller's logic falls through to the next candidate). This is the simulator's first-order analogue of WebRTC's per-browser data-channel limit.

What we do not model. *Connection setup time.* Real WebRTC connections require ICE (negotiation), STUN (NAT discovery), TURN (relay fallback), and DTLS (handshake). The total setup time is typically 1.5–3 seconds in the wild, and successful setup is not guaranteed — ICE failure rates of 10–15% are common in the WebRTC literature.[38]

[38] See the WebRTC measurement studies, e.g. webrtchacks.com, for empirical distributions of setup time and failure rate. The numbers vary strongly by NAT type and by the availability of TURN relays.

The implication: every architectural mechanism in N-DHT that *depends* on rapid synapse establishment — Slice World's re-stitching after partition; annealing's 2-hop-neighborhood replacement; hop caching's deposit-and-use; the entire pub/sub re-subscribe self-heal — looks faster in the simulator than in production. The simulator says "94% Slice World recovery in 500 lookups"; a production system facing 1–3 second connection setup per new synapse would take much longer. Tier 1 Issue #1 of the red team.

What we do not model. *Connection failure.* Once `tryConnect` succeeds, the connection is permanent (until one side dies). Real connections drop intermittently and need re-establishment. The protocol's mechanisms for handling this are equivalent to dead-peer detection, but the rate is much higher than churn alone would predict.

Bandwidth Model: Per-Node Counters, No Saturation

What we model. As of v0.70.04, every routing chokepoint in the simulator is instrumented with per-node send/receive counters. The instrumentation captures: every `SimulatedNetwork.send` call (Kademlia, G-DHT); every direct nodeMap-mediated message in NX-6, NX-17, and NH-1 (routing hops, two-hop probes, LTP feedback walks, hop caching, lateral spread, triadic introductions, local-candidate scans for annealing and dead-peer replacement, axonal tree `routeMessage` forwarding, `sendDirect` entry).

The counters are per-node, per-message-type. Each lookup tick resets them; a snapshot is taken at the end of every benchmark or training cycle. Results: total messages, mean / p50 / p99 / max per-node load, Gini coefficient over the per-node distribution, count of nodes processing more than 10× or 100× the cycle mean. Chapter V

analyzes the resulting distributions.

What we do not model. *Bandwidth saturation proper.* The counters tell us *how many* messages each node is processing; they do not feed back into the protocol's behavior. A node processing 1000× the network mean incurs no additional delay, no drop, no backpressure. The protocol's AP scoring uses static latency, not load-adjusted delay. The implication is that we can characterize the *distribution* of load (and we have, decisively, in Chapter V) but we cannot test whether the protocol oscillates against actual saturation. The deferred concern is Tier 1 Issue #3 in the red team.

The reclassification of Issue #3 in Chapter V is precisely about the *distribution* question, not the *saturation* question. The distribution measurement falsifies the hypothesis that N-DHT's adaptive routing concentrates load (it doesn't; the load is broadly distributed at every tested scale, with fewer hot nodes than Kademlia). The saturation question — "does the protocol oscillate when an actually-saturated peer slows down?" — requires a load-aware AP scoring extension and a saturation simulation, both deferred to the production pathway (Chapter VI).

Adversarial Behavior: Cooperative Trust Throughout

What we model. Nothing adversarial. Every node in the simulator obeys the protocol exactly: correct ID assignment, correct routing-table maintenance, correct pub/sub handler behavior, correct dead-peer signaling.

What we do not model. *Sybil attacks.* An attacker generates many forged identities all bidding for routing-table positions in a target region. The simulator does not produce or defend against Sybils.

What we do not model. *Eclipse attacks.* An attacker controls all peers in a target node's synaptome, isolating the target from the rest of the network. The simulator does not produce or defend against eclipses.

What we do not model. *Prefix forgery.* A node falsifies its S2 prefix to land in a region where it does not physically reside. The simulator's nodes have correct prefixes by construction. The locality benefits we report assume cooperative trust; an adversarial deployment with a fraction of forged-prefix nodes would experience proportionally degraded locality (not catastrophically degraded routing, but the qualitative claims about regional latency would weaken).

What we do not model. *Routing-deviation attacks.* A peer that selectively drops, corrupts, or misroutes messages. The simulator's nodes route honestly.

The full catalog of adversarial threats and the mitigations identi-

fied for each is in Chapter VI, Tier 2 (Issues 5–8: Sybil, churn-class heterogeneity, Byzantine pub/sub, asymmetric reachability) and Tier 3 (Issues 9–13: hardening items). The protocol's correctness does not depend on any of the threat models being defended against in the simulator; performance claims in the presence of these threats require the corresponding mitigations in place.

The Honest Closure

The protocol's results measured in this volume are *accurate for the conditions modeled*. The conditions modeled are not the full conditions a real deployment will face. The gap between the two is documented above and ranked in the red team (Chapter VI); the pathway to closing it is in Chapter VI.

The honest framing: the brain is production-ready; the body is not. The protocol's algorithmic core — the substrate, the five operations, the consolidation around vitality, the axonal-tree pub/sub primitive — is well-characterized and well-measured at scales up to 50,000 nodes. The transport layer — connection-setup time, RPC timeouts, jitter, bandwidth saturation, adversarial peer behavior — is the next, *measurable, scopeable* problem. The simulator extensions described in Chapter VI are how that work gets done.

What this means for the document's claims: every quantitative claim is conditional on the simulator's modeling choices. A careful reader can take any number in this document and ask "does this depend on something the simulator does not model?" Where the answer is yes (e.g., "does the 94% Slice World recovery rate hold under realistic connection-setup time?"), the honest answer is "no, it would be lower." We do not hide that. The simulator's value is that it lets us decompose the protocol's behavior into separable components, each measurable on its own, each addressable in turn.

The Adversarial Test Suite

EACH TEST in the simulator's suite was added with a specific hypothesis to falsify. This chapter walks the suite, giving for each test: *what it does*, *what hypothesis it is designed to break*, *what the result actually is*, and *what counts as a failure* (so a future test run that *does* fail will be unambiguous).

Tests are organized by what they stress: routing correctness under benign conditions; routing correctness under partition; robustness under churn; bandwidth distribution under load; ablation tests that isolate specific mechanisms.

Global and Regional Routing

What it does. The benign baseline. Uniformly-random source and destination; measure mean latency, mean hop count, success rate over 500 lookups per cell. Variants restrict the destination to a regional disk: 500 km, 1,000 km, 2,000 km, 5,000 km radius around the source.

Hypothesis under test. *The protocol does not reach within a small constant of the analytical* 3δ *floor on global lookups, and it does not approach single-hop performance on regional lookups.* If either claim fails, the protocol's value proposition versus Kademlia is gone.

What counts as failure. Global mean latency more than $2\times$ the 3δ floor. Regional mean hop count indistinguishable from global. Success rate below 99% on any benign workload.

Result. The hypothesis is falsified — by the *floor-hugging* property of the actual measurements (Chapter V). NX-17 hits $1.16\times$ the floor on global lookups at 25,000 nodes; NH-1 hits $1.27\times$. Regional routing converges to single-digit-hop performance, and the gap between regional and global widens as N grows. Success rate is 100% across all benign workloads at all tested scales.

Concentrated Workloads (Hot Source / Hot Dest / Hot Pair)

What it does. The realistic-traffic test. "Hot destination" restricts the destination pool to 10% of nodes (the "10% dest" workload); the source is the remaining 90%. "Hot source" is the converse; 10% of nodes generate all traffic. "Hot pair" restricts both: the same 10% source pool routes to a different (or same) 10% destination pool.

The pattern models real applications: a CDN serves content from a small set of replicas; a chat group routes among a small set of frequently-conversing peers; a measurement system aggregates into a small set of collectors.

Hypothesis under test. *Adaptive routing does not provide measurably more benefit on concentrated workloads than on uniform workloads.* If the hypothesis holds, the "learning" part of N-DHT does not justify its complexity; you might as well use G-DHT.

What counts as failure. The latency advantage of N-DHT over G-DHT shrinks toward unity as the workload becomes more concentrated. The hypothesis is intentionally counter-intuitive to provide a sharp falsification target.

Result. The hypothesis is falsified — in the opposite direction. N-DHT's latency advantage *grows* with concentration. At 25,000 nodes:

- Global random: NH-1 at 260 ms, G-DHT at 282 ms (8% improvement)
- 10% dest: NH-1 at 43 ms, G-DHT at 108 ms (60% improvement)
- 10% → 10%: NH-1 at 33 ms, G-DHT at 109 ms (70% improvement)

The mechanism producing the gap is hop caching: under concentrated workloads, the same destinations are reached from many sources; intermediate nodes accumulate shortcuts to those destinations; subsequent lookups bypass entire prefix paths. G-DHT has no such mechanism.

Slice World: Single-Bridge Partition Recovery

What it does. The sharpest stress test in the suite.[39] The network is partitioned almost in half: Eastern hemisphere on one side, Western hemisphere on the other, connected only through a *single bridge node* placed near Hawaii. Every other cross-hemisphere edge is removed before the test begins.

The protocol is then asked to route between hemispheres: 500 lookups whose source is in one hemisphere and destination is in the other.

Hypothesis under test. *The learning mechanisms cannot recover from a network partition with only a single bridge.* The hypothesis

[39] Named after the Tobias Werner short story "Sliceworld," in which Earth is bisected. The reference is loose; the test is sharper.

is plausible because the bridge is a geometric bottleneck — if every cross-hemisphere lookup must route through the bridge, the bridge would become saturated, the partition would not be recovered in any meaningful sense, and the success rate would be far below 100%.

What counts as failure. Cross-partition success rate below 80%. The threshold is generous; even a partial recovery (say, 50%) would be a substantial improvement over Kademlia's 0%, but a protocol whose learning genuinely worked should hit much higher.

Result. The hypothesis is falsified at $\sim 94\%$ cross-partition success. The mechanism is the bridge becoming a *seed crystal*: hop caching deposits cross-hemisphere synapses in every intermediate node along each successful bridge crossing; triadic closure introduces direct edges between observed transit pairs; lateral spread propagates the new edges to geographic neighbors. After the first hundred lookups, hundreds of cross-hemisphere synapses exist that bypass the bridge entirely. The bridge is no longer special.

For comparison: Kademlia achieves 0% (no learning, the partition is permanent); G-DHT achieves 4.6% (geographic prefix points a few peers at the bridge but no mechanism builds on the discovery). NX-17 and NH-1 both achieve $\sim 94\%$.

Pub/Sub Delivery Under Churn

What it does. A topic with a configurable number of subscribers (default 32) is created. The publisher sends a test message; the simulator counts how many subscribers received it. The test is then repeated under increasing churn rates: 5% instantaneous, 10%, 20%, up to 30%.

A separate variant runs continuous churn: 1% of live nodes per 5 ticks, with three refresh intervals between churn events to allow the tree to heal. This produces a sustained-load curve.

Hypothesis under test. *Axonal-tree self-healing cannot maintain* 100% *pub/sub delivery under realistic churn.* The hypothesis is plausible because the tree is built by routed subscribe; under churn, parts of the routing topology change between when the tree is built and when a publish flows; the message could miss subscribers attached to branches that were severed.

What counts as failure. Steady-state baseline delivery below 99%. Recovery from 5% instantaneous churn taking more than three refresh intervals to return to 100%. Continuous-churn delivery dropping below 90% at 5% sustained churn rate.

Result. Steady-state baseline: 100% delivery at 25,000 nodes with 79 topic groups of 32 subscribers each. After 5% instantaneous churn, immediate delivery falls to 99.9% and recovers to 100% after three refresh intervals.

The continuous-churn variant produces a smooth degradation curve rather than a cliff: 98.7% delivered at 5% cumulative churn, 91.2% at 10%, 86.5% at 20%, 70.0% at 25%, 52.4% at 30%. Crucially, no cliff: delivery degrades *linearly* with sustained churn, indicating the protocol finds a steady state where new re-subscribes match losses. The dominant residual failure mode at high churn is not broken routing but *subscribers temporarily captured at relays that have lost their delivery path to the root* — the failure mode the temporal replay cache (Chapter II) was designed to close.

Bandwidth Distribution: The Concentration Audit

What it does. Every per-node send and receive event is counted, by message type. Per cycle (a unit of 500 lookups), the counters are aggregated into a distribution: total messages, mean / p50 / p99 / max per-node load, Gini coefficient over the per-node totals, count of nodes whose per-cycle traffic exceeds 10× or 100× the network mean.

The instrumentation is added at every routing chokepoint, including the direct-nodeMap-mediated paths in NX-6, NX-17, and NH-1 that bypass `SimulatedNetwork.send`. Without explicit instrumentation, the simulator cannot see this traffic and the protocol's true bandwidth profile is invisible.

Hypothesis under test. *Adaptive routing concentrates load on a small number of nodes more aggressively than non-adaptive routing.* The hypothesis was raised in the red-team analysis (Chapter VI) as the *success-disaster* failure mode: LTP reinforces a fast peer until it saturates, AP scoring routes around it when it slows, LTP re-reinforces it when it recovers, oscillation ensues.

What counts as failure. N-DHT producing more $> 100\times$-mean nodes than Kademlia at any tested scale. N-DHT's Gini coefficient growing faster with N than Kademlia's. The max/mean ratio worsening monotonically with N. Any of these would suggest the adaptive mechanism is amplifying concentration.

Result. The hypothesis is falsified, decisively. At 50,000 nodes:

- Kademlia: 56 nodes processing $> 100\times$ the mean, Gini 0.940, max/mean 208×.
- G-DHT: 62 nodes processing $> 100\times$ the mean, Gini 0.932, max/mean 213×.
- NX-17 default: **0** nodes $> 100\times$, Gini 0.694, max/mean 61×.
- NH-1 default: **0** nodes $> 100\times$, Gini 0.661, max/mean 47×.

The classical DHTs produce dozens of catastrophic-bandwidth nodes at 50,000 scale; the adaptive DHTs produce zero. Chapter V

reports this in full detail with the 4×4 sweep and the rate-knee curve.

The episode is the canonical example of falsification-driven development. We added the bandwidth instrumentation specifically to test the red team's hypothesis. The test broke the hypothesis, not the protocol.

Ablation: The geoBits = 0 Test

What it does. The ablation removes the geographic prefix entirely. Node IDs are pure $H_{64}(publicKey)$ — random. Otherwise the protocol runs unchanged: same synaptome capacity, same vitality, same five operations.

Hypothesis under test. *All of N-DHT's performance benefit comes from the geographic prefix; the neuromorphic learning is gravy.* A skeptic's hypothesis, because if it were true the protocol's contribution beyond G-DHT would be marginal.

What counts as failure. N-DHT's latency at GEOBITS $= 0$ being indistinguishable from Kademlia's. The performance attribution would belong entirely to the prefix.

Result. The hypothesis is falsified. With *zero* geographic information, NX-17 still routes 26% faster than Kademlia at 25,000 nodes, and NH-1 routes 8% faster. The learning mechanisms do real work, attributable cleanly to the synaptome shaping itself by traffic.

The smaller advantage for NH-1 vs NX-17 in this ablation (8% vs 26%) is itself informative: NX-17's two-tier machinery and Markov pre-learning give it more starting structure to exploit when the geographic prefix is removed. NH-1, having consolidated those into the unified vitality model, has less independent structural redundancy. The ablation shows the cost of the consolidation: a small performance drop in the no-locality regime, consistent with the broader claim that NH-1 trades $\sim$ 5–10% peak performance for $\sim$ 30% parameter-count reduction.

Ablation: Highway Percentage

What it does. A fraction of nodes (the `highwayPct` parameter) are designated as "highway" — they have unrestricted connection caps, modeling server-class deployments. The remaining nodes are browser-class with `maxConnections` = 100.

The sweep runs at 0% (uniform browser-class), 5%, 15%, 30%, and 100% (uniform server-class) highway.

Hypothesis under test. *Real-world deployment will require server-class infrastructure for acceptable performance, making the protocol effectively centralizing.* If the hypothesis holds, the protocol

fails its mission.

What counts as failure. Latency at 0% highway being much worse than at 100%. A linear performance scaling with highway percentage — showing that infrastructure is doing the work, not the protocol.

Result. The hypothesis is falsified. 15% server-class nodes captures 70% of the latency improvement available from going to 100% server-class. Real-world deployments don't need a uniform server fleet; a modest fraction of capable nodes (residential desktops with no mobile/NAT constraints, plus the operator's own infrastructure) is sufficient.

The shape of the curve is concave: most of the benefit comes from getting the first 5–15% above zero. Pure browser-class deployment (0% highway) is workable, just slower. Pure server-class deployment is not necessary.

The Production Test: 50,000 Nodes End-to-End

What it does. The full benchmark suite at 50,000 nodes: global, regional (500 km, 1,000 km, 2,000 km, 5,000 km), concentrated workloads (source/dest/srcdest at 10%), pub/sub baseline and pub/sub under 5% churn. Each test cell runs 500 lookups; the full suite takes a few hours on a laptop.

Hypothesis under test. *The protocol's results at* 25,000 *nodes do not extend to* 50,000. Some quadratic-cost mechanism could be hiding in the smaller-scale measurements; some concentration effect could appear at scale that does not appear at half-scale.

What counts as failure. Global latency growing faster than $\log N$. Regional latency *growing* with N. Success rate dropping. Pub/sub baseline below 99%. Bandwidth max/mean ratio worsening.

Result. The hypothesis is mostly falsified. Global latency tracks the $\log N$ curve cleanly; NH-1 holds its 1.27× floor multiplier. Regional latency continues to *decrease* with N on the 500 km radius (more nodes per region, more shortcut opportunities). Success rate stays at 100% on benign workloads. Pub/sub baseline stays at 100%. Bandwidth max/mean ratio actually *decreases* with N for all four protocols (because the per-cycle work is spread over more nodes), and N-DHT continues to produce zero $>$ 100×-mean hot nodes while Kademlia produces 56.

The qualifier "mostly" is for one observation: the *absolute* count of $>$ 10×-mean hot nodes in N-DHT *grows* with N (1,419 at 50,000 vs 204 at 25,000). The increase is sub-linear in N (7× growth for 2× scale) and not catastrophic, but it is the one curve we have not flattened. The mitigation, if needed, is the `annealRateScale` knob

discussed in Chapter V.

What the Suite Does Not Yet Cover

HONEST CLOSURE. The test suite catches a wide range of failure modes but is not exhaustive. Tests we have not yet implemented:

- *Adversarial nodes.* Sybils, eclipses, prefix forgery, routing-deviation attacks. Discussed in Chapter VI (Tier 2).
- *Friction-realistic latency.* Connection setup, RPC timeouts, jitter. Tier 1 of the red team; closure path in Chapter VI.
- *Zipf-distributed pub/sub.* A small number of very popular topics with high publish rate; tests the hot-axon-root saturation question. Currently not in the suite; addresses one residual of the bandwidth question not closed by Chapter V.
- *Variable-rate churn.* Continuous churn at a rate that varies over time (peak-hour pattern). Inspired by Ghinita-Teo's adaptive-stabilization work (Chapter I).
- *Heterogeneous-capacity heterogeneity.* The current highway sweep is binary. A continuous distribution of node capabilities is more realistic.

Each gap is named, located in the larger road map (red team or production pathway), and assigned to a future iteration. The fact that the gaps are named is the point: a test we *know* we need to add is a deferred concern, not an unknown unknown. The simulator-as-laboratory commitment makes the gaps part of the engineering plan, not a footnote.

Part V

Evaluation

Methodology

Every measurement reported in this Part was produced by the same simulator, run with the same default parameters, unless explicitly noted. This chapter records those parameters and the procedures we follow so that any number can be reproduced from the open-source code base by a third party.

Default Run Configuration

Unless a chapter says otherwise, every benchmark in Part V uses these settings:

- **Node count.** Reported per measurement; typically 25,000 or 50,000. The full benchmark suite has been run at 5,000, 10,000, 25,000, and 50,000.
- **Bilateral cap (max connections).** 100. This is the WebRTC-realistic deployment target: a typical desktop browser sustains 100–200 data channels comfortably; the cap at 100 leaves headroom for application-level WebRTC traffic outside the DHT.
- **Synaptome capacity.** 50. Half the connection cap; the routing table cannot use all available connection slots, leaving room for bootstrapping new joiners.
- **Highway percentage.** 0%. Uniform browser-class deployment unless we are running the highway-percentage sweep (§V).
- **geoBits.** 8. The standard S2 cell prefix width. Ablations at 0 and 16 are reported separately.
- **Lookups per cell.** 500. Each measurement is the mean over 500 lookups within a single workload pattern. Standard deviation is reported when relevant; on stable workloads it is consistently small (±5% of mean).
- **Bidirectional bootstrap.** Yes. Both sides of every bootstrap edge are recorded.

- **Web limit.** Yes. The bilateral cap is enforced.
- **Random seed.** The benchmarks use a fixed seed per run so results are deterministic. Different seeds produce different absolute numbers but the same qualitative conclusions; we have not seen a measurement where the qualitative ordering of protocols changes with seed.

The full parameter list, including every implementation constant not exposed for routine tuning, is in Appendix VIII.

The Test Suite Cells

A *cell* is one (workload, protocol, parameter) tuple. The benchmark sweep runs a list of cells per protocol and aggregates into a CSV per benchmark. Workloads:

- `global` — uniformly-random source and destination
- `r500`, `r1000`, `r2000`, `r5000` — regional lookups within radius (km) of source
- `src` — 10% source pool, uniform destinations
- `dest` — 10% destination pool (XOR-nearest selection), uniform sources
- `srcdest` — both pools restricted to 10%, non-overlapping
- `pubsub` — pub/sub baseline delivery; topic coverage and group size from the run config
- `churn` — pub/sub delivery under 5% instantaneous churn after warm-up

For neuromorphic protocols, each test runs a warm-up phase first (typically 5,000 lookups for the destination pattern) so the synaptome is shaped for the test workload before measurement begins. K-DHT and G-DHT have no warm-up; their routing tables are static after bootstrap.

The 3δ Floor

What it is. Dabek et al.[40] proved a lower bound on lookup latency in any recursive DHT: the median end-to-end lookup time is at least 3δ, where δ is the median one-way RTT between random pairs of nodes on the underlying network.

The proof is geometric. Each hop in any DHT must, on average, cover half the remaining distance. The hop latencies sum to $\delta + \delta/2 +$

[40] Dabek, Li, Sit, Robertson, Kaashoek, Morris. "Designing a DHT for low latency and high throughput." NSDI 2004.

$\delta/4 + \ldots = 2\delta$. One additional hop is needed for delivery, contributing δ. Total: 3δ.

Where we are. The simulator's δ at 25,000 uniformly-distributed nodes on land is 68 ms (median), 70 ms (mean), 123 ms (p95). The floor is therefore 204 ms (median basis) or 209 ms (mean basis). Every "how close to the floor are we" analysis in this Part is relative to that number.

The floor is a lower bound, not a target. A protocol that beats the floor at small N is not contradicting the theorem: the theorem describes the asymptotic geometric average. A protocol that hugs the floor at large N is in some sense *unimprovable* on this metric without changing the underlying network topology.

Reporting Conventions

Three conventions worth being explicit about.

Latency means RTT. All reported latencies are round-trip times in milliseconds. A "240 ms global lookup" means the total wall-clock time from when the source initiates the lookup to when it receives confirmation; both directions of every hop are accounted for.

Hop counts include the destination hop. A lookup that reaches the target in two intermediate hops is a 3-hop lookup: source $\rightarrow$ peer $\rightarrow$ peer $\rightarrow$ target. This matches the 3δ-floor convention (the floor's third δ is the delivery hop).

Success rate is per-lookup, not per-cell. A success rate of 99.9% on 500 lookups means 499 succeeded and one failed; "99.5%" means at most three failed; etc. We report 100% when no failures were observed, which is the case on all tested benign workloads at all tested scales.

Performance Results

THIS CHAPTER reports the headline numbers for the four-way protocol comparison at 25,000 nodes. We compare K-DHT (Kademlia, the historical baseline), G-DHT (Kademlia plus geographic prefix, our prior work), NX-17 (the prior-generation state-of-the-art neuromorphic protocol), and NH-1 (the consolidated current generation). All numbers are from a single benchmark sweep run on simulator v0.70.05 with the standard methodology of Chapter V.

The Headline: Four Protocols, Ten Workloads, 25,000 Nodes

Table 4 is the headline result. Each cell is the mean latency in milliseconds over 500 lookups; the 3δ floor is 204 ms (median basis), 209 ms (mean basis).

Workload	**K-DHT**	**G-DHT**	**NX-17**	**NH-1**
global random	511.6	282.7	**237.3**	260.8
500 km regional	499.1	153.4	**80.0**	93.9
1,000 km regional	505.1	162.0	**91.6**	107.8
2,000 km regional	514.6	178.5	**103.2**	126.0
5,000 km regional	496.9	205.4	**147.1**	166.6
10% source pool	513.0	292.2	**243.9**	254.3
10% dest pool	242.7	108.0	**40.6**	43.2
10% → 10% pair	244.3	109.1	**31.9**	33.3
5% churn (post-warm)	477.6	279.2	**239.6**	279.5

Table 4: Mean latency (ms) at 25,000 nodes, 500 lookups per cell. The 3δ floor at this scale is 204 ms (median) / 209 ms (mean). **Bold** marks the winning protocol per workload.

The overall picture: NX-17 wins every cell, NH-1 trails NX-17 by 5–15% across the board, and both adaptive protocols dominate G-DHT, which dominates K-DHT.

The 5%-churn row is the resilience test. K-DHT and G-DHT performance under churn is essentially indistinguishable from their global baselines, because they don't actually adapt to churn beyond replacing dead bucket entries when next they're queried. NX-17 holds at 240 ms; NH-1 holds at 280 ms — NH-1 takes a slightly larger relative hit because its consolidated mechanisms for repair are slightly less aggressive

than NX-17's two-tier highway maintenance, and the ablation cost is most visible under churn.

The 3δ Floor: How Close Are We?

The headline. On global lookups at 25,000 nodes: K-DHT runs at 2.45× the median floor; G-DHT at 1.35×; NX-17 at **1.16**×; NH-1 at **1.27**×.

N-DHT (in both NX-17 and NH-1 forms) hugs the analytical floor. The remaining $\sim 20\%$ overhead is structural: N-DHT takes ~ 4.4 hops where an ideal protocol would take 3; each "extra" hop costs roughly $\delta/2$, exactly as the geometric series predicts.

The scaling shape. K-DHT's floor multiplier *grows* with N: 2.01× at 5,000 nodes, 2.45× at 25,000, 2.65× at 50,000. This is the wrong scaling: more nodes means more hops, each at full average latency, each contributing fully to the total. K-DHT's hop count grows as $\log N$ but its hop latency does not shrink, so total latency grows.

N-DHT's multiplier is roughly stable. NX-17 holds at 1.13–1.18× across the 5,000–50,000 range; NH-1 at 1.18–1.30×. The mechanism is clear: N-DHT's hop count grows logarithmically (same as Kademlia), but each hop's average latency *shrinks* with N because the synaptome's denser regional coverage gives shorter hops on average. The two effects roughly cancel; total latency stays near the floor.

Regional scaling. On the 500 km workload at 25,000 nodes, NH-1 is at 94 ms — below the global median 3δ floor of 204 ms. The floor is not a lower bound on *regional* latency, only on global. Regional lookups have a different geometric basis: they cover only the δ_{regional} which is much smaller than the global δ. A 500 km lookup at 25,000 nodes benefits from 4.5 hops at an average of 20 ms each — a total of 94 ms, dominated by the per-hop processing constant rather than great-circle distance.

The Locality Story: Where the Headlines Come From

Three regimes. Real-world workloads are dominated by regional traffic, but global routing is not negligible. The benchmark cells separate three regimes.

Regional (500–2,000 km). The dominant real-world workload pattern. N-DHT cuts latency by roughly 5–6× versus K-DHT and 1.5–2× versus G-DHT. The mechanism is hop caching plus AP-scoring's latency penalty: regional shortcuts accumulate in synaptomes; regional lookups exploit them.

Cross-continental (5,000 km+). Less common in day-to-day workloads but critical when it matters. N-DHT runs at roughly 3× the

speed of K-DHT on 5,000 km, and 1.2–1.4× the speed of G-DHT. The mechanism is two-hop AP lookahead: cross-continental routes are constructed by chaining two regional hops with a transcontinental hop in between, and lookahead lets the protocol find the right intermediary.

*Concentrated (*10% → 10% *pair).* The strongest mechanism for N-DHT. NH-1 routes a 10% → 10% pair in 33 ms versus K-DHT's 244 ms — a **7**× speedup. The mechanism is hop caching's tributary effect: each lookup through an intermediate node deposits a synapse for the destination at that node; subsequent lookups from any source that pass through the intermediate use the shortcut. Repeated access to the same destination set turns a 5-hop journey into a 1-hop direct shortcut.

Pub/Sub Performance and Robustness

Baseline delivery. At 25,000 nodes with 79 topic groups of 32 subscribers each, NH-1 delivers 100.0% of publishes in steady state. Under 5% instantaneous churn, immediate delivery falls to 99.9% and recovers to 100.0% within three refresh intervals.

The publisher-to-relay path averages 4.8 hops at 241 ms. The broadcast tree depth averages 2.9 hops at 198 ms with a maximum fan-out of 31 subscribers per relay.

Sustained-churn graceful degradation. Continuous churn at 1% of alive nodes per 5 ticks, with three refresh passes between kills, produces a smooth degradation curve: 98.7% delivered at 5% cumulative churn, 91.2% at 10%, 86.5% at 20%, 70.0% at 25%, 52.4% at 30%. The curve is linear — there is no cliff — which indicates the protocol finds a steady state where new-subscriber recruitment matches loss.

Slice World: the partition-recovery test. Table 5 reports the cross-hemisphere delivery rate when the network is partitioned into Eastern and Western hemispheres connected only through a single bridge node near Hawaii.

Protocol	**Slice success %**	**Slice ms**
K-DHT	0.0%	— (no recovery)
G-DHT	4.6%	525 (no recovery)
NX-17	94.8%	423
NH-1	**94.4**%	**462**

Table 5: Slice World single-bridge partition recovery. "Slice success %" is the lookup success rate over a 500-lookup cross-partition workload after partition setup.

K-DHT has no learning; the partition is permanent. G-DHT routes the geographic prefix toward the bridge for a few percent of lookups but no mechanism amplifies the discovery. NX-17 and NH-1 both achieve ∼ 94% via hop caching and triadic closure converting the bridge into a seed crystal for new cross-hemisphere edges.

The mechanism is the bridge as *seed crystal*. After just 10 lookups through the partition, hop caching has installed ~ 20 cross-hemisphere synapses (an average of ~ 3 per successful crossing). By 500 lookups, hundreds of cross-hemisphere synapses exist; the partition has effectively dissolved. The 94% success is the integral of recovery, not a snapshot of bridge-finding.

The Highway-Percentage Sweep

The motivating question. Real deployments will not be uniformly browser-class. A fraction of nodes will run as server-class peers (residential desktops with no NAT, organizational infrastructure, deliberate "contributing" deployments). What fraction is needed before the network functions well?

The answer: surprisingly little. The latency curve from 0% to 100% highway is concave. The first 5–15% of server-class capacity captures roughly 70% of the latency benefit available from going to fully-server-class. Past 15%, the marginal return diminishes; by 30% the performance is essentially indistinguishable from 100%.

The architectural implication. The protocol is *deployable today* on browser-class infrastructure with modest server-class augmentation: a small constellation of server-class peers (whether self-hosted by the network operator, contributed by users, or run by community infrastructure projects) is sufficient. We do not need to convince every user to provision a fully-uncapped node; the network functions well at 5–15% heterogeneity.

The geoBits = 0 Ablation

What it asks. If we remove the S2 geographic prefix entirely (`geoBits` = 0), does N-DHT still beat K-DHT? The hypothesis (Chapter IV) is the skeptic's: "all the performance benefit comes from the geographic prefix; the learning is gravy."

What it shows. Removing the prefix degrades N-DHT's performance, but the protocol still beats K-DHT by a substantial margin.

Protocol	**geoBits=8**	**geoBits=0**	**Δ vs Kad**
K-DHT	511	511	—
NX-17	237	377	26% faster than Kademlia
NH-1	260	470	8% faster than Kademlia

Table 6: Global lookup latency at 25,000 nodes, GEOBITS = 8 (default) vs GEOBITS = 0 (ablation). The drop with no prefix is what the geographic mechanism contributes; the residual gap to Kademlia is what the learning contributes.

The ablation falsifies the hypothesis. With *zero* geographic information, both adaptive protocols still beat Kademlia. The learning is doing real work, attributable cleanly to the synaptome shaping itself by traffic.

The smaller residual advantage for NH-1 vs NX-17 in this ablation (8% vs 26%) is informative: NX-17's two-tier machinery and Markov pre-learning give it more starting structure to exploit when the geographic prefix is removed. NH-1, having consolidated those into the unified vitality model, has less independent structural redundancy. The ablation shows the *cost* of the consolidation: a $\sim$ 5–10% peak performance reduction in the no-locality regime, traded for $\sim$ 30% parameter-count reduction and substantially better maintainability.

Heading Toward 50,000 and Beyond

What the 50,000-node measurements show. The full sweep at 50,000 nodes is reported in Chapter V in the context of bandwidth distribution. The headline observation: every protocol's latency multiplier on the 3δ floor stays roughly constant from 25,000 to 50,000, but K-DHT's gets slightly worse and N-DHT's stays flat. The qualitative ordering does not change.

What we have not yet measured. 100,000 nodes and 250,000 nodes. Preliminary runs at 100,000 in earlier generations indicated the protocol continues to scale; we have not yet done a full benchmark sweep at that scale on the current NH-1 generation. The simulator can run it; the limitation is analyst time, not compute. Future work (Chapter VIII).

Bandwidth Concentration: An Empirical Resolution

The companion red-team analysis ranks *bandwidth saturation and congestion collapse* as a critical deploy blocker (Issue #3, Tier 1). Of the four Tier-1 issues, three (connection setup, RPC timeouts, latency jitter) are textbook engineering problems with known mitigations. Issue #3 was different: the simulator's uniform-cost-per-hop model (Chapter IV) made it *architecturally unfalsifiable* whether N-DHT's adaptive routing concentrated load (the success-disaster oscillation hypothesized by the red team) or distributed it. Until that question was answered, every other claim about the protocol was contingent.

This chapter answers it. The instrumentation, the 4×4 sweep, the rate-knee, the churn validation, and the resulting reclassification are reported in detail.

Instrumentation

We instrument every routing chokepoint in the simulator with per-node send/receive counters. Two paths to cover:

Kademlia and G-DHT use `SimulatedNetwork.send` for all on-the-wire traffic. The instrumentation lives there: each call increments the sender's `msgsSent` counter and the receiver's `msgsReceived` counter, plus a per-type subtotal in `msgsByType`. One chokepoint, one modification, complete coverage.

NX-6, NX-17, and NH-1 access peers via `nodeMap.get` directly — the 50,000-node simulator benefits from skipping the message-dispatch layer for the intra-node-graph operations the neuromorphic protocols perform. Direct access bypasses `SimulatedNetwork.send`, so per-node counters at that chokepoint never fire for neuromorphic routing. We added a parallel `_msg(from, to, type)` helper invoked from each conceptual wire site:

- Each lookup-hop transition (`find_node`)
- Each two-hop AP probe (`lookahead_probe`)

Protocol	Total/cyc	Gini	max/mean	Hot > 10×	Hot > 100×
K-DHT	14,463	**0.940**	208×	551	**56**
G-DHT	17,633	**0.932**	213×	423	**62**
NX-17	359,459	0.694	61×	1,524	**0**
NH-1	416,071	0.661	47×	1,419	**0**

Table 7: Steady-state per-cycle traffic distribution at 50,000 nodes. "Total/cyc" counts each wire message twice (sender + receiver, the per-type counter convention). "Hot > 100×" is the count of nodes processing more than 100× the network mean per cycle — our operational definition of a bandwidth-saturation hazard.

- Each LTP reinforcement walk step (`reinforce`)
- Each hop caching and lateral spread write (`hop_cache`, `hop_cache_lateral`, `lateral_spread`)
- Each triadic-closure introduction (`triadic_introduce`, `triadic_probe`)
- Each bilateral bootstrap handshake (`bootstrap`)
- Each 2-hop neighbour scan in `_localCandidate` (`local_probe`)
- Each pub/sub `routeMessage` forward (`route_msg`) and `sendDirect` entry (`direct_*`)

Per-cycle counters are reset after each training session so the captured numbers are deltas. The instrumentation is opt-in via the simulator config and adds negligible overhead.

The Four-by-Four Headline

The 4 × 4 sweep: four protocols (K-DHT, G-DHT, NX-17, NH-1) at four sizes (5,000, 10,000, 25,000, 50,000), 10 training sessions of 500 lookups each. Steady-state averages over sessions 1–10. Default parameters throughout.

The headline result, in three observations:

1. The hypothesis is inverted. Kademlia and G-DHT develop 56–62 catastrophic-bandwidth nodes at 50,000 with max/mean ratios above 200×. N-DHT (in either form) produces *zero* such nodes at any tested scale. The classical DHTs, not the adaptive ones, are the protocols with the bandwidth-concentration problem at scale.

2. Gini scaling is the cleanest signal. Kademlia's Gini grows monotonically with N: 0.876 → 0.940 from 5,000 to 50,000. N-DHT's Gini grows more slowly: 0.549 → 0.661 over the same range. Past some threshold, classical DHTs concentrate *worse* as you scale; N-DHT tends toward a flatter floor.

3. Volume is the cost. N-DHT's total per-cycle traffic is 25–30× Kademlia's. The cost of broad load distribution is broader sustained activity. The next two sections decompose where the cost goes and how to control it.

Where the Volume Goes

The per-type instrumentation lets us decompose NH-1's traffic at 50,000 nodes:

Message type	**Count/cyc**	**%**
`local_probe` (2-hop annealing)	370,566	**89.1**%
`lookahead_probe` (two-hop AP)	20,525	4.9%
`lateral_spread`	7,474	1.8%
`hop_cache_lateral`	6,404	1.5%
`find_node` (actual routing hop)	5,538	1.3%
`hop_cache`	3,737	0.9%
`reinforce` (LTP feedback)	1,823	0.4%

Table 8: Per-type traffic decomposition for NH-1 at 50,000 nodes. Routing-hop `find_node` traffic is 1.3% of all traffic; the other 98.7% is learning side-effects.

A single mechanism — the 2-hop neighbour scan in `_localCandidate`, used by simulated annealing and dead-peer replacement — accounts for 89% of N-DHT's wire traffic. This is the lever for reducing the volume cost without disturbing the learning.

The annealRateScale Knee

We added a single parameter, `annealRateScale`, that multiplies the per-hop annealing trigger probability without disturbing temperature decay or the dead-peer-detection reheat mechanism. Default 1.0 preserves prior behavior. A sweep at 50,000 nodes:

The shape is clean. `local_probe` scales linearly with the rate (perfect 10× reduction at rate 0.10). **Routing quality is unchanged across the entire range:** hops 5.48–5.54, time 268–271 ms, success rate 100%. The throttle does not cost performance.

The *knee* is at rate ≈ 0.10 — the setting at which the absolute count of $> 10\times$-mean hot nodes drops to its minimum (575, versus 1,428 at default and 852 at overshoot). Past the knee, traffic concentrates on the established hot paths (Gini rises from 0.834 to 0.866); before the knee, more nodes do mild amounts of work and trip the 10× threshold. The middle is the sweet spot.

The Robustness-Under-Churn Validation

The throttle would be a Pyrrhic victory if it broke churn-recovery. We tested. At 25,000 nodes with 5% per-cycle node turnover:

Throttled NH-1 holds. 100% routing success, zero $> 100\times$-mean hot nodes, Gini 0.79 (still better than Kademlia's 0.92), and the smallest Gini-under-churn increase of any tested protocol (+0.012). The `T_REHEAT` mechanism (§II) fires on dead-peer detection regardless of base annealing rate, so churn-recovery is uncompromised by the throttle.

rate	Total/cyc	local_probe	Gini	Hot $> 10\times$	vs default
0.05	63,847	18,224	0.866	852	6.5× cut
0.10	**82,650**	**36,989**	**0.834**	**575**	**5.0× cut**
0.25	137,363	92,453	0.770	735	3.0× cut
0.50	232,749	187,366	0.713	1,215	1.8× cut
1.00	414,929	369,552	0.661	1,428	(default)

Table 9: The `annealRateScale` knee for NH-1 at 50,000 nodes. Routing quality (hops, time, success) is unchanged across the entire range. The knee is at rate = 0.10: 5× volume reduction at 90% lowest absolute hot-node count.

Protocol	Total/cyc	Gini	Hot $> 100\times$	Δ Gini	Success
K-DHT	12,725	0.919	**7**	+0.006	100%
G-DHT	15,817	0.910	**7**	+0.006	99.9%
NX-17 (default)	496,256	0.678	0	+0.028	100%
NH-1 (default)	356,358	0.654	0	+0.037	100%
NH-1 (rate=0.10)	**71,695**	**0.786**	**0**	**+0.012**	**100%**

Table 10: Steady-state distribution at 25,000 nodes with 5% per-cycle churn. Throttled NH-1 has the smallest churn-induced Gini increase of any tested protocol.

Classical DHTs degrade. Both Kademlia and G-DHT develop 7 catastrophic-bandwidth nodes under the same churn load. The 5% instantaneous turnover *worsens* their already-concentrated load distribution.

Reclassification of Red-Team Issue #3

The red-team analysis (Chapter VI) ranks bandwidth saturation as Critical / deploy blocker / 6–10 weeks of work. The proposed mitigation plan was Phase 3A–3E: per-node load reporting, load-aware AP scoring, hot-axon-root migration, MaxDisjoint replication, replay-cache load awareness. The plan only made sense if the foundational hypothesis (*adaptive routing concentrates load*) was correct.

The hypothesis is not correct. The instrumentation falsified it empirically: at every tested scale, N-DHT distributes load broadly (Gini 0.55–0.69) while Kademlia and G-DHT concentrate it catastrophically (Gini 0.87–0.94, with 56–62 nodes processing $> 100\times$ the mean at 50,000 nodes). The protocol's adaptive routing does not amplify the success-disaster failure mode; it actively prevents it.

The remaining concern is the Zipf-distributed pub/sub case — a single popular topic's root saturating from publish volume. This is a bounded sub-problem whose mitigation (hot-axon-root migration plus MaxDisjoint replication) was already specified by the red team. The deploy-blocker framing of the broader issue is retired.

This is the canonical example of falsification-driven development (Chapter IV). The simulator's job was to break the protocol; in this case, it broke the *red team's hypothesis*, not the protocol. The reclassification was earned by the measurement, not by the argument.

Operational consequence. A defensible operating point emerged: NH-1 with `annealRateScale` = 0.10. At 25,000 nodes that is 5× the

bandwidth of Kademlia (not 25× as default), Gini 0.79 (lower than Kademlia's 0.92), 100% routing success, 0 catastrophic hot nodes even under churn. The operational argument for this protocol no longer requires a 6–10-week mechanism build. It requires setting a parameter.

Architectural Wins Beyond the Headline

The headline numbers (Chapter V) do not exhaust the protocol's value. Several architectural properties show up indirectly — in the protocol's behavior under stress, in its scaling shape, in what it does *not* need — and these properties matter as much for production viability as the latency multipliers. This chapter walks them.

Iterative Fallback as Graceful Degradation

The mechanism. When a routing decision encounters no synapse with forward XOR progress, NH-1's iterative fallback takes the closest unvisited peer regardless of strict progress (§II). The lookup keeps moving, even at a small cost in optimality.

Why it matters. Without iterative fallback, a network partition or a churn-induced gap in the routing structure causes *permanent* lookup failure: the protocol arrives at a local minimum and stops. With iterative fallback, the protocol takes a sub-optimal step and continues; if a path exists, it will be found, even if longer than ideal.

The Slice World result depends entirely on this. NH-1's 94% cross-partition success rate is impossible without iterative fallback — the local routing structure cannot give a forward-progress synapse across the partition until the bridge has been used and shortcuts have been deposited, which requires *some lookup* to find the bridge first. Iterative fallback is what lets that first lookup succeed.

The cost is small: in steady-state benign workloads, the fallback fires on a small fraction of lookups (typically $< 5\%$), and each fallback adds at most one extra hop. The benefit is qualitative: *the protocol degrades gracefully under conditions the routing table does not yet know how to handle.*

Hop Caching as Implicit Replication

The mechanism. On every successful lookup, every intermediate node along the path installs a synapse pointing to the destination

(§II). After a path $A \to B \to C \to D$, both B and C now know how to reach D in one hop.

Why it matters. The mechanism is *implicit replication*: a destination ID, after enough lookups, becomes reachable from many intermediate nodes via direct synapse, not just from its XOR-nearest neighbors. The pattern is structurally identical to Tapestry's soft-state location pointers (Chapter I); we make it active on every successful path rather than only on explicit `publishObject` events.

The empirical result is the 7× latency advantage on 10% → 10% concentrated workloads (NH-1 at 33 ms vs Kademlia at 244 ms). Repeated traffic to the same destination set turns multi-hop paths into single-hop shortcuts. The protocol *learns the shape of the workload* and configures itself accordingly.

Triadic Closure as Structural Plasticity

The mechanism. A node observing the same transit pair $A \to (\textit{this}) \to C$ multiple times introduces A directly to C (§II). The triangle "$A-$this$-C$" is closed into the direct edge "$A-C$."

Why it matters. Triadic closure is the mechanism by which the network's *communication graph* reshapes itself to reflect the actual traffic patterns. Two peers that consistently route through the same intermediate are revealed as a peer pair that should know each other directly.

The mechanism is the network-level analogue of the brain's NMDA AND-gate: NMDA detects coincidence at single synapses; triadic closure detects coincidence at the network level, between groups of peers that keep flowing through the same intermediate. Both produce structural changes: NMDA strengthens existing synapses; triadic closure creates new ones.

The empirical signature is in the Slice World recovery: the 94% cross-partition success rate would not be reachable by hop caching alone. Triadic closure is what introduces the peer pairs that converge on the bridge into direct relationship, and once direct, those pairs no longer need the bridge as intermediary. The partition dissolves through both mechanisms working together.

Temperature Reheat as Event-Driven Repair

The mechanism. When a node detects a dead peer (a synapse pointing at a node that no longer responds), the node *reheats*: temperature is raised to $T_{\text{reheat}} = 0.5$. Subsequent hops anneal more aggressively until the temperature cools again (§II).

Why it matters. Most adaptive DHT designs maintain their

routing table on a schedule: stabilization timers, bucket refreshes, periodic checks. Ghinita-Teo (Chapter I) made the case that fixed-schedule maintenance is wrong almost everywhere — too low produces failure spikes during churn bursts; too high wastes bandwidth during quiet periods.

NH-1's temperature reheat solves the same problem differently. The trigger is the event itself: a dead-peer encounter says "the synaptome content is stale here, fix it now." Recovery is local, opportunistic, and proportional to actual damage. No periodic schedule, no global parameter to tune, no overhead during quiet operation.

The mechanism is part of why N-DHT can sustain 5% churn indefinitely with 100% routing success: the protocol doesn't *wait* for a maintenance interval to repair churn damage; it repairs at the moment of damage, with effort proportional to damage.

Latency-Aware AP Scoring as Speed-First Priority

The mechanism. AP scoring's hop selection formula (equation 3) combines XOR progress with observed latency. The latency factor is exponential with a 100 ms half-life: a synapse with 200 ms latency loses one factor of 2 in its score; a synapse with 400 ms latency loses two factors.

Why it matters. Pure XOR-progress routing (Kademlia) treats a 20 ms hop and a 200 ms hop as equivalent. AP scoring weights the 20 ms hop ten-fold higher. The protocol's regional-traffic latencies emerge from this: candidates that happen to be physically close get strongly preferred even when their XOR distance is slightly worse.

The mechanism interacts with the geographic prefix to produce the locality story: the prefix biases XOR distance toward physical proximity (so most candidates are nearby anyway), and AP scoring's latency penalty refines the choice within the biased candidate set. Either alone gives some locality benefit; together they give the 5–6× speedup of N-DHT over K-DHT on regional workloads.

The mechanism's elegance is that it is *pure routing logic*, not a separate module. There is no "geographic routing layer" in NH-1; locality emerges from a single line of the AP score formula plus the structural seed of the prefix.

Unified Vitality as Principled Pruning

The mechanism. Every admission decision in NH-1 — when a new synapse competes for entry into a full synaptome — is governed by a single equation: vitality = weight × recency. Twelve rules, twelve parameters, one admission gate.

Why it matters. Earlier generations had eighteen rules across many ad-hoc admission decisions: should this hop-cache entry evict that highway-tier entry? Should this triadic introduction evict that bootstrap synapse? Each pairwise decision had its own rule, its own parameters, its own trade-offs.

The unification collapses the bookkeeping. Every entry has a vitality score; the lowest-vitality non-LTP-locked entry is the eviction target. The bookkeeping is uniform across all mechanisms that introduce new synapses (hop caching, triadic, lateral spread, anneal, dead-peer replacement, incoming promotion). The behavior is uniform too: peers that the network's actual traffic uses accumulate weight and recency; peers that don't, lose their slot to peers that do.

This is the consolidation NH-1 is named for. The benchmark numbers (Chapter V) show that the consolidation is essentially behavior-preserving: NH-1 trails NX-17 by 5–15% on absolute metrics but matches it on every *qualitative* property — learning, locality, partition recovery, churn resistance, pub/sub delivery.

What N-DHT Does Not Need

The architectural wins listed above describe what NH-1 *has*. There is a complementary list: things the protocol does *not* need that other adaptive DHT designs do.

- *No periodic stabilization.* Unlike Chord, Pastry, CAN. Repair is event-triggered.
- *No two-tier routing structure.* Unlike NX-17 (and Pastry's R/L/M). One synaptome.
- *No K-closest replication for pub/sub.* Unlike NX-15. One root per topic, healed by re-subscribe.
- *No parent tracking in pub/sub.* Children re-subscribe upstream; that's the only state needed.
- *No explicit failure detection.* Dead-peer detection is a routing primitive, used opportunistically.
- *No global parameters.* Every parameter is per-node; no synchronization required.
- *No protocol negotiation.* All nodes run the same code; specialization emerges from traffic.
- *No central coordinator.* The protocol is *strictly* peer-to-peer at every layer.

The list is itself a contribution. Each item is a piece of machinery that some other DHT design carries because it was needed there; N-DHT does not carry it because the unified vitality model and the five-operation frame make it unnecessary.

What This Adds Up To

The *quantitative* picture is in Chapter V. The *qualitative* picture, the one this chapter has tried to make visible, is that N-DHT produces these properties from a small set of mechanisms in combination. No single mechanism delivers a property by itself; they interact. Iterative fallback alone wouldn't recover Slice World; you need iterative fallback plus hop caching plus triadic closure plus lateral spread, all firing together. AP scoring alone wouldn't give regional latency; you need the prefix plus AP plus learning.

The combination is what we report. Chapter VI catalogues what we still don't have, and Chapter VI describes the closure path. The brain is production-ready; the body is a measurable, scopeable problem.

Part VI

Deployment and Hardening

Red Team Analysis

What would break if we shipped this tomorrow?

The simulator results of Part V show N-DHT working under the conditions the simulator models. The conditions the simulator does not model (Chapter IV) are where the questions live. The companion red-team analysis[41] is the adversarial audit: thirteen ranked deployment-relevant gaps, each one a way real-world deployment can fail in a way the simulator cannot currently observe.

This chapter summarizes the audit and documents the changes that followed from the bandwidth-concentration analysis of Chapter V.

[41] NH1-RedTeam-v0.3.38.md in the project archive. Three independent red-team passes were performed against the protocol over the course of its development; the v0.3.38 ranking is the composite, ordered by occurrence × severity × detectability × time-to-fix × deploy-blocker status.

Methodology of the Audit

The audit identifies deployment risks through three lenses:

1. What the simulator does not model. Connection setup cost, RPC timeouts, bandwidth limits, jitter, Byzantine peers — absent or trivially modeled. Anything the protocol depends on for correctness that the simulator removes is a candidate failure mode (Chapter IV).

2. What prior art warns about. Castro et al. (OSDI 2002), Harvesf-Blough (P2P 2007), Makris et al. (BIGDATA 2017), Ghinita-Teo (IPDPS 2006), Dabek et al. (NSDI 2004) each catalogue a class of distributed-system failure mode. Issues that map onto known failure classes from this literature are weighted heavily.

3. What asymmetric or adversarial workloads expose. The simulator runs uniform-rate lookups on uniformly-distributed nodes against uniformly-distributed targets. Real networks have Zipf-distributed traffic, heterogeneous churn, asymmetric reachability, adversarial participants. Anything whose analysis assumes uniformity is suspect.

The audit ranks issues into three tiers: *deploy blockers* (the protocol's claimed properties cannot be verified in production until these are addressed), *correctness under stress* (the protocol works in nominal conditions but degrades under realistic adversarial or heterogeneous conditions), and *operational hardening* (known gaps that should be

closed before scale-out, but where staged rollout can manage the risk).

Tier 1: Deploy Blockers

Issues 1–4 in the original ranking. Three remain; one was reclassified following the bandwidth analysis.

Issue 1: Connection-setup friction. Real WebRTC requires ICE + STUN/TURN + DTLS, taking 1.5–3 seconds per new synapse under typical NAT conditions, with a 10–15% ICE-failure rate in the wild. N-DHT's most important behavioral results — Slice World re-stitching, annealing replacement, hop-caching cascade — all depend on synapses becoming usable rapidly. Production recovery times will be substantially worse than simulator measurements suggest until `CONNECTION_SETUP_MS` is modeled.

The mitigation is mostly engineering, not protocol redesign: new synapses sit in `PENDING` state, excluded from AP scoring until setup elapses; browser-aware concurrency cap ($\sim$ 4 ICE in flight on Chrome desktop, lower on Safari/mobile); GC-pause tolerance to suppress spurious timeout suspicion. Estimated 4–6 weeks. Priority: highest; this is the gating issue for any production deploy claim.

Issue 2: RPC timeouts and asynchronous black holes. RPCs to dead nodes return instantly in the simulator because the simulator knows. No timeouts, no dropped packets, no asymmetric reply paths (request goes through, reply doesn't). The 100% pub/sub recovery under 5% churn assumes instant detection; in the wild, every churn round costs multi-second timeout windows during which messages are lost.

The mitigation is the request/reply RPC refactor: `RPC_TIMEOUT_MS` (default 3,000 ms) sets a stall timeout before iterative-fallback / next-AP-hop; `routeMessage` traces forward and reverse paths independently; either failure fails the RPC; reply may take a different path back. Estimated 4–6 weeks.

Issue 3: Bandwidth saturation. ***Reclassified.***

The original Tier-1 hypothesis was that AP routing concentrates load on overloaded nodes (the success-disaster failure mode). The empirical resolution (Chapter V) showed the opposite: N-DHT distributes load broadly at every tested scale ($0 > 100\times$-mean nodes); the classical DHTs concentrate it catastrophically at scale (K-DHT produces 56 such nodes at 50,000 nodes). The 25$\times$ volume tax is a single tunable parameter (`annealRateScale`) with a clean knee at rate ≈ 0.10.

The residual concern is the **Zipf-distributed pub/sub case**: a single popular topic's root saturating from publish volume rather than from routing traffic. This is bounded sub-problem; mitigation by hot-

axon-root migration (Makris et al. DFE pattern) plus MaxDisjoint topic replication (Harvesf-Blough placement) is sketched in §VI. The deploy-blocker framing of Issue #3 is retired.

Issue 4: Latency jitter in AP routing. Real RTTs fluctuate ±30% from bufferbloat, asymmetric paths, queueing. Our simulator's distance-derived latency is monotone and clean. The LTP exponential moving average has an easy job in the simulator; high-frequency noise could prematurely decay good synapses or promote lucky-but-unstable ones.

Mitigation: jitter injection in the simulator (Gaussian noise on every hop's RTT, σ calibrated against measured production RTT distributions); validate that LTP-EMA does not oscillate; tune the EMA constant if it does. Estimated 2–3 weeks.

Tier 1 summary after reclassification. Three engineering issues with known mitigations and bounded cost. Total estimated work: 10–15 person-weeks. The protocol's algorithmic claims are not gated on these; they are gated on the simulator extensions that close the simulator/reality gap.

Tier 2: Correctness Under Stress

Issues 5–8 in the original ranking. The protocol works in nominal conditions; under realistic adversarial or heterogeneous workloads it degrades. Staged rollout can manage these — but each must be measured, not assumed.

Issue 5: Sybil and cell-eclipse hardening. The S2 prefix is self-declared (Chapter II). A node can claim any prefix it wants. The benign failure mode is degraded routing; the adversarial failure mode is a Sybil swarm in a target cell (cell eclipse). Mitigations: Vivaldi RTT integration (Sybil-resistant locality without trusting peer self-claims); geographic proof-of-work / IP-ASN binding (raises the cost of cell eclipse drastically); trusted-witness schemes. Estimated 6–10 weeks for an initial defense layer.

Issue 6: Heterogeneous-churn convergence. The benchmark suite uses uniform-rate churn. Real networks churn unevenly — corporate workdays, time-of-day peaks, regional outages. Mitigation: variable-churn benchmark inspired by Ghinita-Teo (alternate periods of high churn with periods of quiet, measure adaptation time at transitions); adaptive anneal cooling rate driven by locally-observed (μ, λ); liveness vs accuracy decoupled into independent threshold-triggered channels (Ghinita-Teo style); Patchwork-style background liveness probes (Tapestry 2004). Estimated 4–6 weeks.

Issue 7: Byzantine pub/sub mitigation. A malicious peer in the axon tree can drop, duplicate, or reorder publishes. Mitigations:

MaxDisjoint topic replication (Harvesf-Blough): replicate every axon-tree root at d disjoint locations so no single node or single S2-cell eclipse can silence a topic; Zipf-distributed publish workload benchmark + hot-axon-root migration (Makris et al.): threshold-triggered topic migration with DFE-style redirect at the original location, closing the gateway-concentration failure mode under content popularity. Estimated 6–8 weeks.

Issue 8: Asymmetric reachability and bilateral failure. The simulator assumes that if A connects to B, B can reach A on the same edge. Carrier-grade NAT, asymmetric firewalls, and asymmetric bandwidth break this. Every learning rule (LTP, hop caching, lateral spread, triadic) silently encodes the bilateral assumption. Mitigations: per-direction synapse fields (`forwardReachable`, `reverseReachable`, `forwardLatencyEMA`, `reverseLatencyEMA`, `asymmetryFlag`); direction-specific AP scoring; loop detection in iterative fallback; bidirectional eviction agency where both sides independently run admission. Estimated 4–6 weeks.

Tier 3: Operational Hardening

Issues 9–13. Known gaps that should be closed before scale-out but where staged rollout can manage the risk.

Issue 9: Identity provisioning and key rotation. Node identity is a 64-bit ID derived from a public key. The production system needs key generation, key storage, and key rotation policy — none of which the simulator addresses.

Issue 10: Storage durability. The DHT stores ephemeral routing state, but applications layered on top will store persistent data. The whitepaper covers routing only; the storage layer (replication, durability, garbage collection) is its own design problem.

Issue 11: Replay-cache semantics under load. The bounded ring buffer (default 100 messages per topic) is sized for typical workloads. High-rate topics can overflow the cache between subscriber refresh intervals, leaving gaps. Mitigation: dynamic cache sizing as a function of publish rate plus explicit "replay window expired" responses to subscribers.

Issue 12: Diagnostic observability. Per-node metrics, network-aggregate dashboards, pub/sub-specific metrics, alerting rules, diagnosis playbooks. The simulator instruments the counters; production needs the dashboards. Chapter VI is the production-grade specification.

Issue 13: Bootstrap rendezvous. New nodes need at least one known peer to start. The current simulator uses a fixed sponsor list; production needs a published rendezvous mechanism (DNS-based

seed list, on-chain DID registry, or similar). Standard peer-to-peer engineering; not a protocol design problem but a deployment one.

Total Estimated Work

To clear deploy blockers (Tier 1): 10–15 person-weeks after the bandwidth reclassification. Realistic timeline: 4–6 weeks with two engineers; 8–12 weeks with one. Issues 1, 2, 4 are independent and can be parallelized.

To clear all 13 issues: 30–50 person-weeks total (reduced from the pre-reclassification estimate of 65–90 person-weeks because the bandwidth Phase 3 mechanism build is no longer required). With staged rollout, Tier 2 and Tier 3 can be addressed during early production runs without blocking initial deployment.

What this means for the deployment plan. The audit's output is not "the protocol is unfit for production" but "the protocol is fit for staged production with the following extensions in flight." Chapter VI is the plan for the extensions.

Production Pathway

THE RED TEAM catalogues the gaps. This chapter sketches the extensions that close them: a friction-realistic simulator, learned locality via Vivaldi, adaptive parameter tuning, MaxDisjoint replication, hot-axon-root migration, and the deployment-staging plan that ties them together. Each extension is described at the level of "what it does and where it goes," with detailed implementation deferred to the engineering phase.

The Two-Layer API

BEFORE ANY OF THESE EXTENSIONS the codebase had to be unbound from its harness. A working DHT has two interfaces, not one: the *application* above wants to publish, subscribe, and look things up; the *network* below wants to open a channel, send a message, and notice when a peer goes silent. The protocol is the thing in between. N-DHT is now structured so the protocol code is genuinely between those two layers — neither the application nor the network is hard-wired into the routing logic — which is what makes the same body of code reusable in both the simulator and the deployed system.

The DHT contract: application-facing

The application sees one running node. The contract has eight verbs across three bands:

- **Lifecycle.** `start`, `stop`, `join(sponsor)`, `leave`.
- **Operations.** `lookup(targetKey)`, `subscribe(topic, h)`, `unsubscribe(sub)`, `publish(topic, payload)`.
- **Identity & observability.** `getNodeId`, `getSynaptome`, `getMetrics`, `onEvent(handler)`.

The *forbidden* methods are as instructive as the present ones. There is no method that enumerates "all nodes," no method that takes a

peer-id and returns that peer's state, no method that lets the application mutate the routing table. A DHT instance is responsible for one node's view of the network; the simulator's Engine creates many DHT instances and orchestrates them, but that orchestration is a simulator concern — it does not appear in the contract.

The Transport contract: network-facing

One Transport instance per running node; the protocol calls *into* it. Twelve methods in four bands:

- **Lifecycle.** `start(localNodeId)`, `stop()`, `getLocalNodeId()`.
- **Channel pool.** `openConnection(peerId)`, `closeConnection(peerId)`, `isConnected(peerId)`. This maps directly onto synaptome semantics: protocol admits peer to synaptome ⇒ `openConnection`; protocol evicts peer ⇒ `closeConnection`. The bilateral connection cap is enforced *inside* the transport — `openConnection` resolves with `false` if the remote refused. Connection-setup cost (WebRTC ICE/DTLS handshake in production, free in the simulator) is paid here, once per peer, not per message.
- **Messaging.** `send(peerId, type, payload)`, `notify(peerId, type, payload)`, `onRequest(type, h)`, `onNotification(type, h)`. Two outbound primitives: request/response and fire-and-forget. The split matters because production transports pay round-trip latency for `send` but not for `notify`. N-DHT uses `send` for the routing-tick chain (`lookup_step`, `lookahead_probe`, `find_closest_set`, `local_probe`) and `notify` for the LEARN side-effects (`reinforce`, `hop_cache`, `lateral_spread`, `triadic_introduce`). Canonical pattern for parallel probes: `Promise.allSettled` over `transport.send(...)` — a slow or dead peer in one probe should not fail the whole batch.
- **Liveness & latency.** `onPeerDied(handler)`, `getLatency(peerId)`. The transport runs a 1 Hz ping/pong heartbeat on every open channel; missed pongs (default 3-second timeout) trigger channel close and an `onPeerDied` event. The protocol consumes `getLatency` for AP scoring and registers an `onPeerDied` callback that populates a per-node `_deadPeers` set. The candidate-enumeration step in every routing tick filters against that set — the same mechanism a real deployment uses, retiring the legacy god's-eye `nodeMap.get(p)?.alive` check.

The Transport contract *forbids* the protocol from reaching around it: no `nodeMap.get(peerId)`, no `peer.synaptome.values()` reads, no cross-peer field access. Every cross-peer read happens via `send` or `notify`.

BootstrapService: cold start

A third, smaller contract. A new node starts with no peers; bootstrap is how it gets its first channel.

```
BootstrapService.bootstrap(sponsor)
  -> { sponsorId: bigint, transport: Transport }
```

The `sponsor` is a discriminated union: `{ kind:'simulator', sim, sponsorId }` for in-process pointers, `{ kind:'rendezvous', url, manifestSig }` for WebSocket signaling with a signed manifest, and `{ kind:'qr', sponsorAddr }` for direct device pairing. The contract returns the sponsor's id and a Transport with that one channel open. The DHT's `join(sponsor)` then runs a self-lookup through the sponsor and proceeds with stratified bootstrap on top of the freshly-opened transport.

Splitting BootstrapService out of Transport matters because production bootstrap is an out-of-band concern (signature verification, manifest fetching, signaling-server handshake) that the routing protocol shouldn't see. Once `bootstrap` returns, the DHT just has a transport with a channel open; it doesn't know whether that channel came from a QR code or a rendezvous server.

The simulator as deployment vehicle

The simulator's `SimulatedNetwork` and the production `WebRTCTransport` both implement the same Transport contract. Twelve methods, one signature each. The simulator runs handlers synchronously inside the call (in-process, no real RTT); production runs them through WebRTC data channels with real RTT. *The protocol code does not know which it is talking to.*

This is the property that lets the simulator be the deployment vehicle. Years of N-DHT benchmark numbers — the hop counts, latency distributions, pub/sub coverage, churn-resilience curves in Chapter **??** — transfer directly to production because the same code path runs in both worlds. The 25 K-node parity-gate benchmark (post-refactor sim v0.70.22 against the pre-refactor v0.70.04 reference) confirmed every protocol within a 10% target band: N-DHT within 5–8% on hops, 1–4% on latency; Kademlia, G-DHT, NX-17 within 0.5–3%. The protocol's central algorithm — the recursive-forwarding `lookup_step` chain — is free of cross-peer state reads; the remaining `nodeMap.get(peerId)` sites in the protocol code are all sim-only orchestration (Engine bookkeeping for node creation/destruction, the simulator's god's-eye stratified bootstrap, and the local self-resolution in `lookup(sourceId, ...)`).

Migration: fifteen commits

The codebase reached this shape over fifteen commits between sim v0.70.06 and v0.70.22. The first three introduced the contracts and converted Kademlia's iterative lookup to `await transport.send`. Commits four through nine walked N-DHT's routing primitives one at a time onto the Transport: hop-by-hop liveness via `onPeerDied`, two-hop lookahead via parallel `lookahead_probe` RPCs, dead-synapse eviction via `_localCandidate` over `local_probe`, the admission gate via `openConnection` and `getLatency`, and the LEARN side-effects via `notify`. Commit ten lifted the pub/sub primitives onto the Transport and made `AxonManager` async-aware while preserving its sync external API. Commit eleven was the structural endgame for the routing tick: the legacy source-orchestrated walk became a recursive-forwarding chain where each receiver runs the entire per-hop logic on its own local state and forwards via another `lookup_step`. Commits twelve and thirteen documented the deliberate scope decision to leave NX-15, NX-17, and NX-6 on the legacy path as research/comparison protocols. Commit fourteen retired `_nodeMapRef` and the last protocol-layer `nodeMap.get` sites. Commit fifteen implemented the DHT contract's observability surface: `getMetrics`, `getSynaptome`, `onEvent`, `snapshotMetrics`, with canonical event-emit sites at `peer-joined`, `peer-left`, `lookup-completed`, `dead-peer-detected`, `anneal-fired`, and `cycle-snapshot`.

What this gets us, operationally.

- The same body of routing code runs in the simulator and on the deployed mesh. No "production fork" of N-DHT; no separate code path for "real" networks.
- Async is the universal acid test. Every cross-peer interaction is async even in the simulator, so the recursive-forwarding lookup chain handles a dropped peer mid-walk (rejected Promise → `exhausted: true`) the same way in both worlds.
- Observability is contract-level, not back-channel. Smoke tests, the simulator's traffic distribution plot, and production load-balance dashboards all consume `getMetrics()` and `onEvent()`. Same fields, same event shapes.
- The production transport is one file. Swapping `SimulatedTransport` for `WebRTCTransport` is the deployment migration. The DHT contract, the protocol body, and `AxonManager` stay.

Friction-Realistic Simulator Extensions

The simulator is missing four classes of friction (Chapter IV). Each is closed by a parametrized extension to the simulator that produces production-realistic versions of the existing benchmarks.

Connection-setup time. Add `CONNECTION_SETUP_MS` (1,500–3,000 ms with a Gaussian distribution); new synapses enter a `PENDING` state on creation, excluded from AP scoring until setup elapses; ICE-failure rate of 10–15% is modeled as a Bernoulli flip per attempt with retry semantics. Acceptance criteria: Slice World re-stitching time degrades from $\sim$ 500 lookups to $\sim$ 5,000 lookups in the extension run, but cross-partition success holds at $\geq 85\%$.

RPC timeouts. Add `RPC_TIMEOUT_MS` (3,000 ms default); requests to dead peers stall for the timeout window before being declared dead. `routeMessage` is refactored into separate forward and reverse path tracing; reply may take a different path back. Acceptance criteria: pub/sub recovery from 5% instantaneous churn lengthens from 3 refresh intervals to 5–7, but final delivery still reaches 100%.

Bandwidth saturation. Per-node load tracking (`outgoingMsgRate`, `bandwidthCap`, `loadFactor`) becomes part of the AP score: effective delay = base_delay $\times$ (1 + msg_rate/bandwidth_cap), in the form Makris et al. describe. A relay at 50% load gets a 0.71$\times$ penalty in AP scoring; at 80%, 0.57$\times$; at 100%, 0.50$\times$. Smooth gradient away from saturated nodes without the binary "in/out" cliff that drives oscillation. Acceptance criteria: Zipf-distributed pub/sub at $\alpha = 1.0$ does not produce oscillation in any topic root's load over a 1-hour window.

Latency jitter. Add Gaussian noise to per-hop RTT, $\sigma = 0.30\times$ mean (calibrated to production RTT measurements). Verify that LTP-EMA does not oscillate; tune the EMA constant if it does. Acceptance criteria: lookup latency variance grows by $\sim$ 30% but mean is unchanged; LTP convergence on the optimal-path corpus does not slow by more than 20%.

Vivaldi-Style Learned Locality

The S2 prefix is structural, self-declared, and a Tier-2 risk. Vivaldi (Chapter I) replaces it with a learned synthetic coordinate that approximates RTT geometry.

The migration. Each peer maintains a 3D position vector + height. On every message exchange, the peer observes the actual RTT and applies a small spring-force update to its own coordinate. Confidence-weighted, decentralized, no coordinator needed. After convergence (typically $\sim$ 1,000 exchanges), pair-wise Euclidean distance

approximates pair-wise RTT.

The AP score's latency factor (equation 3) is then fed by the predicted Euclidean distance instead of the S2-derived haversine. Existing routing logic is unchanged; the *source* of the latency estimate changes.

Coordinate-as-prefix. The high bits of the node ID can be derived from the learned coordinate, replacing the S2 prefix. This is more involved (it requires a coordinate-to-bit-prefix discretization scheme) but it gives Sybil-resistant locality: an attacker that misclaims must also fake RTTs, which is expensive at scale.

Estimated work. 4–6 weeks for the basic Vivaldi implementation as a *latency-prediction layer* on top of the existing protocol; 4–8 additional weeks for the coordinate-as-prefix migration that retires the S2 dependence.

Adaptive Parameter Tuning (Metaplastic NH-1)

NH-1's twelve parameters are currently hand-picked. Following Ghinita-Teo (Chapter I), the parameters themselves can adapt based on local observation.

The local observables. Each peer maintains a rolling estimate of node-failure rate μ and node-join rate λ in its synaptome neighborhood. From these plus an estimate of network size (from successor-list density), the peer computes the probability that a given pointer's target has died and the probability that a new joiner now sits between this pointer and its ideal target.

The adaptive parameters. Annealing cooling rate, reheat amount, staleness threshold for synapse eviction, the LTP-lock window — all become functions of (μ, λ) rather than fixed constants. A peer in a stable region anneals slowly; a peer in a high-churn region anneals aggressively.

The QoS knob. The protocol exposes a single tunable knob: *target lookup-failure rate* P_f. Each peer adjusts its own thresholds to hit it. The operational interface is the target failure rate; the underlying parameters self-tune.

This is the natural endpoint of the path NH-1 lays out. The metaplastic NH-1 takes the consolidation a step further: not just collapsing eighteen rules into twelve, but eliminating the need to manually tune the twelve. The shape would converge on something like "one operator dial, twelve self-tuned parameters underneath."

Estimated work. 6–10 weeks for a working implementation; longer for the calibration and stability work to make the self-tuning actually produce the QoS-knob promise.

MaxDisjoint Replication

The pub/sub layer's residual concern is the Zipf-distributed case: a single popular topic's root saturating from publish volume. Two complementary mitigations are specified in the literature, and we adopt both.

MaxDisjoint topic replication (Harvesf-Blough P2P 2007). For topics whose subscriber count exceeds a threshold (say, 1,000 subscribers), replicate the root across $d = 3$ disjoint locations. Subscribers select their root by a deterministic hash of their own ID (load-balancing across replicas). Publishers send to all replicas in parallel.

The cost: $d\times$ the publish cost, but each replica handles $1/d$ of the subscriber base. The benefit: scales pub/sub linearly with replica count rather than concentrating on a single root. Architectural escape from the success-disaster trap.

Hot-axon-root migration (Makris et al. BIGDATA 2017, DFE pattern). When an axon root's load factor exceeds 0.8 for more than 60 seconds, the root *migrates* to a less-loaded peer in its synaptome neighborhood. The original root installs a redirect entry; new subscribes hitting the original root are redirected to the new one. After 30 seconds the redirect is dropped and the topic is fully migrated.

The redirect is the DFE — a "directory for exceptions" indicating that the canonical placement (publisher's S2 cell) has been overridden for hotspot mitigation.

Estimated work. 3–4 weeks for MaxDisjoint; 3–5 weeks for hot-axon-root migration; 2 weeks for end-to-end testing under a Zipf workload benchmark.

Deployment Staging

THE DEPLOY PLAN is staged. Each stage closes a specific concern; failures at each stage block the next.

Stage 0: simulator extensions. Implement the four friction-realistic extensions of §VI. Re-run the benchmark suite. The numbers from Chapter V will degrade — this is intentional, the new numbers are production-realistic. Any benchmark cell where the degradation is more than $\sim 30\%$ is investigated before progressing.

Stage 1: small-scale staging. Deploy a 5,000-node network on real hardware (cloud VMs, or a federated testbed of contributing partner nodes). Validate that the friction-realistic simulator predictions match. Tier-1 alerts (network partition, lookup collapse, pub/sub failure) below threshold for 30 days of operation.

Stage 2: medium-scale public. Deploy a 50,000-node public net-

work. Add MaxDisjoint replication for hot topics ($> 1{,}000$ subscribers). Add hot-axon-root migration. Run under realistic workload (i.e., real applications, not synthetic). Operability dashboards (Chapter VI) in production.

Stage 3: large-scale public + Vivaldi. Migrate to Vivaldi-style learned locality. Add adaptive parameter tuning (metaplastic NH-1). Validate at 250,000 and 1,000,000 nodes. The protocol at this stage is the "one-operator-dial, twelve-self-tuned-parameters" end state.

Total timeline. 4–6 months from clean checkout to Stage 1 with two engineers full-time. Stage 2 within 12–18 months. Stage 3 is a research direction, not a fixed schedule.

What the Deployment Will Look Like

The protocol's intended deployment shape is informed by the mission (Chapter I): a privacy-first peer-to-peer substrate carrying the routing and pub/sub workload of a decentralized internet. Three properties shape the deployment.

Browser-first. Most peers are browsers, running N-DHT in WebAssembly with JavaScript glue, communicating over WebRTC data channels. The bilateral cap is 50–100 connections. Average uptime is short; churn is high.

Server-augmented. A modest fraction ($\sim$ 5–15%) of peers are server-class: residential desktops with no NAT constraints, organizational infrastructure, contributing community nodes. The highway-percentage analysis (Chapter V) shows this is enough.

Federated bootstrap. New nodes need at least one known peer to start. The bootstrap rendezvous is a federated list of seed peers, published via DNS or on-chain DID registry. No central authority; multiple operators can publish seed lists.

The operability requirements that flow from this deployment shape are described in Chapter VI.

Operability

This chapter describes the operational surface a production deployment of N-DHT presents: what an operator measures, what they alert on, and what playbooks they run when something goes wrong. The simulator instruments most of the relevant counters already (Chapter IV); this chapter promotes them to operational dashboards and ties them to specific diagnostic procedures.

Per-Node Metrics

Each node exposes the following metrics, exported in the standard Prometheus / OpenTelemetry format and scraped by the operator's monitoring stack.

Routing metrics.

- `ndht_lookup_total` — counter; increments on every lookup originating at this node.
- `ndht_lookup_hops` — histogram; per-lookup hop count.
- `ndht_lookup_latency_ms` — histogram; per-lookup wall-clock RTT.
- `ndht_lookup_failures_total` — counter; failed lookups.
- `ndht_iterative_fallback_total` — counter; lookups that triggered iterative fallback.
- `ndht_synaptome_size` — gauge; current synaptome capacity used.
- `ndht_synaptome_evictions_total` — counter, labelled by mechanism (anneal / hop_cache / triadic / promote).

Bandwidth metrics.

- `ndht_msgs_sent_total` — counter, labelled by message type (`find_node` / `lookahead_probe` / `local_probe` / etc.).
- `ndht_msgs_received_total` — counter, labelled by type.

- `ndht_msgs_per_second` — derived rate (sender + receiver).
- `ndht_temperature` — gauge.

Pub/sub metrics.

- `ndht_pubsub_subscriptions` — gauge; active subscriptions originating here.
- `ndht_pubsub_published_total` — counter; publishes originating here.
- `ndht_pubsub_delivered_total` — counter; publishes delivered to this node as subscriber.
- `ndht_pubsub_dropped_total` — counter; publishes that hit a full replay cache.
- `ndht_axon_children` — gauge per topic the node serves as axon for; current direct-child count.

Network-Aggregate Dashboards

Operators need network-wide aggregates, not just per-node. Four top-level dashboards.

Routing health. P50, p95, p99 of lookup latency across the network. Lookup-success rate. Iterative-fallback rate. Anomaly detection: deviation from the rolling baseline. The operational target: p95 lookup latency $< 1.5\times$ rolling baseline, success rate $> 99.5\%$, iterative-fallback rate $< 5\%$. Sustained breach for > 5 minutes triggers an alert.

Bandwidth distribution. Gini coefficient of per-node message rates across the live population. Count of nodes exceeding $10\times$ and $100\times$ the network mean. Per-type breakdown. The operational target: Gini < 0.85 (above this is classical-DHT territory; if it climbs there, something has forced N-DHT into a degenerate regime). Hot $> 100\times$ count < 10 network-wide. Persistent breaches indicate either a parameter misconfiguration or a workload pattern that needs hot-axon-root migration to handle.

Pub/sub health. Per-topic delivery rate, subscriber count, axon tree depth. Replay-cache overflow rate. Per-topic churn (subscriber join/leave rate). The operational target: delivery rate $> 99\%$ on any topic with ≥ 100 subscribers; replay-cache overflow rate $< 1\%$.

Churn signal. Network-wide node-failure rate, node-join rate, average synaptome dead-peer count. The operational signal: churn rate above 5% per cycle is unusual; sustained periods of $> 10\%$ require investigation.

Alerting Rules

Critical alerts (page operator immediately).

1. *Network partition.* Sustained $> 5\%$ lookup failure rate across the network for ≥ 60 seconds. Diagnosis playbook: §VI.
2. *Lookup latency collapse.* P95 lookup latency more than $5\times$ rolling baseline. Indicates either a high-bandwidth-saturation event, a bug in the protocol, or an attacker.
3. *Pub/sub delivery failure.* Per-topic delivery rate drops below 90% on a high-traffic topic for ≥ 5 minutes. Diagnosis playbook: §VI.

Warning alerts (slack channel, follow up within 1 hour).

1. *Bandwidth concentration.* Hot $> 100\times$-mean node count exceeds 10 for $\geq$ 5 minutes. Indicates workload skew (Zipf-distributed pub/sub) requires hot-axon-root migration intervention.
2. *Synaptome thrashing.* Eviction rate per node above a threshold (operationally, > 1 eviction per second sustained). Indicates parameters need adjustment, or churn rate has spiked.
3. *Latency creep.* P95 lookup latency between $1.5\times$ and $5\times$ rolling baseline. Pre-collapse signal. Diagnosis playbook: §VI.

Diagnosis Playbooks

Three playbooks for the most-common operational failures. Each is a structured checklist: symptoms, diagnosis steps, likely causes, mitigations.

Playbook 1: Network partition

Symptom. Lookup failure rate $> 5\%$ for ≥ 60 seconds, distributed across many sources. Pub/sub delivery drops on multiple topics simultaneously.

Diagnosis steps.

1. Check the network-wide *churn signal* dashboard. Is node-failure rate elevated?
2. Check geographic distribution of failed lookups. Are failures concentrated in a single S2 region?
3. Check the bandwidth dashboard for max/mean ratio. Is one node carrying disproportionate traffic?

Likely causes. Real network outage in a region (cable cut, ISP failure); coordinated attack on a single S2 cell (eclipse); hot-axon-root saturation on a popular topic.

Mitigations. If outage: wait for restoration; the protocol's iterative fallback and hop caching will rebuild cross-region edges automatically (Slice World property). If attack: apply Tier-2 hardening (MaxDisjoint replication, prefix-binding). If pub/sub saturation: trigger manual hot-axon-root migration on the affected topic.

Playbook 2: Pub/sub delivery failure

Symptom. Per-topic delivery rate below 90% on a high-traffic topic.

Diagnosis steps.

1. Check the topic's axon tree depth. Has the tree grown deep (> 5 levels) due to overflow?
2. Check the topic's subscriber-count history. Has the subscriber count grown faster than the recruit-on-overflow rate can keep up?
3. Check the topic's replay-cache overflow rate.
4. Check the topic root's load factor.

Likely causes. Subscriber surge (viral content); replay-cache overflow on high-rate publish; root saturation.

Mitigations. For subscriber surge: ensure recruit-on-overflow is firing (not an obvious bug); manually trigger MaxDisjoint replication if available. For replay-cache overflow: increase `replayCacheSize` for the affected topic, or shorten cache time-to-live and instruct subscribers to expect window-expired responses. For root saturation: trigger hot-axon-root migration.

Playbook 3: Latency creep

Symptom. P95 lookup latency between 1.5× and 5× rolling baseline. Pre-collapse signal.

Diagnosis steps.

1. Check per-protocol per-region latency breakdowns. Is the creep concentrated regionally or network-wide?
2. Check bandwidth distribution. Is one node or a small cluster of nodes processing disproportionate traffic?
3. Check anneal-rate parameters; has someone recently changed `annealRateScale` or temperature defaults?
4. Check the overall churn signal.

Likely causes. Workload pattern shift (e.g., a new viral topic creating concentrated traffic); parameter misconfiguration; rising churn outpacing repair rate; transient saturation.

Mitigations. Identify the workload pattern via topic trends; trigger targeted hot-axon-root migration if a topic is the cause. Review recent parameter changes; revert if needed. If churn-driven, increase reheat threshold temporarily; the protocol will catch up.

The Production Readiness Checklist

Before Stage 1 deployment (small-scale staging, $\sim$ 5,000 nodes):

- Friction-realistic simulator (Tier 1 issues 1, 2, 4 closed)
- Full benchmark suite re-run; baseline established
- Per-node metrics exported and dashboards built
- All Tier-1 critical alerts wired up with paging
- Three diagnosis playbooks tested (failure injection)
- Bootstrap rendezvous mechanism deployed

Before Stage 2 deployment (medium-scale public, $\sim$ 50,000 nodes):

- MaxDisjoint replication implemented for hot topics
- Hot-axon-root migration implemented and tested
- Tier-2 hardening (Sybil resistance via Vivaldi or proof-of-location) at least partial
- Operational runbook documented; on-call rotation set
- Operator dashboards stable and alerting-ready

Before Stage 3 deployment (large-scale public, $\sim$ 250,000 nodes):

- Vivaldi locality (or other learned-locality) implemented
- Adaptive parameter tuning (metaplastic NH-1) deployed or in flight
- All Tier-2 issues addressed
- All Tier-3 hardening complete

The progression is the production pathway. The current N-DHT codebase, post-bandwidth-resolution, sits between "ready for Stage 0" (the simulator extensions) and "ready for Stage 1" (small-scale real-world). The work catalogued in Chapter VI is what closes that gap.

Part VII

Applications

Existing Applications: A Deep Dive

This chapter examines each of the dozen applications that depend on a DHT today. For each: what it does, the current DHT layer it uses, the performance profile of that layer, where it falls short of user expectations, and what changes if it migrates to N-DHT. The improvement projections are back-of-envelope from Chapter V's measurements, applied to the application's typical workload pattern.

Streaming-class applications — those that become possible with the latency budget N-DHT provides but that don't exist today — are out of scope for this volume. They are the subject of a separate forward-looking analysis.

BitTorrent (Mainline DHT)

What it does. BitTorrent's Mainline DHT is the trackerless peer-discovery layer for the BitTorrent file-sharing protocol. A torrent's metadata (the `info_hash`) is mapped into the DHT keyspace; peers participating in the swarm register themselves under that key; new peers query the DHT to discover the swarm.

Current implementation. Kademlia, 160-bit IDs, $K = 8$ buckets, $\alpha = 3$ parallel queries.[42]

Performance. Real-world measurements: lookup latency from cold start is typically 5–30 seconds for a `get_peers` query, dominated by the iterative α-parallel walk through random-ID-positioned peers spread globally. A single torrent's bootstrap can take 10–60 seconds before transfer begins. Mainline DHT's churn rate is high (median session length under one hour) and its routing tables drift continuously.

Where it falls short. The lookup latency is the most visible. Modern torrent clients hide it behind a tracker (a centralized fallback) for the first peer-discovery round; the DHT is used as a backup for tracker-resistance, not the primary path. The 5–30 second cold-start latency is unacceptable for a primary-path discovery system, and the trackers exist specifically to paper over it.

What N-DHT changes. Mainline DHT lookups translate di-

[42] The original BEP-0005 specification: `https://www.bittorrent.org/beps/bep_0005.html`. Mainline DHT is the largest production DHT in the world by node count, estimated at 10–20 million nodes at peak.

rectly to N-DHT lookups (the same operation, the same data model). Latency drops from 5–30 seconds to $\sim$ 250–300 ms at 25,000 nodes. Cold-start swarm discovery goes from a tracker-dependent round to a primary-path DHT round. Trackers become obsolete for peer discovery (they remain useful as analytics endpoints, but not as required infrastructure).

The qualitative shift: N-DHT makes the DHT *the* primary discovery layer, not a fallback. The 20–100$\times$ latency improvement collapses the user-visible delay below the threshold where tracker fallback adds value.

IPFS / Filecoin / libp2p

What it does. IPFS (the InterPlanetary File System) is a content-addressable peer-to-peer storage and retrieval protocol. Each piece of content is referenced by a hash (its content identifier, CID); the libp2p Kad-DHT routes lookups to peers that have announced "I have this CID." Filecoin is the incentive-aligned storage layer built on the same substrate.

Current implementation. libp2p Kad-DHT, a Kademlia variant. Default $K = 20$, $\alpha = 3$. Production deployment runs on tens of thousands of nodes; published numbers vary, but the IPFS team's measurements indicate cold-start CID lookup latencies in the 1–10 second range under good conditions and minutes-long failures under poor.[43]

[43] See the IPFS team's measurement studies and the libp2p Kad-DHT specification: `https://github.com/libp2p/specs/tree/master/kad-dht`.

Performance. CID lookup typically averages 1–5 seconds; provider-record retrieval (the second-stage operation that locates a peer holding the content) adds another 1–3 seconds. End-to-end first-byte-of-content latency from cold cache is in the 5–15 second range.

Where it falls short. The latency profile makes IPFS unviable as a CDN replacement. Centralized HTTP CDNs deliver first-byte latency in the 50–200 ms range; IPFS is 25–300$\times$ slower from cold cache. The IPFS gateway infrastructure exists to provide HTTP-fronted access for the common case, paying for the gap with centralized servers.

There is a separate problem: IPFS pub/sub (the `libp2p-pubsub` layer for IPNS updates and other mutable-name resolution) uses gossipsub, which is mesh-based and has its own scaling challenges.

What N-DHT changes. The DHT-layer lookups drop to $\sim$ 250–300 ms (Chapter V), comparable to a centralized HTTP CDN's first-byte for far-away content. End-to-end first-byte for IPFS becomes $\sim$ 500 ms–1 second — a 10–20$\times$ improvement. CDN-replacement viability becomes a real question rather than a hypothetical.

The pub/sub side: N-DHT's axonal trees would replace gossipsub for IPNS updates, with the load-distribution properties of Chapter V. The combined effect makes IPFS deployable on browser-class infras-

tructure for both storage discovery and mutable-name updates, which it currently is not.

Ethereum (discv5)

What it does. discv5 is Ethereum's peer-discovery layer — the system by which a new Ethereum node finds peers to join the network. It uses a Kademlia DHT variant for identity-aware routing, with TLS-like authenticated handshakes on top.

Current implementation. Kademlia base, with extensions for verifiable-identity ENRs (Ethereum Node Records). Lookup operations are similar to Mainline DHT but with stricter identity requirements.

Performance. Peer discovery from cold start is typically 30 seconds to several minutes for a new node to acquire enough peers to be operational. The latency is dominated by the Kademlia walk plus the per-peer ENR verification round trips.

Where it falls short. For a node joining the network on demand (a fresh validator coming online, a new MEV searcher), the multi-minute join time is operational friction. Mempool gossip via gossipsub adds another scaling dimension — transaction propagation times in the 10–50 ms range across the mempool require careful peer selection that the static Kademlia table cannot adapt to.

What N-DHT changes. Discovery latency drops by 10–50× for the DHT layer. The sponsor-chain join mechanism (§III) gives new nodes locality-aware initial peer sets, which matters more for Ethereum than for BitTorrent because mempool propagation is latency-sensitive.

For mempool: N-DHT's axonal trees are a natural fit for "broadcast a transaction to all interested validators in a region." The combination of locality-aware routing plus self-healing pub/sub trees would address the gossipsub scaling limitation directly.

Bitcoin Block / Transaction Propagation

What it does. Bitcoin nodes propagate blocks and transactions through a peer-to-peer mesh. The bootstrap is via DNS seeds; subsequent peer discovery is via the `addr` message exchange; propagation is via gossip.

Current implementation. Bitcoin does not use a DHT for peer discovery in the strict sense; it uses a flooded peer-table exchange and assumes that random-walk discovery will produce adequate coverage. Block and transaction propagation use a gossip protocol — "send to all peers, deduplicate via inventory exchange."

Performance. Block propagation across the global network aver-

ages 1–3 seconds for full blocks; *compact-block* propagation reduces this to a few hundred milliseconds for the announcement plus a re-request for missing transactions. Mempool transaction propagation latencies are in the 50–200 ms range across the network.

Where it falls short. The flooded gossip is not locality-aware: a transaction propagating from a US-east miner to an Asian node may take a long path through Europe. Compact-block propagation requires both peers to have the transactions in mempool already; for novel transactions (non-standard opcodes, large transactions) the propagation is slow. MEV concentrates around peers with low-latency access to miners.

What N-DHT changes. A locality-aware DHT for peer discovery solves the regional-clustering problem; nodes find nearby peers by default, transactions propagate along short-path edges. Compact-block propagation becomes *routed-pubsub* per chain: every node subscribes to the "block-announcement" topic; the publisher (miner) publishes; the axonal tree fans out with ~ 100 ms latency at 50,000 nodes. The structural advantage over Bitcoin's current gossip is that the tree is built once and reused, rather than the inventory exchange per peer per round.

The MEV concentration argument also changes: with locality-aware routing and pub/sub, peer-to-miner latency becomes a property of geography, not of peering relationships. The privileged-peer problem partially dissolves.

Tor Onion Services v3

What it does. Tor onion services (formerly hidden services) provide self-authenticating, location-hidden endpoints. The descriptor lookup (which Tor relays should be contacted to reach the service) is mediated by a hash ring of hashing servers, called the HSDir.

Current implementation. A Kademlia-like DHT, but with strong privacy constraints. Each onion-service descriptor is stored on six HSDir servers chosen from the consensus directory. The lookup latency is dominated by the multi-hop Tor circuit that wraps the DHT query, not by the DHT's own performance.

Performance. Onion-service rendezvous typically takes 5–15 seconds end-to-end, of which the Tor circuit setup dominates. The descriptor lookup itself adds maybe 1–3 seconds.

Where it falls short. The HSDir nodes are infrastructure-tier (operated by trusted directory authorities), which is a centralization concession. The descriptor caching is short-lived; popular services experience descriptor-fetch storms.

What N-DHT changes. The descriptor lookup itself becomes

faster ($\sim$ 300 ms versus 1–3 seconds), but the dominant cost is still the Tor circuit. The structural benefit is that the HSDir centralization can be reduced: N-DHT's load-distribution properties (Chapter V) mean that descriptor caching can be done at any node along the lookup path (the hop-caching mechanism), reducing the load on designated HSDir relays. The directory authority centralization is not removable without a different trust model, but the descriptor-fetch tier could be widened.

Storj / Sia (Decentralized Storage)

What it does. Storj and Sia are decentralized filecoin-like storage networks. Each user's file is sharded, encrypted, and replicated across a set of provider nodes; the DHT mediates shard placement and retrieval.

Current implementation. Storj uses a Kademlia-derived DHT for node-id and bucket discovery. Sia uses a chain-based discovery layer; the DHT plays a smaller role.

Performance. Storj's shard-retrieval latency is typically 200–800 ms for a 4 KB shard, dominated by the network connection to a remote storage provider rather than by the DHT lookup itself. The provider-discovery latency at network edge is $\sim$ 1 second.

Where it falls short. Locality-aware shard placement is the obvious next step. Storj's current placement does not account for the user's geographic location, so a user in Berlin may have shards stored in Sao Paulo. The retrieval pattern exposes the random-IDs latency tax.

What N-DHT changes. Locality-aware placement: shards are placed on providers in the user's S2 cell with high probability; retrieval latency drops to the regional-routing budget ($\sim$ 100–200 ms). The repair-coordination pub/sub channel (when a shard provider goes offline, the network needs to coordinate replacement) becomes an axonal-tree topic with 100% delivery under churn.

Tox / Briar (Peer-to-Peer Messengers)

What it does. Tox and Briar are peer-to-peer messengers; users communicate directly without any central server. Tox uses a Kademlia DHT for contact discovery ("Bob is online and reachable at this address"); Briar uses multiple transports including Bluetooth and a DHT-like overlay.

Current implementation. Tox: Kademlia, 32-byte public-key IDs as DHT keys. Briar: custom, hybrid local + overlay routing.

Performance. Tox contact-availability check takes 2–10 seconds

typically. Initial contact establishment can take longer if the peer is behind aggressive NAT.

Where it falls short. Group-chat membership is expensive: every message must be sent to every member, which makes large groups infeasible. Push notifications ("a contact came online") require periodic polling because the protocol has no proper pub/sub.

What N-DHT changes. Contact-discovery latency drops to $\sim 250\,\text{ms}$ (N-DHT's global lookup baseline), comparable to centralized messengers' latency for an offline-status check. Group chat becomes a topic in the axonal-tree pub/sub layer: 100% delivery to all subscribers with 5 ms-per-hop tree fan-out, scaling to thousands of group members with bounded cost per message. "Contact came online" becomes a pub/sub event delivered automatically. The combination is the first peer-to-peer messenger architecture that can scale to large groups with the latency profile of a centralized service.

YaCy (Decentralized Search)

What it does. YaCy is a peer-to-peer search engine. Each peer crawls a portion of the web; a custom DHT mediates shared indexing and query distribution.

Current implementation. A custom DHT layer roughly analogous to Kademlia but with index-specific extensions. Per-peer search latency varies widely; published numbers indicate 2–30 seconds for a query depending on word count and match coverage.

Performance. The latency cost is mostly index lookup plus query distribution, both of which scale with $\log N$ on top of a high constant (per-peer query takes 200–500 ms).

Where it falls short. Compared to centralized search (Google: $\sim 200\,\text{ms}$ median), YaCy is unusable for interactive search. The user-experience gap is what has kept YaCy a research curiosity rather than a competitive search engine.

What N-DHT changes. The DHT-layer query distribution drops to $\sim$ 250 ms; ranked-result aggregation across shards is a separate algorithmic problem, but the DHT is no longer the bottleneck. Combined with shard-level result caching via hop caching, an interactive YaCy-like search engine becomes plausible. Whether this is enough to replace Google is a different question (centralized search has decades of ranking infrastructure, not just speed); but the latency floor stops being the disqualifying factor.

Radicle (Decentralized Git Forge)

What it does. Radicle is a peer-to-peer git collaboration platform. Repositories are addressed by their peer-IDs; pull requests are routed via a custom protocol; the DHT layer handles repository discovery.

Current implementation. A custom DHT (gossiping libp2p underneath); $\sim$ 1,000-node production network.

Performance. Repository discovery: 1–3 seconds typical. Change-set propagation: depends on workflow but typically 5–30 seconds for a new commit to be visible to all interested watchers.

Where it falls short. Discovery latency makes the "find a project, browse it" workflow feel slow compared to GitHub. Change propagation latency makes "watch this repo for new commits" poll-based rather than push-based.

What N-DHT changes. Repository-discovery latency drops to $\sim$ 300 ms. Change-set propagation becomes an axonal-tree topic ("commits-on-this-repo"), with $\sim$ 200 ms fan-out to all subscribers. The platform becomes interactive in the GitHub-like sense, not just browsable.

Handshake / ENS (Decentralized DNS)

What it does. Handshake and ENS provide decentralized name resolution. Domain ownership is recorded on a blockchain; a DHT-like layer caches recent lookups for performance.

Current implementation. ENS uses Ethereum's discv5 plus a separate caching tier. Handshake uses a custom Kademlia-like DHT for cache lookups.

Performance. Cached lookups are fast (under 100 ms when hot); cold lookups go to the underlying chain and take seconds to minutes. The cache hit rate is the dominant performance variable.

Where it falls short. Cold-cache latency makes first-time visits to a domain slow. The cache tier is itself operated by infrastructure peers (a centralization tradeoff) because random-Kademlia caching does not scale.

What N-DHT changes. Hop caching makes the "cache tier" implicit: every routed lookup deposits the result at intermediate nodes; popular domains accumulate shortcuts naturally; the explicit caching infrastructure becomes unnecessary. Cold-cache latency drops because the routing itself is faster.

WebTorrent / WebRTC P2P

What it does. WebTorrent is BitTorrent for browsers, running on WebRTC data channels. It uses a bridge to Mainline DHT for peer discovery (because most browsers cannot speak the Mainline DHT protocol directly).

Current implementation. A Mainline-DHT-bridge plus WebRTC overlay. The bridge introduces a centralization point and adds latency.

Performance. Peer discovery: 5–30 seconds for a new torrent (the same as Mainline DHT, plus bridge overhead). Browser-imposed limits restrict the swarm size and the WebRTC connection count.

Where it falls short. The Mainline-DHT bridge is a centralization concession driven entirely by the protocol's inability to run in browsers. The bridge operators are a single point of failure and surveillance.

What N-DHT changes. N-DHT *is* a browser-native DHT — it was designed for the WebRTC environment from the start. WebTorrent on N-DHT eliminates the bridge: peer discovery is direct, latency is the regional-routing budget (100–300 ms), and the browser becomes a first-class peer-to-peer participant. The connection-cap analysis (Chapter V) shows this is sustainable on browser-class infrastructure with the highway-percent augmentation.

Mastodon-Style Federations (No DHT Today)

What it does. Mastodon (and the broader ActivityPub fediverse) is a federation of independent servers. Users follow accounts on other servers; the home server fetches and serves the federated content.

Current implementation. No DHT. Discovery is via WebFinger (a centralized handle resolution protocol per domain). Cross-server delivery is via direct HTTP push from publisher's home server to each subscriber's home server.

Performance. Per-user latency depends on home-server performance; typical Mastodon home servers run on small VPS hardware and exhibit 200–2,000 ms response times under load. Cross-server federation can lag by minutes for popular posts due to the per-subscriber-server fan-out.

Where it falls short. The home-server-as-bottleneck pattern: a viral post on a small instance can take down the whole instance because every other instance's followers must fetch from the publisher's home. Discovery is brittle: WebFinger queries hit the home server on every cold-cache lookup.

What N-DHT changes. A proposed N-DHT layer underneath the fediverse would decentralize both discovery and delivery. User-

handle resolution becomes an N-DHT lookup (independent of home-server load). Status-update delivery becomes an axonal-tree topic per follower-graph cluster: a viral post fans out via the tree at a bounded per-relay cost, never saturating any single home server. Home servers continue to provide durable storage and the WebFinger origin authority, but the fan-out load moves to the DHT layer.

This would make Mastodon-style federations *actually* federation-friendly: small servers stop being single points of failure for their users' follower-base reach.

Summary of Improvements

Application	**Current bottleneck**	**N-DHT improvement**
BitTorrent (Mainline)	5–30 s cold lookup	$\sim$ 300 ms; primary path possible
IPFS / Filecoin	5–15 s first-byte	$\sim$ 1 s; CDN-replacement viable
Ethereum discv5	Multi-minute join, mempool latency	$\sim$ 1 s join, locality-aware mempool
Bitcoin propagation	Region-blind, MEV-prone	Locality-aware, axonal-tree announce
Tor onion services	DHT minor; HSDir centralization	Wider HSDir tier, faster lookups
Storj / Sia	Region-blind shard placement	Locality-aware shards + repair pubsub
Tox / Briar messengers	Slow group chat, polling status	Pubsub group chat, push status
YaCy	5–30 s queries	$\sim$ 300 ms DHT layer; interactive plausible
Radicle	5–30 s changeset propagation	$\sim$ 200 ms via axonal pub/sub
Handshake / ENS	Cold-cache slow	Implicit caching via hop caching
WebTorrent	Mainline-DHT bridge	Bridge eliminated; browser-native
Mastodon-style federation	Home-server bottleneck	Decentralized fan-out via axonal trees

Table 11: Cross-application summary of N-DHT improvements. Latency numbers in this table are projections based on Chapter V; production deployments will need the friction-realistic simulator extensions (Chapter VI) to confirm.

The pattern across applications: *the DHT layer is rarely the only bottleneck, but it is consistently a bottleneck*, and moving to N-DHT consistently moves it out of the way of the user-visible latency. The applications that get the most benefit are the interactive ones (messengers, search, dev forges) where the DHT-layer latency is in the user's critical path. The applications that get the least benefit are the ones where the DHT is already a small fraction of total latency (Tor, Storj retrieval).

The applications that become *architecturally different* on N-DHT are the federation patterns (Mastodon-style), because N-DHT's pub/sub primitive replaces the home-server fan-out that those patterns suffer under. This is the closest this chapter comes to a forward-looking claim, and we keep it modest: the architecture is enabled, not necessarily preferable in every dimension.

Part VIII

Reference

Future Work

THE FUTURE-WORK AGENDA follows the red-team prioritization (Chapter VI) and integrates the comparative-literature mechanisms identified in Chapter I. We distinguish *near-term* (work required for production deployment), *medium-term* (extensions that improve the protocol but are not blocking), and *long-term* (research directions that change the protocol's operating model).

Near-Term: The Production Pathway

The work that turns NH-1 from a simulator-validated protocol into a production system. Detailed plans in Chapter VI; itemized here for completeness.

Friction-realistic simulator. Connection setup time (`CONNECTION_SETUP_MS`) with new synapses in a PENDING state until setup elapses; RPC timeouts (`RPC_TIMEOUT_MS`) for silently-dropped peers, with request/reply path tracing; load-dependent AP scoring (Makris-style); Gaussian jitter injection on every hop. The resulting measurements are production-realistic versions of the results in Chapter V.

Operational hardening. Per-node metrics export, network aggregate dashboards, alerting rules, the three diagnosis playbooks, the production readiness checklist. Most of this is implementation work; the design is in Chapter VI.

Bootstrap rendezvous. Federated DNS-based seed lists plus on-chain DID registry as the primary discovery mechanism for new nodes. Standard peer-to-peer engineering, not a protocol design issue, but it must exist before deployment.

Medium-Term: Architectural Extensions

Extensions that improve specific properties of the protocol without changing its operating model.

Vivaldi-style learned locality. Replace the self-declared S2 prefix with synthetic Euclidean coordinates (Dabek et al., SIGCOMM

2004) that converge from observed RTTs. The migration path is described in §VI: a Vivaldi-style coordinate system slots in under the AP score's latency factor without disturbing the rest of the protocol. Closes the cell-eclipse gap (Chapter II) by removing the cooperative-trust assumption: an attacker cannot falsify a coordinate that is verified against measured RTT.

Byzantine pub/sub via MaxDisjoint replication. Replicate each axon-tree root at d disjoint locations using Harvesf-Blough's placement formula. With $d = 8$ replicas this delivers 90% pub/sub success at 50% malicious nodes per their measurement. Combined with surrogate routing on the multicast tree, this closes the Byzantine-resistance gap without abandoning the axonal-tree primitive.

Adaptive synaptome capacity. The current synaptome is fixed at 50. A dynamically-sized synaptome — bumping capacity during join-heavy periods, pruning in steady state — is an engineering refinement not yet attempted. The mechanism is straightforward (capacity is just a parameter); the question is what trigger to use and how to ensure capacity changes don't disturb the steady state's behavior.

Global-pool annealing. Currently, annealing's 2-hop neighborhood scan is the entire candidate pool. A periodic "replace the lowest-vitality synapse with a globally sampled candidate" would close the gap between the bootstrap-synaptome and the omniscient-synaptome (Chapter V). Tradeoff: more aggressive exploration costs more bandwidth (the bandwidth question of Chapter V reappears, though now with the `annealRateScale` knob to tune it).

Long-Term: Metaplastic NH-1

The natural endpoint of the path NH-1 lays out: a protocol where the parameters *themselves* self-tune. The brain analogue is metaplasticity — "plasticity of plasticity" — in which the rules governing learning change based on the brain's activity level (Abraham & Bear, 1996). A neuron that has been very active becomes harder to strengthen further (otherwise everything saturates). A neuron that has been quiet becomes easier. The learning rules adapt.

NH-1's parameters — decay rate, protection window, exploration rate — are currently hand-picked constants. A metaplastic version would let the network self-tune them based on local conditions:

- Each peer maintains rolling estimates of local failure rate μ and join rate λ (Ghinita-Teo style).
- Annealing cooling rate, reheat amount, and the staleness threshold for synapse eviction become functions of (μ, λ) rather than fixed constants.

- A peer in a stable region adapts slowly; a peer in a high-churn region adapts aggressively.
- The user / operator sets one knob: target lookup failure rate P_f. The network self-tunes the underlying constants.

This is the next layer of brain-inspired self-organization, and the natural endpoint of the path the protocol lays out. The deployment would still be NH-1's twelve-rule architecture; the operational interface would shrink from twelve parameters to one.

Larger-Scale Evaluation

The current benchmark evaluation tops out at 50,000 nodes. We expect the protocol's qualitative behavior to extend to larger scales but have not measured. Specific sizes worth running:

- 100,000 nodes — the next doubling. Bootstrap behavior is visibly different at 100,000 in earlier-generation work; we should measure NH-1 there.
- 250,000 nodes — the production target's lower bound. A serious deployment should expect to scale here.
- 1,000,000 nodes — the production target's aspirational upper bound. We have not run NH-1 at this scale; the simulator can run it (with patience), but the analyst time to drive it has not been allocated.

The hypothesis to test at each scale: the latency multiplier on the 3δ floor stays roughly constant; the Gini coefficient of the per-node bandwidth distribution does not climb; the hot-> 100×-mean count remains zero. If any of these hypotheses fails, the protocol's claim to "scale-invariant" behavior is challenged and an architectural review is required.

Open Research Directions

Beyond the medium- and long-term extensions, three open research questions worth raising.

Proof-of-location. The S2 prefix is currently cooperative trust. A verifiable location primitive (a TURN-server handshake, an ASN-bound IP attestation, RTT-triangulation via trusted witnesses) would prevent prefix forgery. Each candidate has tradeoffs; none has been deeply investigated by this project. We catalog them as a research direction rather than a planned extension.

Privacy under DHT routing. The protocol's routing discloses "the source is asking for this target" to every node along the path.

Tor solves this with onion encryption at high latency cost; N-DHT does not address it directly. A privacy extension — onion-style relay encryption, or Garlic-routing-style multi-message encryption, or a pure information-theoretic technique like PIR — is a research question larger than this protocol's scope.

Federation across non-uniform networks. The protocol assumes a single network with consistent S2 cell space. In a real-world deployment, multiple operators might run separate N-DHT networks (bridged through specific peers). The federation semantics — how lookup requests cross between networks, what identity-resolution model spans them — is not currently specified.

The Through-Line

The work catalogued in this chapter falls into three groups, and the groups have a natural sequence.

The *near-term* group is engineering: implement the simulator extensions, set up the operational tooling, finish the production-readiness checklist. The protocol design is fixed; the work is to make it deployable.

The *medium-term* group is principled extension: Vivaldi locality, Byzantine pub/sub, adaptive synaptome capacity. Each can be evaluated against the existing benchmark suite; each adds a measurable property to the protocol's repertoire.

The *long-term* group is metaplastic: the protocol learns to tune itself, the operator's interface shrinks to one knob, the protocol becomes *self-administering*. This is the direction the consolidation work in NH-1 was always pointing toward, and it is the direction the deployment trajectory in Chapter VI is preparing for.

The mission — a privacy-first, decentralized internet substrate — is realized progressively, not in a single release. Each stage of this work shrinks the gap between what the protocol does and what the mission requires.

References

Foundational DHT Work

- Maymounkov, P., & Mazières, D. "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric." *Proceedings of IPTPS*, 2002.
- Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. "Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications." *SIGCOMM*, 2001.
- Ratnasamy, S., Francis, P., Handley, M., Karp, R., & Schenker, S. "A Scalable Content-Addressable Network." *SIGCOMM*, 2001.
- Rowstron, A., & Druschel, P. "Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems." *Middleware*, 2001.
- Zhao, B. Y., Huang, L., Stribling, J., Rhea, S. C., Joseph, A. D., & Kubiatowicz, J. D. "Tapestry: A Resilient Global-Scale Overlay for Service Deployment." *IEEE JSAC* 22(1), 2004.

Latency-Aware DHTs

- Dabek, F., Li, J., Sit, E., Robertson, J., Kaashoek, M. F., & Morris, R. "Designing a DHT for low latency and high throughput." *NSDI*, 2004.
- Castro, M., Druschel, P., Kermarrec, A., & Rowstron, A. "SCRIBE: A large-scale and decentralized application-level multicast infrastructure." *IEEE JSAC*, 2002.
- Freedman, M. J., Freudenthal, E., & Mazières, D. "Democratizing Content Publication with Coral." *NSDI*, 2004.

Adaptive Coordinates

- Dabek, F., Cox, R., Kaashoek, M. F., & Morris, R. "Vivaldi: A Decentralized Network Coordinate System." *SIGCOMM*, 2004.

- Ledlie, J., Gardner, P., & Seltzer, M. "Network Coordinates in the Wild." *NSDI*, 2007.

Byzantine Resistance and Route Diversity

- Castro, M., Druschel, P., Ganesh, A., Rowstron, A., & Wallach, D. S. "Secure routing for structured peer-to-peer overlay networks." *OSDI*, 2002.
- Harvesf, C., & Blough, D. M. "The Effect of Replica Placement on Routing Robustness in Distributed Hash Tables." *IEEE P2P*, 2007.

Load Balancing and Hotspot Mitigation

- Makris, C., Tserpes, K., & Anagnostopoulos, D. "A novel DHT placement strategy for hot data using directory of exceptions." *IEEE Big Data*, 2017.
- Godfrey, B., Lakshminarayanan, K., Surana, S., Karp, R., & Stoica, I. "Load Balancing in Dynamic Structured P2P Systems." *INFOCOM*, 2004.

Adaptive Maintenance and Churn Handling

- Ghinita, G., & Teo, Y. M. "An Adaptive Stabilization Framework for Distributed Hash Tables." *IPDPS*, 2006.
- Saroiu, S., Gummadi, P. K., & Gribble, S. D. "A Measurement Study of Peer-to-Peer File Sharing Systems." *MMCN*, 2002.

Pub/Sub Substrates

- Castro, M., Druschel, P., Kermarrec, A.-M., Nandi, A., Rowstron, A., & Singh, A. "SplitStream: High-bandwidth multicast in cooperative environments." *SOSP*, 2003.
- Ratnasamy, S., Handley, M., Karp, R., & Shenker, S. "Application-level multicast using content-addressable networks." *NGC*, 2001.

Substrate: Learning, Decay, Pruning

- Hebb, D. O. *The Organization of Behavior.* Wiley, 1949.
- Bliss, T. V. P., & Lømo, T. "Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path." *Journal of Physiology* 232, 1973.

- Frey, U., & Morris, R. G. M. "Synaptic tagging and long-term potentiation." *Nature* 385(6616), 1997.
- Abraham, W. C., & Bear, M. F. "Metaplasticity: the plasticity of synaptic plasticity." *Trends in Neurosciences* 19(4), 1996.
- Drachman, D. A. "Do we have brain to spare?" *Neurology* 64(12), 2004.

Geometry and Geography

- Google. "S2 Geometry Library." `https://s2geometry.io/`, 2011.
- Hilbert, D. "Über die stetige Abbildung einer Linie auf ein Flächenstück." *Mathematische Annalen* 38(3), 1891.

NX-1 to NX-17: Detailed Evolution

THIS APPENDIX traces the experimental sequence that produced NH-1. Each NX generation either added a mechanism that survived consolidation, added a mechanism that was later retired, or refactored an existing mechanism into a cleaner form. The list below is concise; the full simulator codebase preserves each generation under `src/dht/neuromorphic/`, and Appendix B's parameter tables include the configurations used.

Capsule Per Generation

NX-1 (the routing-table Hebbian baseline). Single-tier synaptome with weighted entries. LTP increments weights on successful paths. Fixed-rate decay. Bootstrap by random sampling. *Established that simple Hebbian reinforcement on a routing table improves over Kademlia.*

NX-3 (geographic three-layer initialization). Introduced geographic-prefix node IDs (G-DHT). Bootstrap allocates entries across three layers: intra-cell local, inter-cell structured (one per geographic-prefix bucket), random global. *Locality became part of the protocol.*

NX-4 (iterative fallback). When forward XOR-progress candidates are exhausted, take the closest unvisited peer regardless of strict progress. Enables routing through partitions and around routing-table gaps. *The protocol stopped failing on partitioned topologies.*

NX-5 (stratified bootstrap and incoming-synapse promotion). Bootstrap allocates entries with explicit per-stratum floors; new node coverage of every XOR-distance bucket is guaranteed. Incoming synapses (passive learning) added: peers that reach this node are recorded and promoted on use. *The synaptome population became routing-quality balanced from the start.*

NX-6 (churn-resilient routing). Temperature reheat on dead-peer detection. Dead-synapse eviction and replacement. Bootstrap-preference in the eviction gate. *Churn recovery became event-triggered,*

eliminating the need for periodic stabilization.

NX-7 (dendritic pub/sub v1). First attempt at pub/sub: 25% peel-off relay tree. *Worked, but the* 25% *threshold was an ad-hoc choice; tree balance was uneven.*

NX-8 (dendritic pub/sub v2). Balanced binary-split relay tree. *Better balance but the splitting logic was brittle under churn.*

NX-9 (geographic dendritic pub/sub). S2-clustered relay tree with direct 1-hop delivery. *Locality benefit on pub/sub, but the tree was still fragile.*

NX-10 (routing-topology forwarding tree). Pub/sub tree follows the routing topology directly: every first-hop synapse is a forwarder. Strong load distribution, but tree was rigid. *The routing topology and the pub/sub topology should be the same thing.*

NX-11 (diversified bootstrap + axonal pub/sub). 80/20 stratified/random bootstrap. Axonal-tree pub/sub. The first generation that combined good routing with good pub/sub in a unified architecture.

NX-12 (load-tracked AP scoring). Experimental load-aware AP scoring, where AP factor includes a load penalty. *Did not survive ablation: at the scales tested, the load penalty was either redundant (vitality already preferred under-used peers) or destabilizing (oscillation when the load measure was noisy). Retired.*

NX-13 (G-DHT three-layer init refinement). Refined three-layer bootstrap. Mostly a parameter sweep over NX-11.

NX-15 (AxonManager abstraction). Introduced the `AxonManager` abstraction over pub/sub state, separating the topic-membership logic from the routing protocol. Enabled pluggable pub/sub policies (recruit-on-overflow, relay selection).

NX-17 (publisher-prefix topic IDs and routed axonal trees). Final pre-NH-1 generation. Topic IDs include the publisher's S2 cell prefix; pub/sub uses pure routed-tree mode (no K-closest replication). Eighteen rules and forty-four parameters. *State-of-the-art performance, but the rule set was at the edge of comprehensibility.*

NH-1 (the consolidation). Audit of NX-17. Six rules absorbed or retired (highway tier into vitality; stratum-floor into bilateral cap; Markov pre-learning into triadic; load-tracked AP retired; relay-pinning into standard recruit; `weightScale` into removal). Twelve rules remain; twelve parameters; one equation (vitality) governing admissions. *Same behavior as NX-17 within* 5–15% *on every benchmark; substantially more maintainable.*

Lessons from the Sequence

Three patterns are visible across the NX-1 → NH-1 progression.

Most generations added one mechanism. Each was ablation-tested before being committed; mechanisms that didn't survive ablation were either reverted or marked for retirement at the next consolidation.

The biggest gains came from removing things, not adding them. The NX-17 → NH-1 audit removed six rules and thirty-two parameters with essentially no behavioral cost. The "learning" is in part learning what *not* to keep.

Pub/sub took five generations to get right (NX-7 through NX-11). Each iteration tried a different tree topology; the winning answer (axonal trees rooted at publisher-prefix topic IDs, with self-healing via routed re-subscribe) emerged only after four failed attempts. This is normal in distributed-systems research; it is documented here so that future readers understand why the current design is the way it is.

Full Parameter Tables

THIS APPENDIX consolidates every constant in the NH-1 implementation: the twelve tunable parameters from Chapter III (recapitulated here for one-stop reference), the routing-layer fixed constants, the pub/sub operational tunables, and the simulator-side configuration.

Tunable Parameters

The twelve operational parameters of NH-1 (also listed in Table 2):

Parameter	**Default**	**Legal range**	**Effect**
`maxSynaptome`	50	20–256	Capacity tradeoff vs WebRTC connection budget
`lookaheadAlpha`	5	1–10	2-hop probe breadth; cost is per-hop network traffic
`maxHops`	40	20–100	Hard ceiling; rarely reached in practice
`epsilon`	0.05	0–0.20	First-hop random-choice probability
`annealCooling`	0.9997	0.99–0.9999	Per-hop temperature decay rate
`annealRateScale`	1.0	0–1.0	Master volume on annealing (the bandwidth knob)
`decayGamma`	0.995	0.99–0.9999	Per-decay-tick weight multiplier
`vitalityFloor`	0.05	0–0.5	Below this, freely replaceable regardless of recency
`inertiaDuration`	20	5–100	LTP-lock window; protects fresh reinforcements
`promoteThreshold`	2	1–10	Incoming-synapse promotion trigger
`triadicThreshold`	2	1–10	Triadic-closure firing threshold
`recencyHalfLife`	50	20–200	Tick half-life of post-inertia recency decay

Routing-Layer Fixed Constants

These are not intended for routine tuning; they are implementation choices documented here for completeness.

- $\mathrm{T\_INIT} = 1.0$ — initial node temperature.
- $\mathrm{T\_MIN} = 0.05$ — temperature floor.
- $\mathrm{T\_REHEAT} = 0.5$ — temperature on dead-peer detection.
- $\mathrm{DECAY\_INTERVAL} = 100$ — lookups per decay tick.

- ANNEAL_LOCAL_SAMPLE = 50 — per-call cap on `_localCandidate` probes.
- GEO_BITS = 8 — S2 cell prefix width.
- GEO_REGION_BITS = 4 — granularity for lateral spread.
- STRATA_GROUPS = 16 — bucketing for stratum-diversity logic.
- LATERAL_K = 2 — lateral-spread fanout.

Pub/Sub Operational Tunables

Five parameters governing the axonal-tree pub/sub layer:

- `maxDirectSubs` = 32 — max direct children per axon before recruit-on-overflow fires.
- `minDirectSubs` = 4 — min direct children before relinquishing role.
- `refreshIntervalMs` = 30000 — per-member refresh interval.
- `maxSubscriptionAgeMs` = 90000 — TTL for child entries before sweep.
- `replayCacheSize` = 100 — ring buffer messages per axon-role.

Simulator Configuration

The simulator's per-run configuration:

- `nodeCount` — total nodes (typically 5000–50000 in this volume).
- `maxConnections` = 100 — bilateral cap.
- `highwayPct` = 0 — fraction of server-class peers (default 0% uniform browser-class).
- `webLimit` = true — enforce bilateral cap.
- `lookupsPerCell` = 500 — benchmark sample size per workload type.
- `warmupSessions` — per-protocol warmup before measurement (variable; see methodology).
- `churnRate` — percentage replacement per cycle in churn tests.
- `nodeDelayMs` = 10 — per-hop processing constant.

Extended Glossary

THIS APPENDIX repeats the glossary from Chapter I, extended with cross-references to source code symbols and chapter sections. A reader who knows what they are looking for and wants to find it in the codebase or in the chapter where the concept is fully developed should start here.

The structure: term, one-line definition, source location (implementation symbol or chapter / section), and forward references where relevant.

addByVitality The unified admission gate for synapses. Implementation: `NeuromorphicDHTNH1._addByVitality()`. Algorithm in Chapter III.

annealRateScale The bandwidth-tuning master volume on annealing. Implementation: `r.annealRateScale` in NH-1 constructor; default 1.0. Knee analysis in Chapter V.

AP scoring The hop-selection mechanism. Equation 3. Implementation: `NeuronNode.bestByAP()` for single-hop; `NeuromorphicDHTNH1._bestByTwoHopAP()` for two-hop lookahead. Discussion in Chapter II.

axon A node serving as a relay for a pub/sub topic. Implementation: `AxonManager` per node, with a `role` record per topic. Chapter II.

axonal tree The complete delivery structure for a topic. Built by `routeMessage` subscribes; healed by periodic re-subscribe. Chapter III.

bilateral cap Bilateral connection-cap enforcement. Implementation: `node.tryConnect(peer)` returns false if either side at capacity. Chapter IV.

Dabek 3δ floor The analytical lower bound on lookup latency in any recursive DHT. Calibrated in methodology, Chapter V.

decay Weight erosion via multiplicative γ. Implementation: `_tickDecay()`. Chapter II (FORGET).

Gini coefficient Per-node bandwidth distribution inequality measure. Definition in Chapter I; analysis in Chapter V.

hop caching Every intermediate node along a successful path remembers the destination. Implementation: `_hopCache()`. Chapter II (learn).

incoming synapse A passive learning record of peers that have reached this node. Implementation: `node.incomingSynapses`. Chapter II.

inertia The protection window after a synapse is reinforced. INERTIA_DURATION = 20 ticks. Chapter II.

iterative fallback Routing when forward XOR progress is exhausted. Implementation: scan unvisited synapses, take closest. Chapter II.

k-bucket Kademlia distance-bucket structure. Bucket-organized at bootstrap; entries migrate via traffic. Chapter I.

lateral spread Hop-cache propagation to geographic neighbors. Implementation: `_hopCache()` recursion at depth 1. Chapter II.

LTP / LTD Long-term potentiation / depression. The biological basis for weight reinforcement and decay. Chapter II.

N-DHT Generic family name for the neuromorphic DHT. Chapter I.

NH-1 The current generation; consolidation of NX-17. Twelve rules, twelve parameters. Chapter III.

node ID 64-bit identifier. High 8 bits = S2 cell prefix; low 56 bits = H_{56}(public key). Chapter II.

publisher-prefix topic ID Topic ID with high 8 bits set to publisher's S2 prefix. Chapter II.

recency Exponential-decay multiplier in vitality. Within inertia: 1.0. Outside: $\max(0.1, e^{-\Delta t/\tau})$. Chapter II.

replay cache Bounded ring buffer of recent publishes per topic. Chapter II.

S2 cell prefix Geographic encoding via Hilbert curve. geoBits = 8 default. Chapter II.

Slice World Single-bridge partition recovery test. Implementation: `src/dht/sliceWorld.js`. Chapter IV.

STRATIFIED BOOTSTRAP Bucket-balanced bootstrap with explicit geographic-strata fill. Implementation: `NeuromorphicDHTNH1.buildRoutingTables()`. Chapter II.

STRATUM An XOR-distance bucket; equivalent to "Kademlia bucket index."

SYNAPSE Single peer entry in the synaptome. Implementation: `Synapse` class. Chapter II.

SYNAPTOME Routing table; the set of synapses a node maintains. Implementation: `node.synaptome`.

TEMPERATURE Per-node scalar governing annealing trigger probability. Cools by ANNEAL_COOLING; reheats on dead-peer detection. Chapter II.

TRIADIC CLOSURE Network-level coincidence detection: same transit pair $\rightarrow$ direct introduction. Chapter II (LEARN).

TWO-HOP LOOKAHEAD AP-scoring extension that explores two hops of candidates. Implementation: `_bestByTwoHopAP()`. Chapter II.

VITALITY WEIGHT $\times$ RECENCY. The single equation governing every admission decision. Equation 1. Chapter II.

WEIGHT Strengthening factor in $[0, 1]$. Reinforced by LTP, decayed multiplicatively. Chapter II.